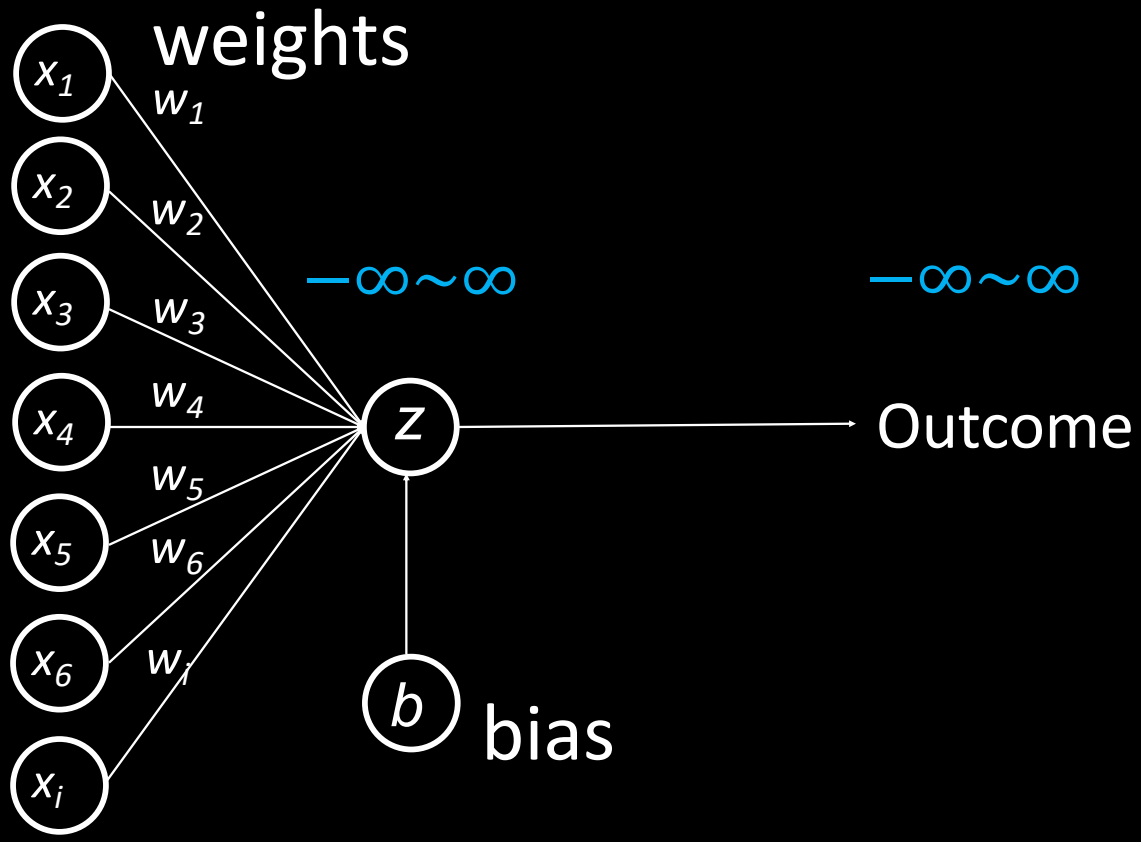
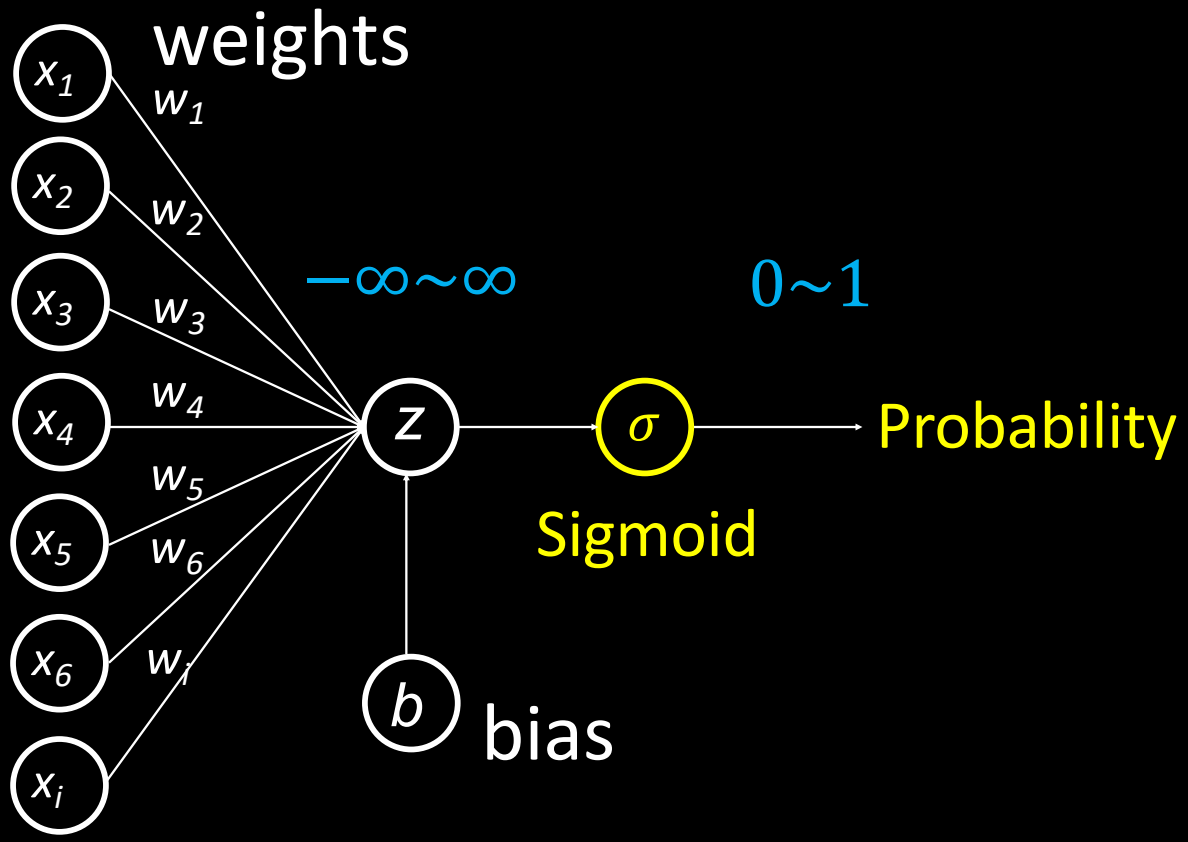
Recap: Regression vs Classification

Graph of Linear Regression

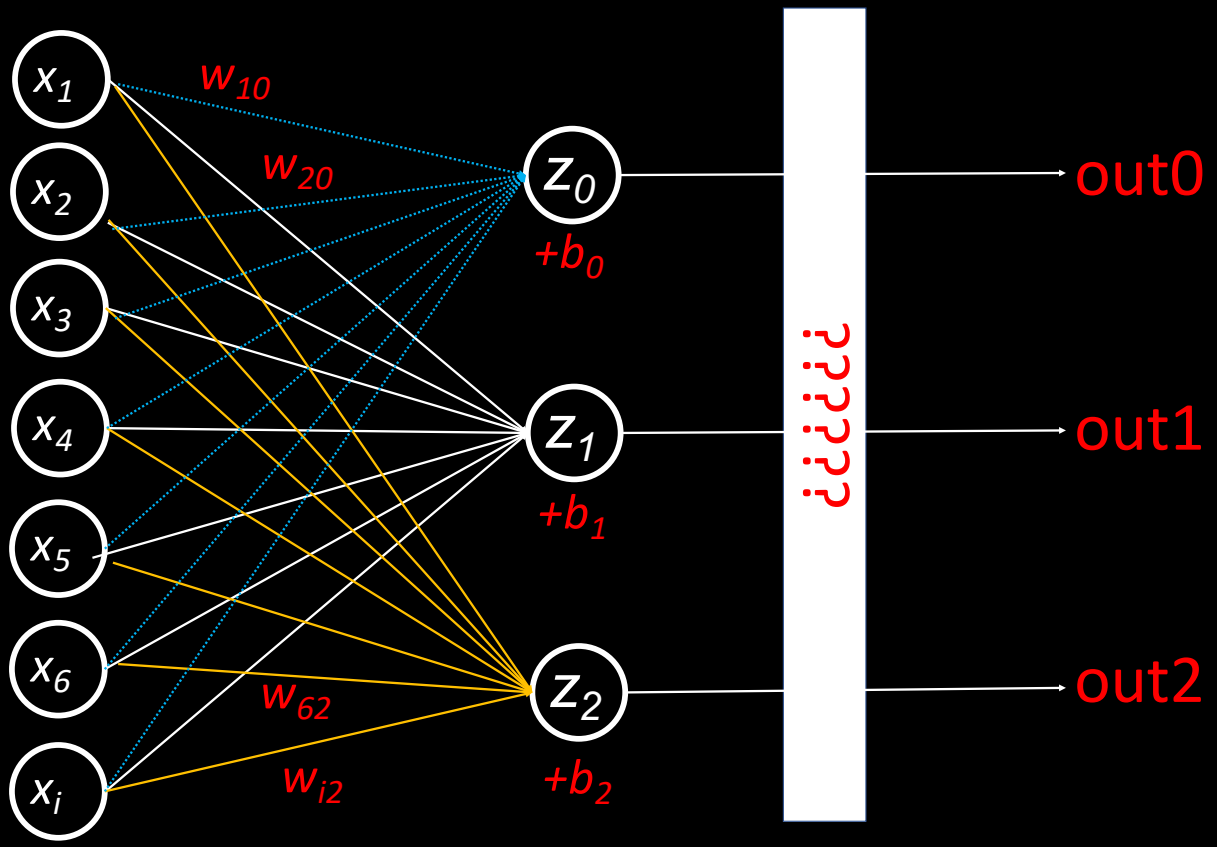


Graph of Logistic Regression

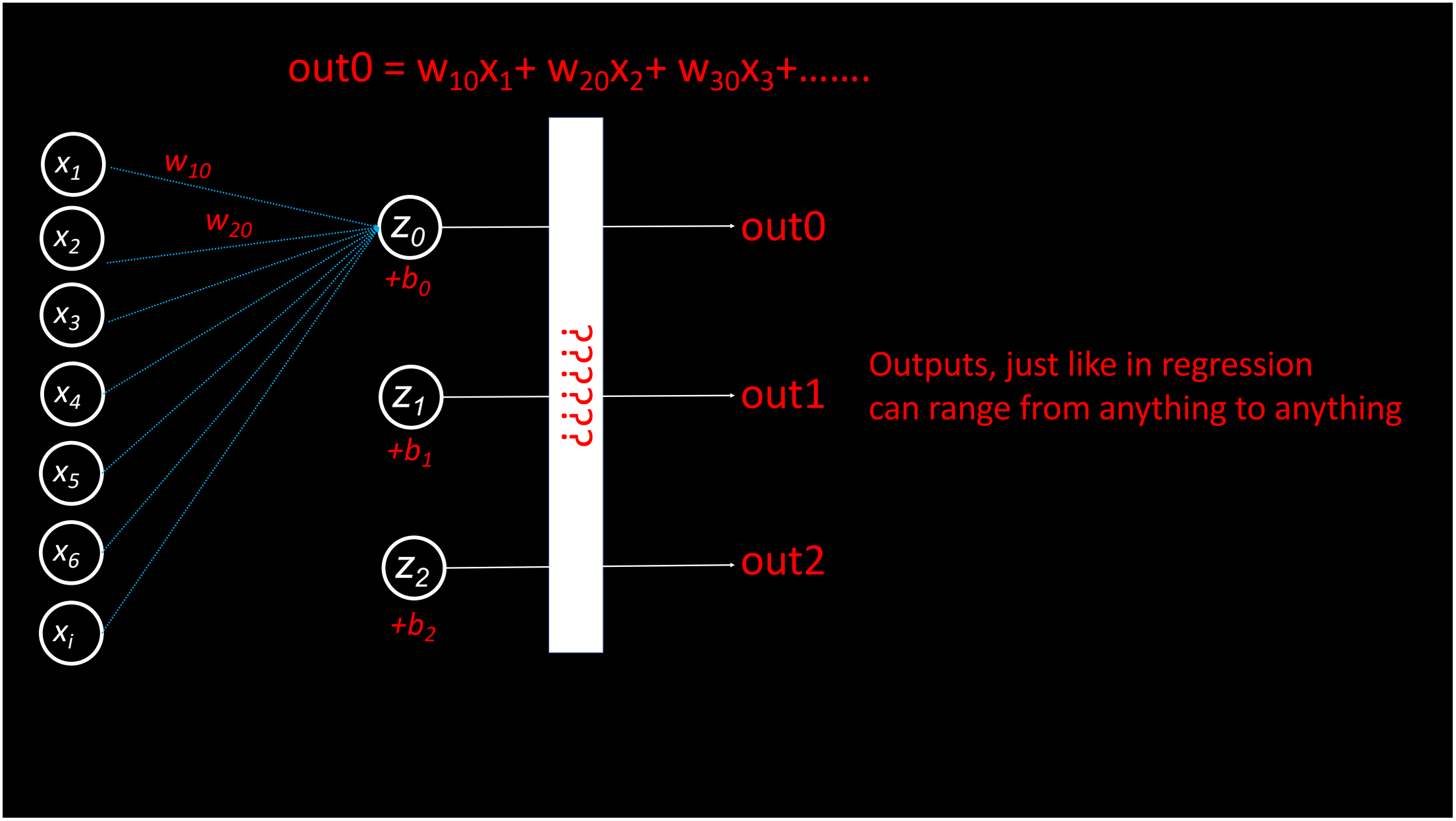
$$p = \frac{1}{1+exp-(\Sigma_i w_i x_i + b)} = \text{Sigmoid}(\Sigma_i w_i x_i + b) = S(WX)$$

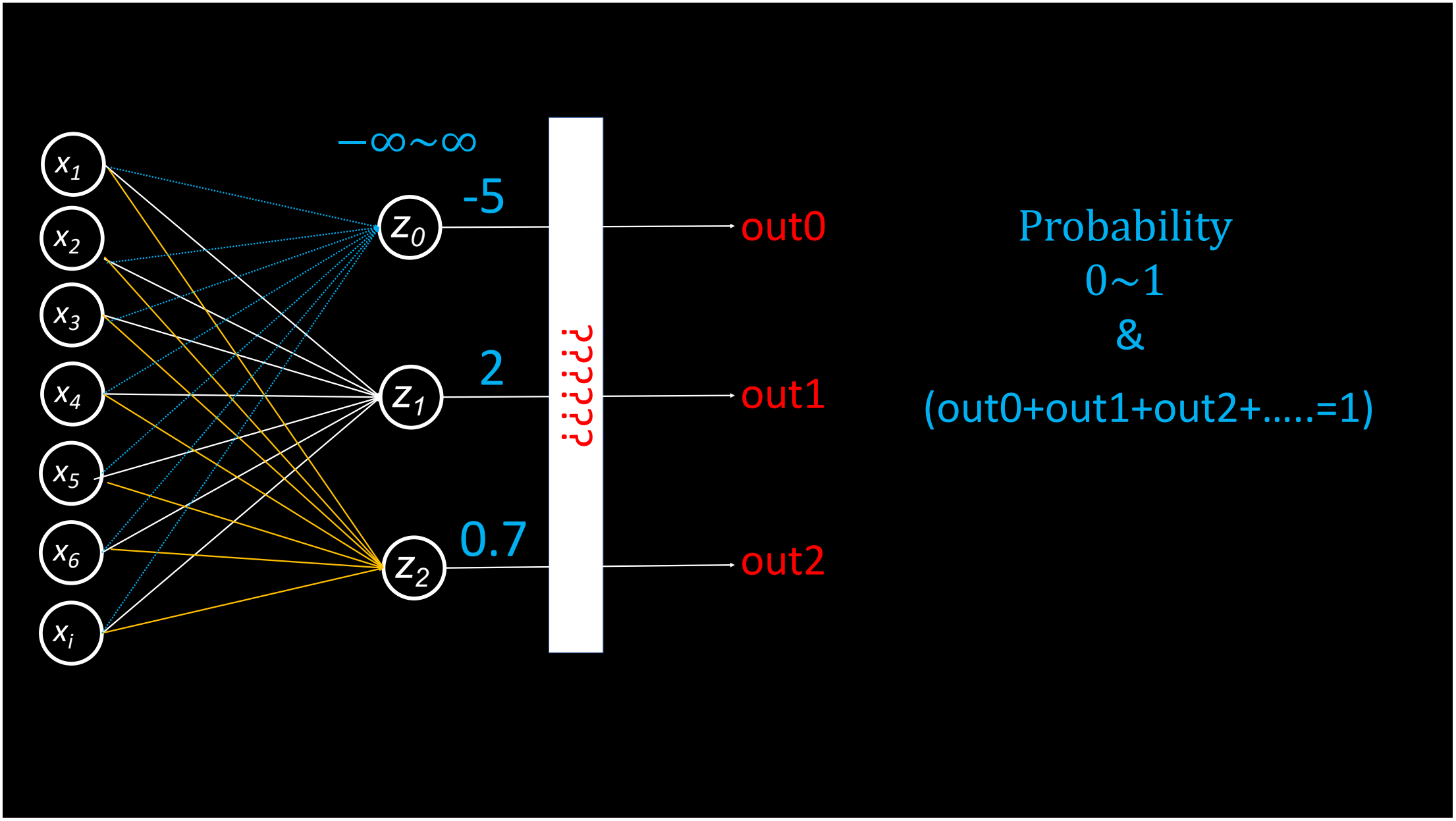


Multi-class Logistic Regression

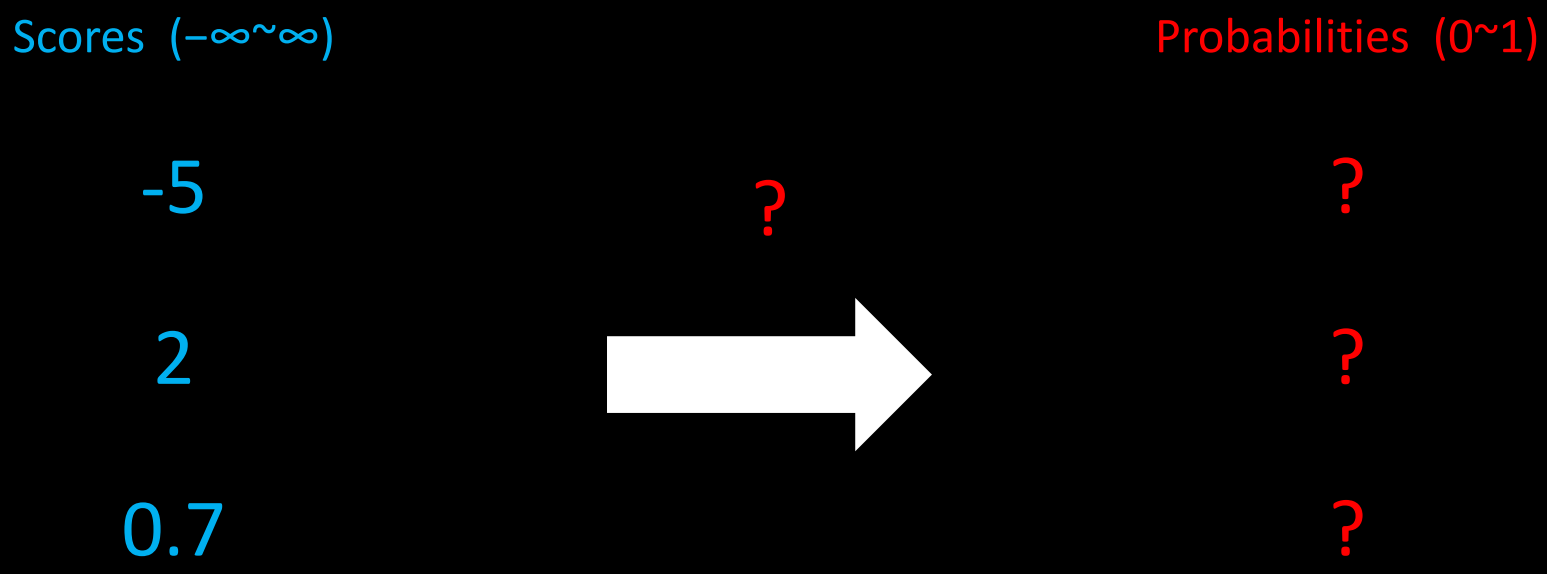


$out_j = w_{1j}x_1 + w_{2j}x_2 + w_{3j}x_3 + \ldots\ldots$


out0 = w10x1+ w20x2+ w30x3+.....
x1
x2
x3
x4
x5
x6
xi
w10
w20
z0
+b0
z1
+b1
z2
+b2
out0
out1
out2
Outputs, just like in regression
can range from anything to anything


x_1
x_2
x_3
x_4
x_5
x_6
x_i
$-\infty\sim\infty$
z_0
z_1
z_2
-5
2
0.7
??????
out0
out1
out2
Probability
0~1
&
(out0+out1+out2+.....=1)

The network gives us 3 scores. We need 3 probabilities.



Need: each one between 0 and 1, and all of them add up to 1

Why not just divide each score by the total?

$-5 + 2 + 0.7 = -2.3$

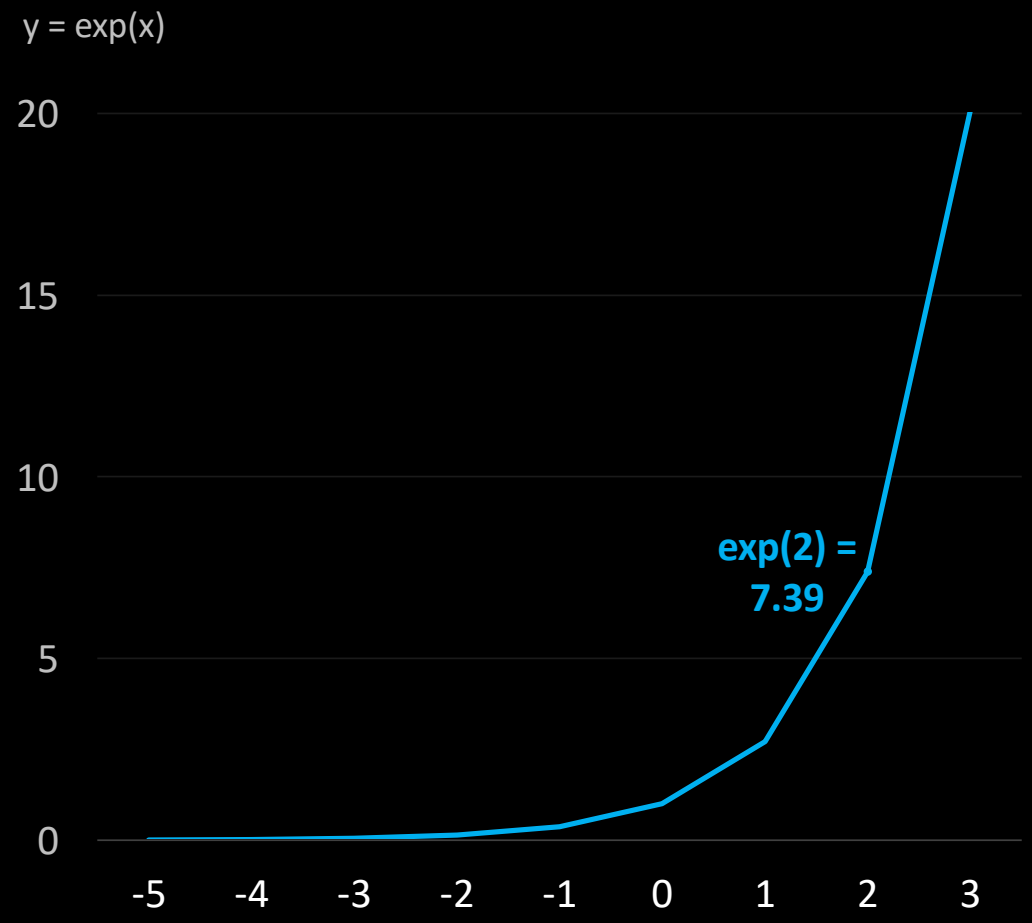
$-5 / -2.3$	$= 2.17$	> 1
$2 / -2.3$	$= -0.87$	< 0
$0.7 / -2.3$	$= -0.30$	< 0

Negative scores break it.

The total can be negative or zero, and the results fall outside 0~1.

Fix: make every score positive first.

Step 1: exp( ) makes every score positive

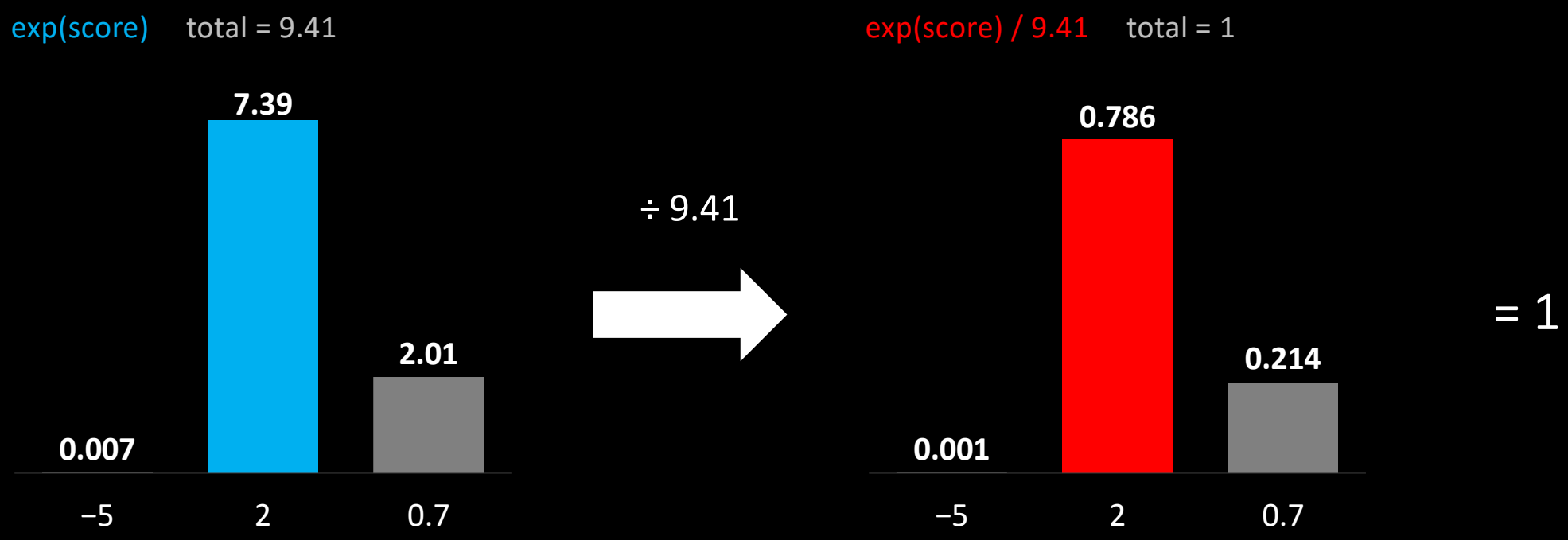


score		exp(score)
−5	→	0.007
2	→	7.39
0.7	→	2.01

Always positive, so dividing will work.

Big scores get much bigger: 2 is now 1,000× the −5.

Step 2: divide by the total, so everything adds up to 1



$$\text{Softmax}(z_j) = \exp(z_j) \,/\, \Sigma_k \exp(z_k) \quad = \text{Step 1 then Step 2}$$

With only 2 classes, Softmax is the Sigmoid you already know

Two scores: a (class 1) and b (class 0)

$$p(y=1) = e^a / (e^a + e^b)$$ Softmax

$$= 1 / (1 + e^{b-a})$$ divide top and bottom by e^a

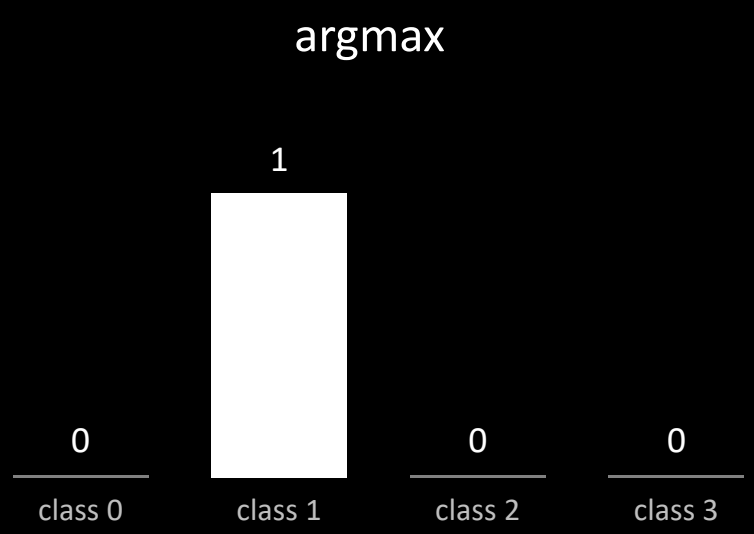
$$= 1 / (1 + e^{-(a-b)})$$

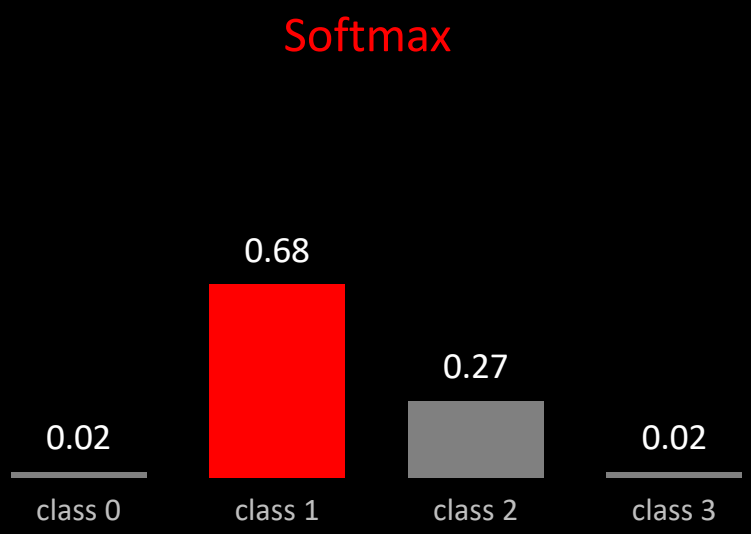
$$= \text{Sigmoid}(a - b)$$ the binary logistic regression formula

One sigmoid output == softmax over 2 classes. Softmax is just the N-class version.

Softmax is a “soft” argmax

Scores (Wx): −0.84, 2.69, 1.76, −0.69





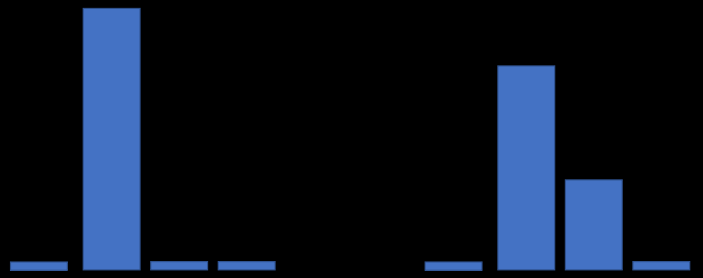
Hard: picks one winner, ignores the rest. Not differentiable, so it cannot be trained.

Soft: the winner gets most, close runners-up still get some. Smooth, so gradient descent works.

Softmax ==
Sigmoid for multi-class

$$p(y = N) = Softmax(W_N X) = \frac{exp(W_N X)}{\sum_n exp(W_n X)}$$

Data	Output (Wx)	exp(Wx)	argmax	Softmax exp(Wx) / sum(exp(Wx)))
3	-0.84	0.43	0	0.02
4	2.69	14.73	1	0.68
5	1.76	5.81	0	0.27
6	-0.69	0.50	0	0.02



$$p = \frac{1}{1+exp-(\Sigma_i w_i x_i + b)} = \text{Sigmoid}(w_i x_i + b) = S(WX)$$

V S

$$p(y=0) = SX(W_0 X) = \frac{exp-(W_0 X)}{\sum_n exp-(W_n X)}$$

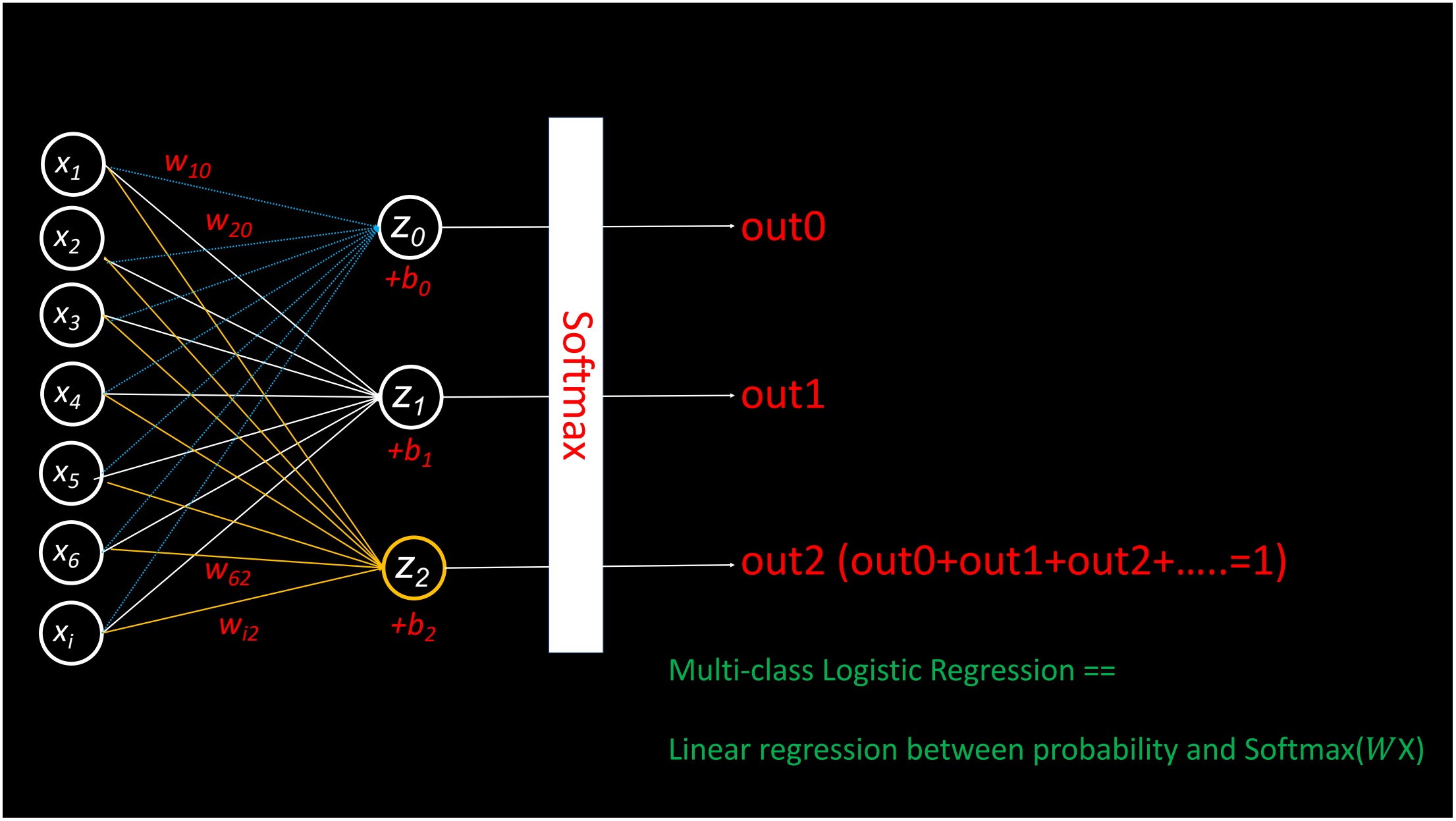
$$p(y=1) = SX(W_1 X) = \frac{exp-(W_1 X)}{\sum_n exp-(W_n X)}$$

•
•
•

$$p(y=N) = SX(W_N X) = \frac{exp-(W_N X)}{\sum_n exp-(W_n X)}$$

For multi-classes, you will have one *W* for each class

The summation for all the P Should be 1


x_1
x_2
x_3
x_4
x_5
x_6
x_i
w_{10}
w_{20}
w_{62}
w_{i2}
z_0
z_1
z_2
$+b_0$
$+b_1$
$+b_2$
Softmax
out0
out1
out2 (out0+out1+out2+.....=1)
Multi-class Logistic Regression ==
Linear regression between probability and Softmax(WX)

Binary Cross Entropy

$$-\sum_n [\mathrm{y} \cdot \log(p(\hat{y})) + (1-y) \cdot \log(p(1-\hat{y}))])$$

Multi-Class Cross Entropy

$$-\sum_n \mathrm{y} \cdot \log(p(\hat{y}))$$

y	$\hat{y}$		
0	0.6		
1	0.3	=	-1 * log(0.3)
0	0.1		

Details of Model Training

https://tensorspace.org/html/playground/alexnet.html

ML2025 / Projects / mnist /

changlabntu Add files vi

Name

..

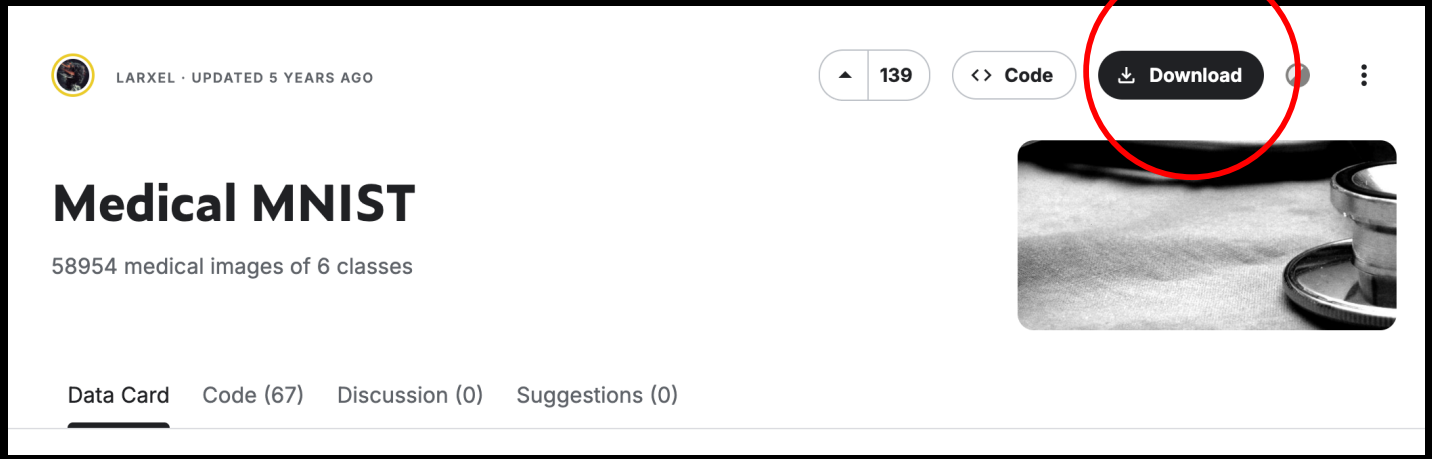
.__init__

train.py

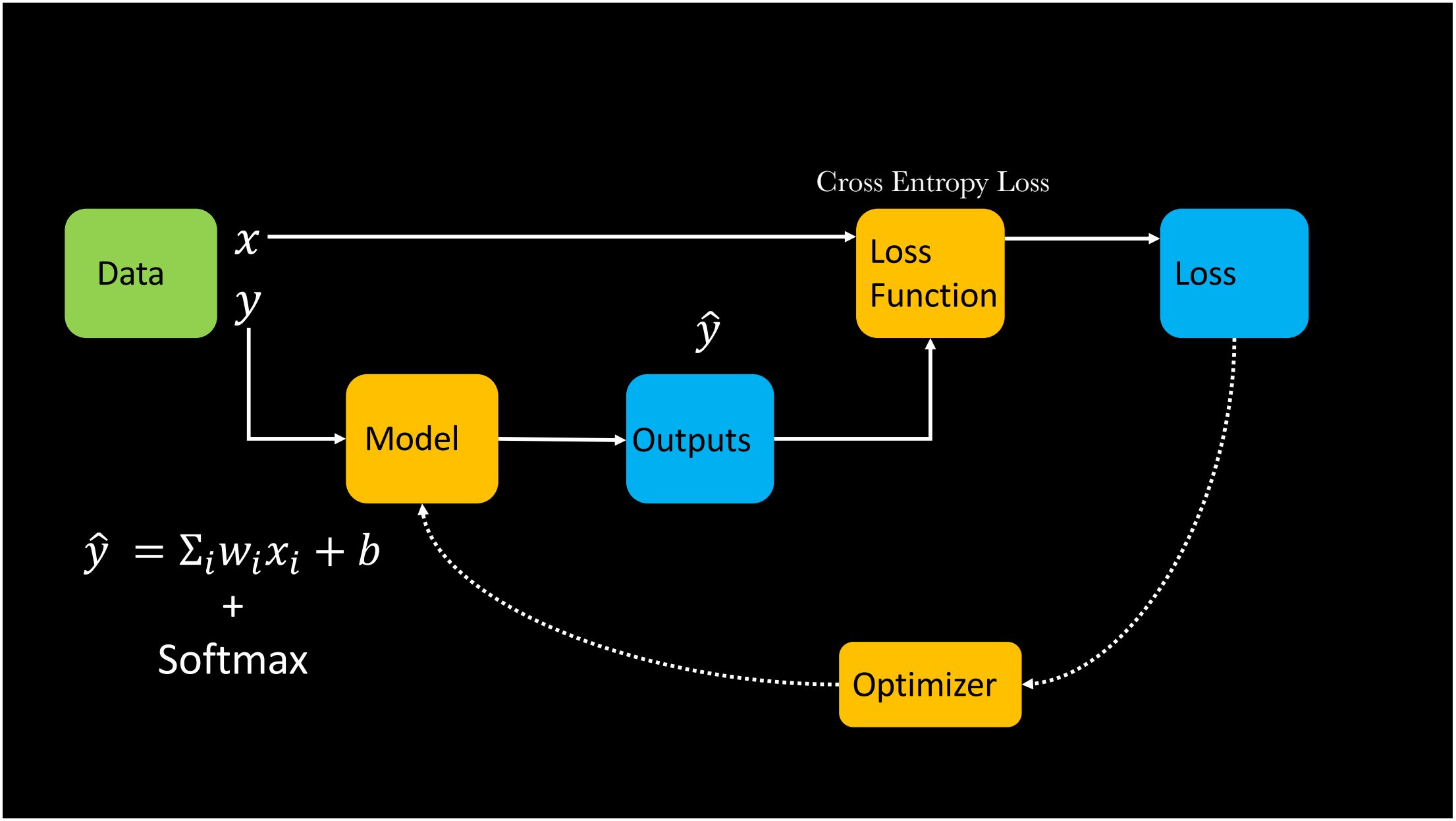
train_template.py

utils_mnist.py

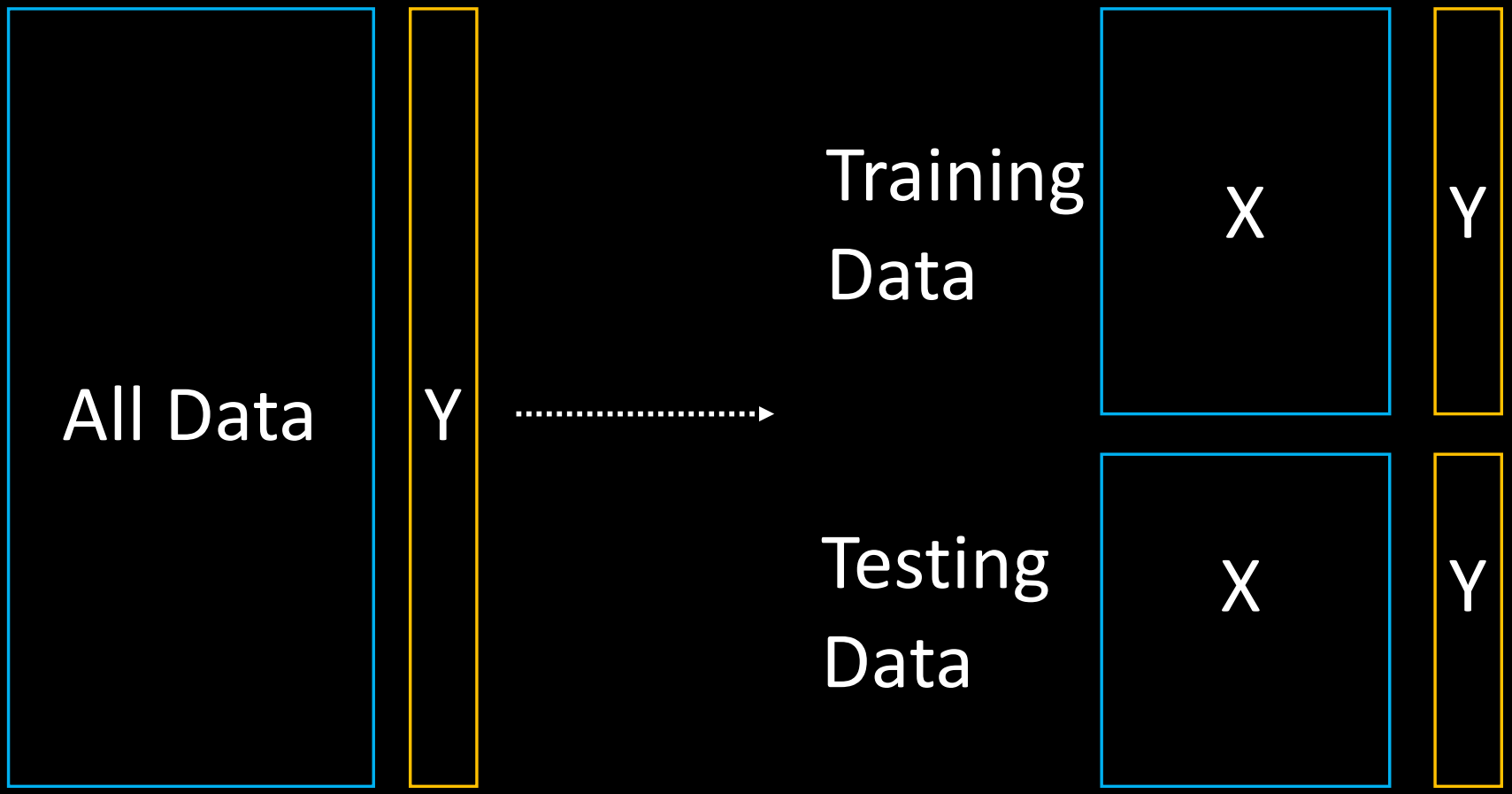
Kaggle






Cross Entropy Loss
Data
x
y
Loss Function
Loss
$\hat{y}$
Model
Outputs
$\hat{y} = \Sigma_i w_i x_i + b$
+
Softmax
Optimizer

Train-Testing Splitting



Scenario A

Training in :



Using (testing) in :



Scenario B

Training in :



Using (testing) in :



Why can’t the data be the same?

Training Data

Testing Data



Source 1

Source 2





Training Data

Validation

Testing

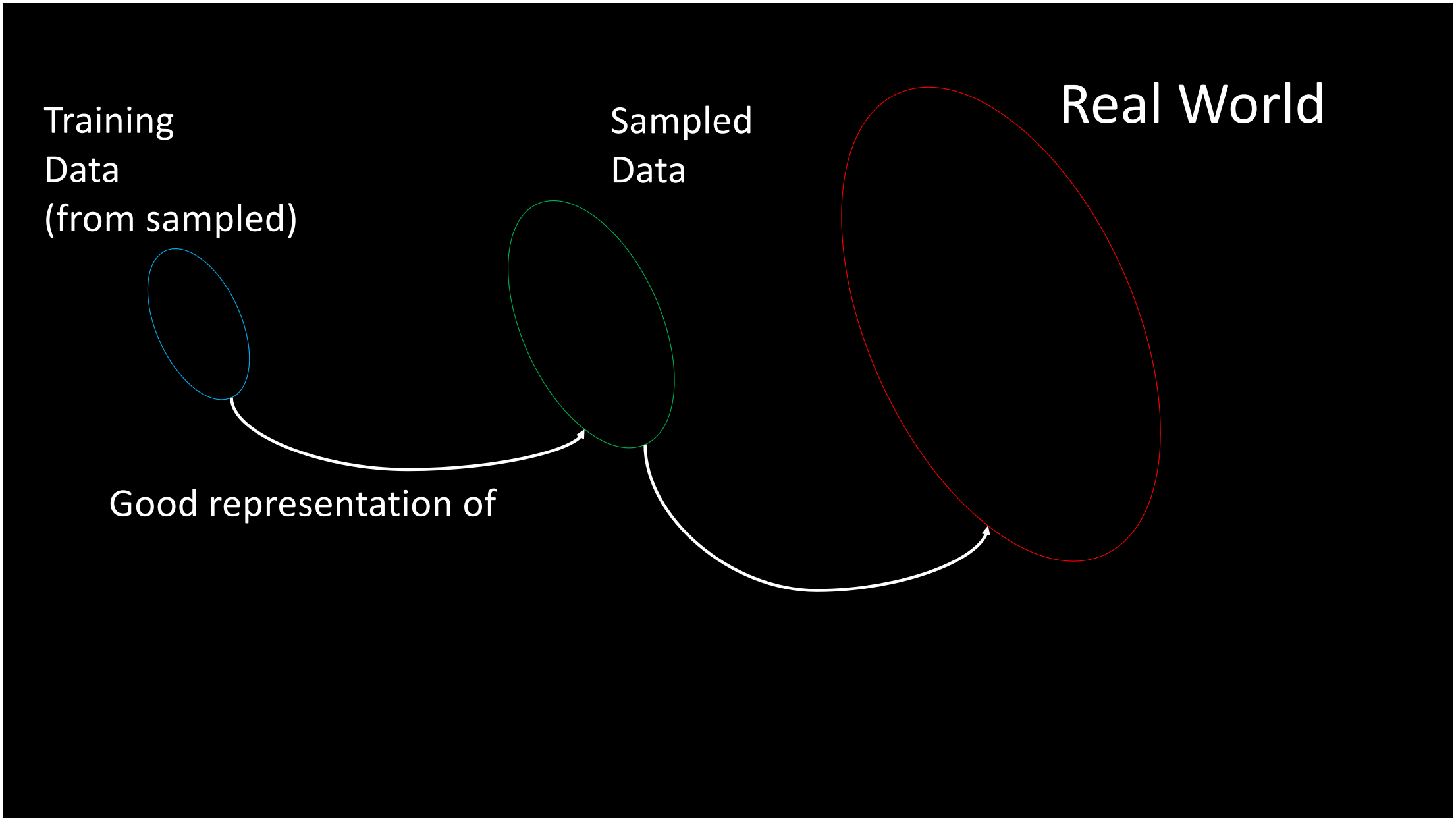
Source 1

Source 2



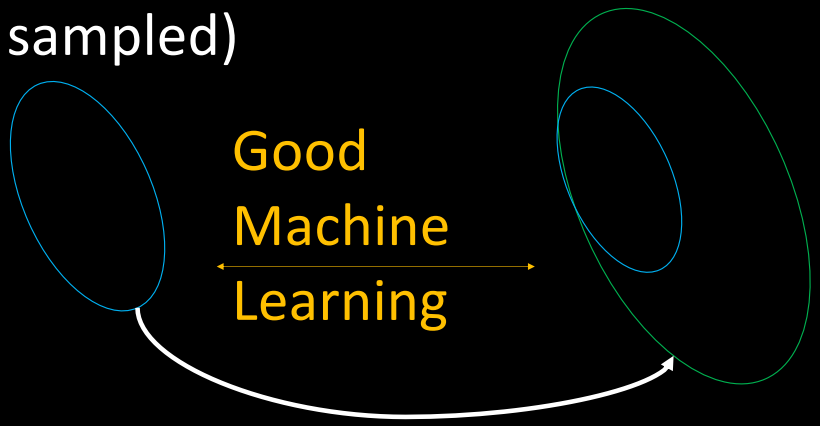


“Internal Testing”


Training
Data
(from sampled)
Sampled
Data
Real World
Good representation of

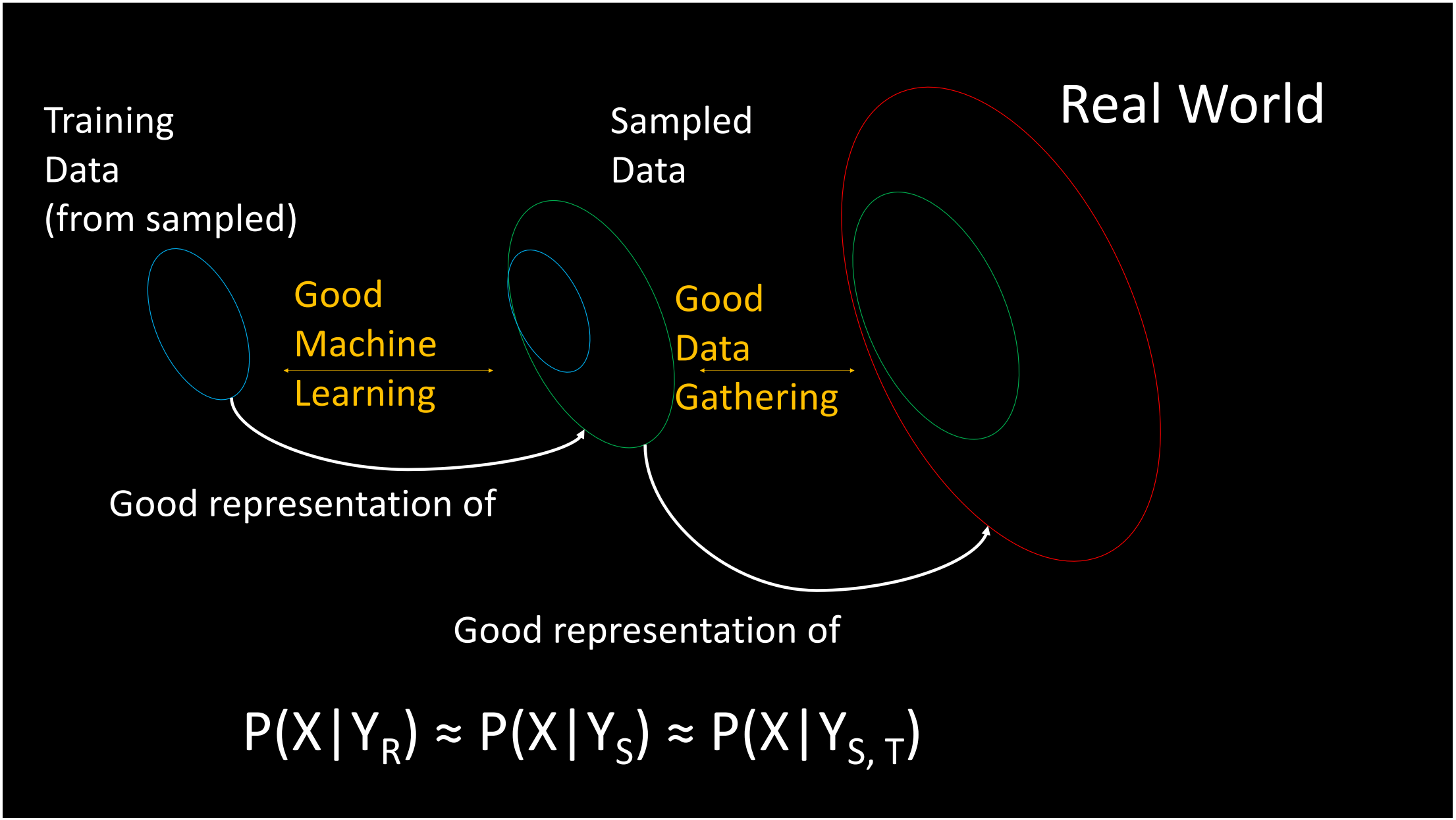
Training
Data
(from sampled)

Sampled
Data

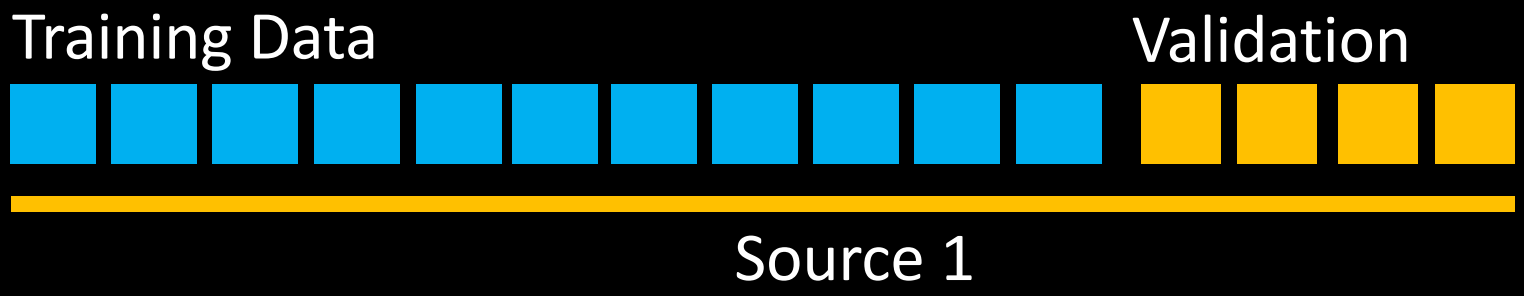


Good representation of

$$P(X|Y_R) \approx P(X|Y_S)$$


Training
Data
(from sampled)
Good
Machine
Learning
Sampled
Data
Good
Data
Gathering
Real World
Good representation of
Good representation of
$P(X|Y_R) \approx P(X|Y_S) \approx P(X|Y_{S,T})$

How to split
Train / validation?

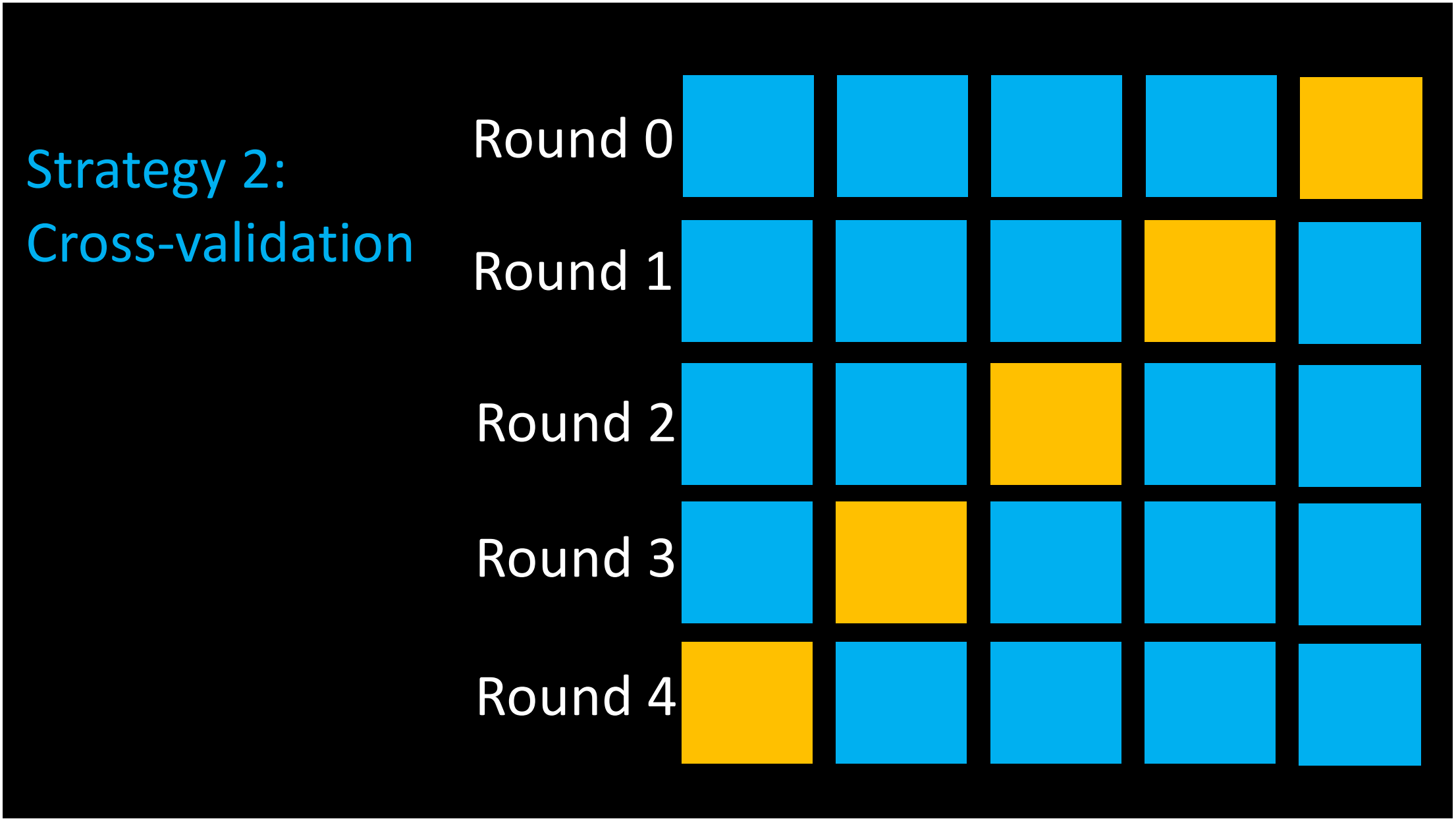


Strategy 1:

Random Splitting:



Strategy 2: Cross-validation



Train-Validation:

Easy / wasting data

10 (k) fold:

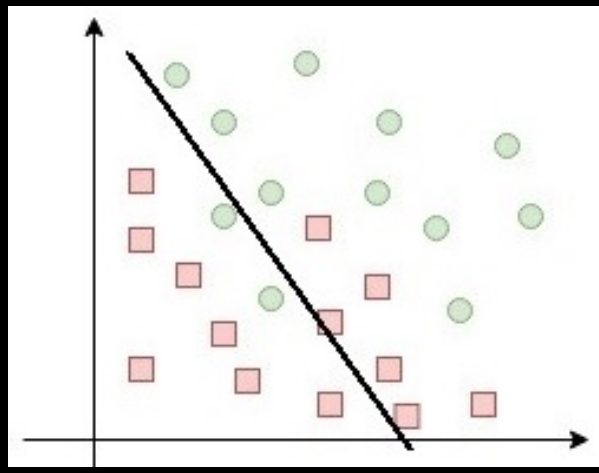
Waste 1/k data, stable

Leave one out:

Waste 1 data, unstable

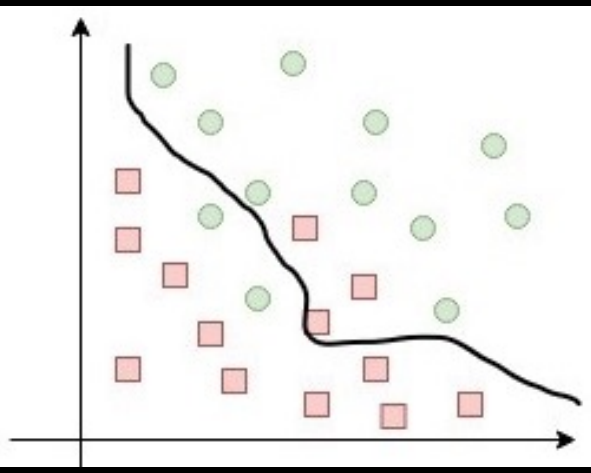
Training / Testing Error

High Training error



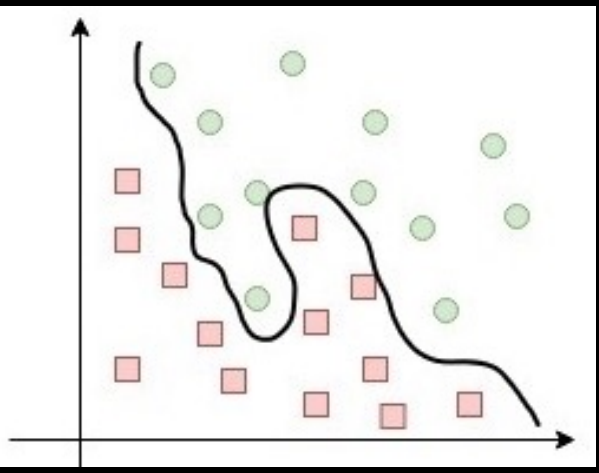
2 parameters
(linear model)

Some Training error



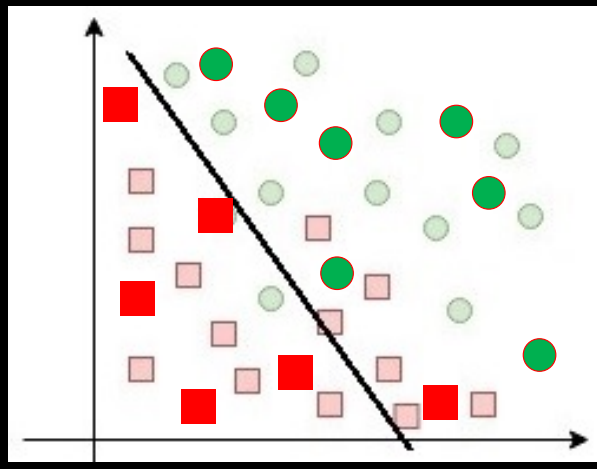
> 2 parameters
(nonlinear model)

0 Training error



Even more parameters
(nonlinear model)

High Training error

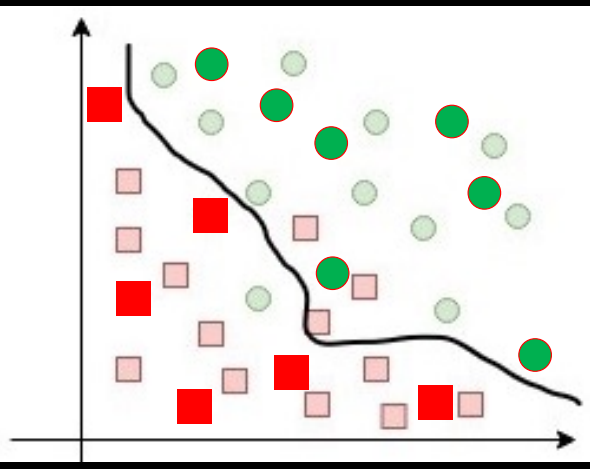


Under-fitting

2 parameters

(linear model)

Some Training error

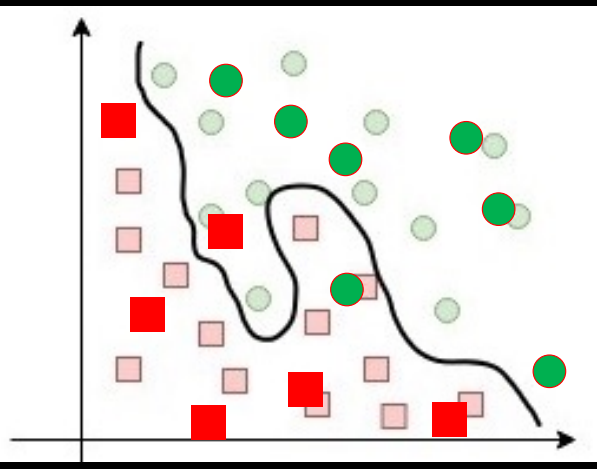


Normal Fitting

> 2 parameters

(nonlinear model)

0 Training error



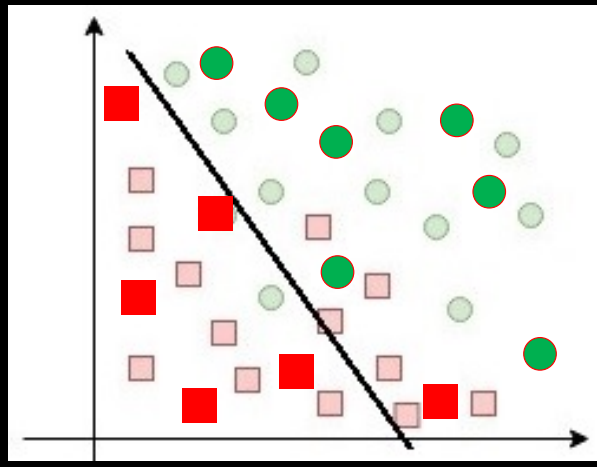
Over-fitting

Even more parameters

(nonlinear model)

Low Testing error

High Training error



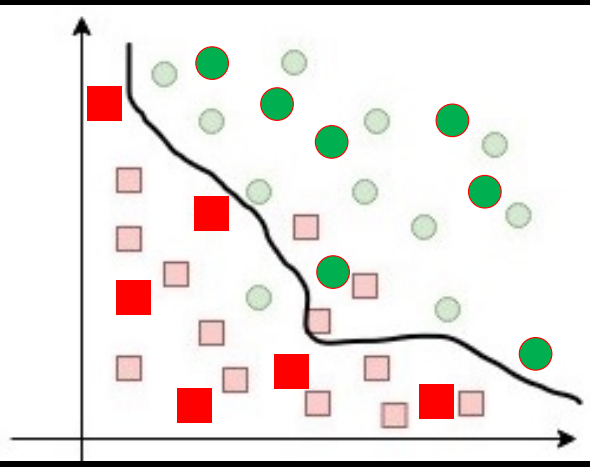
Under-fitting

2 parameters

(linear model)

Some Training error

Some Training error



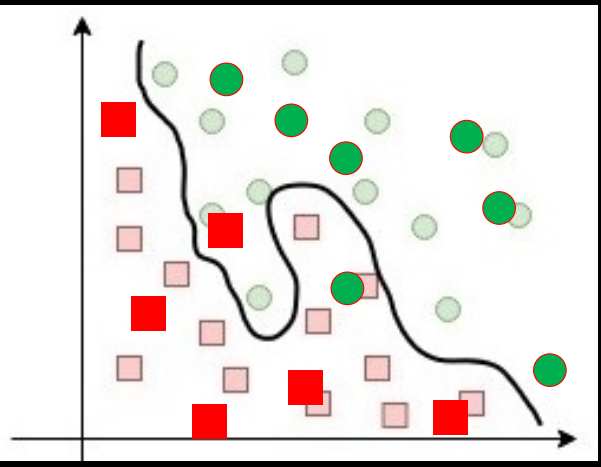
Normal Fitting

> 2 parameters

(nonlinear model)

High Testing error

0 Training error

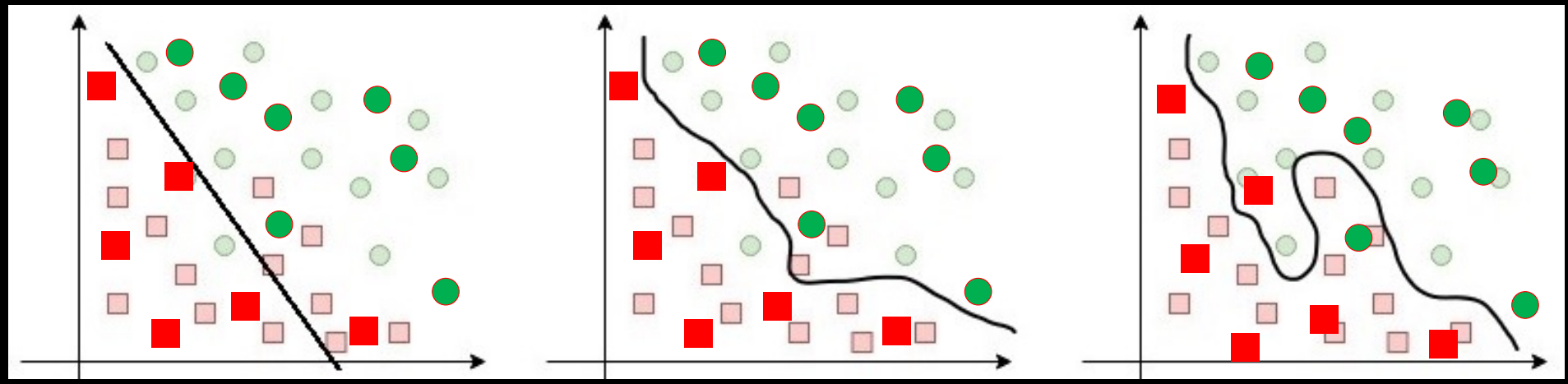


Over-fitting

Even more parameters

(nonlinear model)

More probable according to data



Less probable according to some assumption about model (prior)

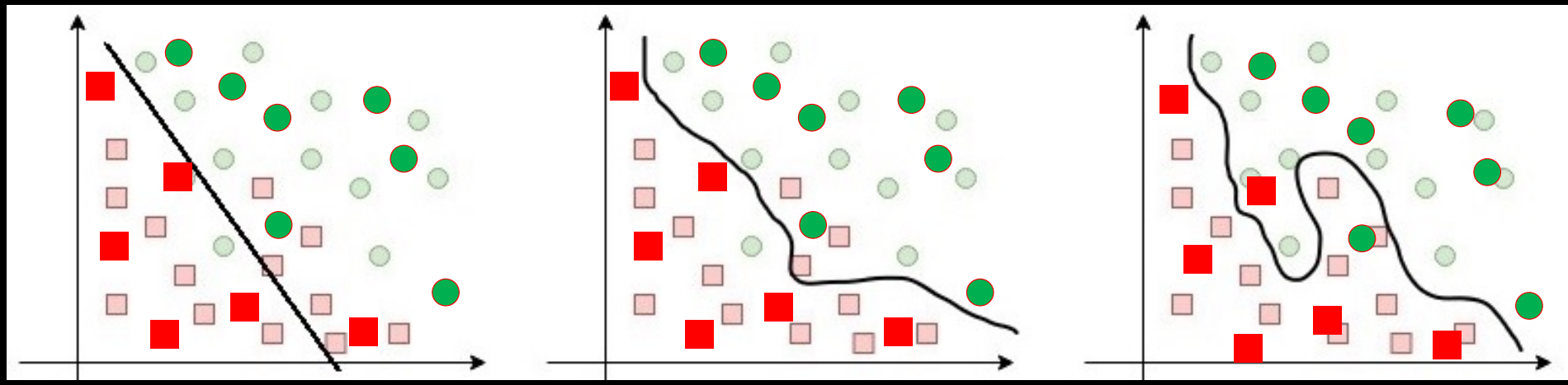
Maximum Likealihood (MLE)

More probable according to data

High Training error

Some Training error

0 Training error



Low Testing error

Some Training error

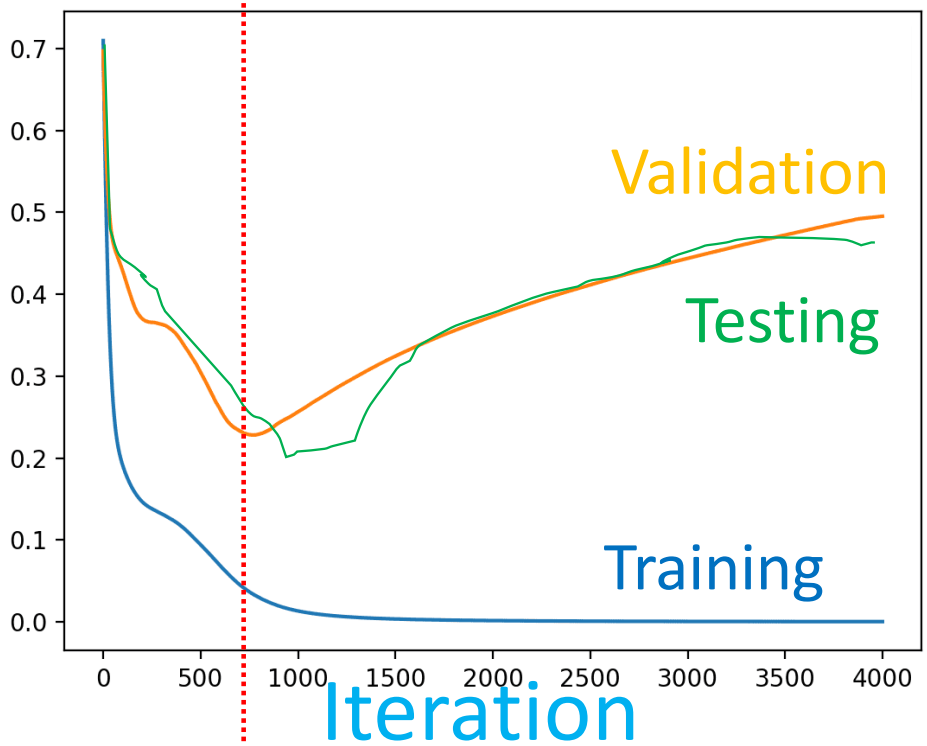
High Testing error

Maximize a Priori (MAP)

Less probable according to some assumption about model (prior)


Training Data
Validation
Testing
Source 1
Source 2

Loss


0.7
0.6
0.5
0.4
0.3
0.2
0.1
0.0
0
500
1000
1500
2000
2500
3000
3500
4000
Validation
Testing
Training
Iteration

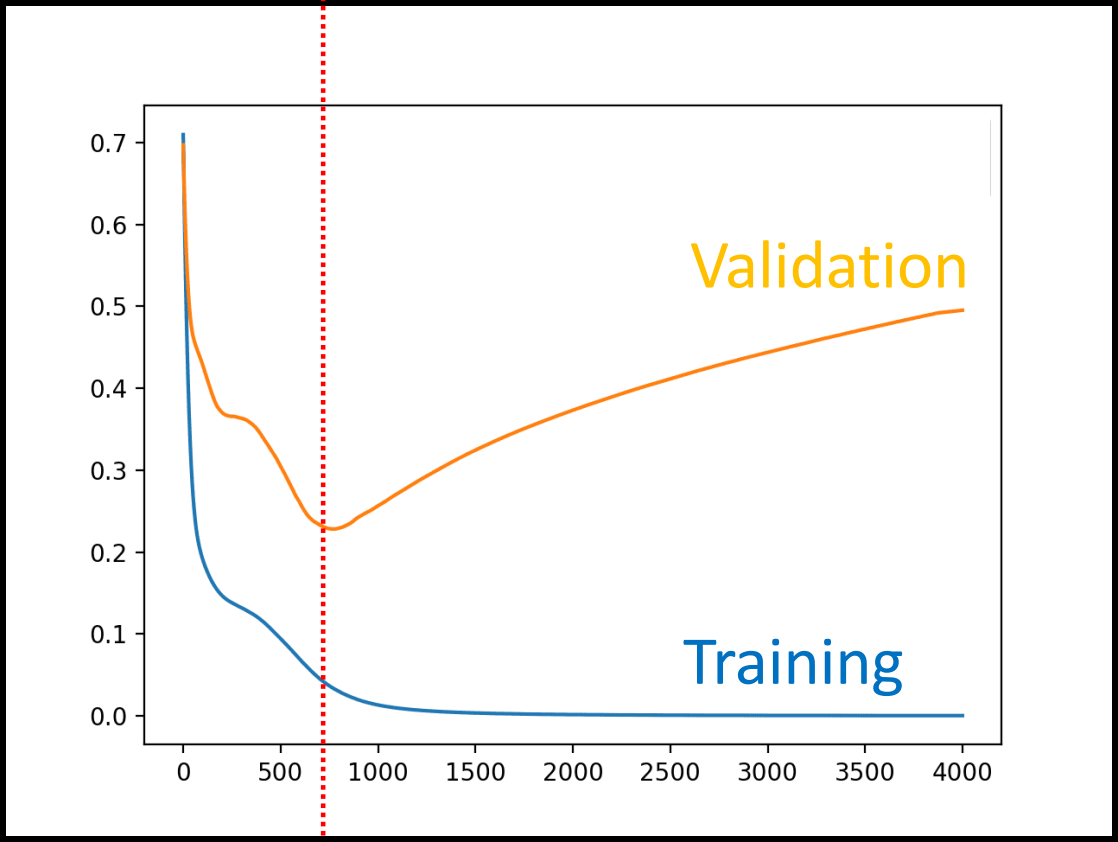
Training Data

Validation Data



Source 1

Loss



Underfitting

Overfitting

What cause overfitting?

-- too little training data

-- too much features (compared to the number of training data)

-- model too complicated (again,
compared to the number of training data)

Combat overfitting

--less complicated model

-- tune hyperparameter (learning rate scheduling….)

-- using ensemble models (will talk later)

-- make your data stronger (will talk later)

What's considered cheating (data leakage)

Training mixed up with validation

Training and validation are correlated by some factor
(same subject, related instance)

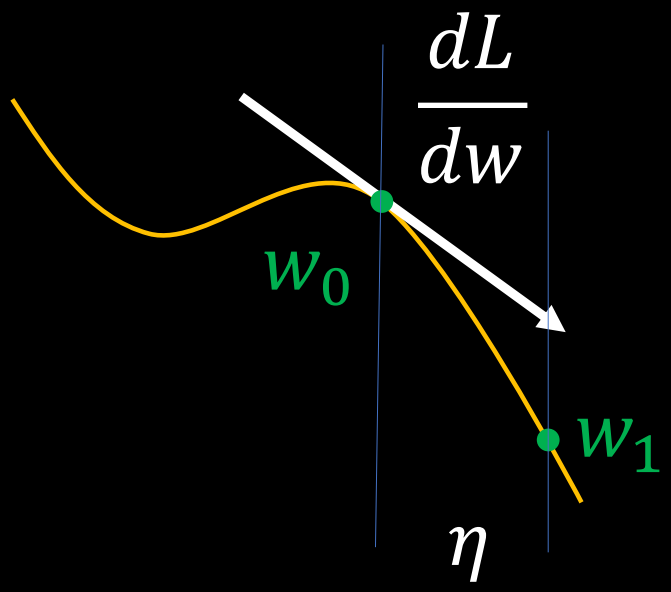
Training and validation had different and identifiable noise / pattern

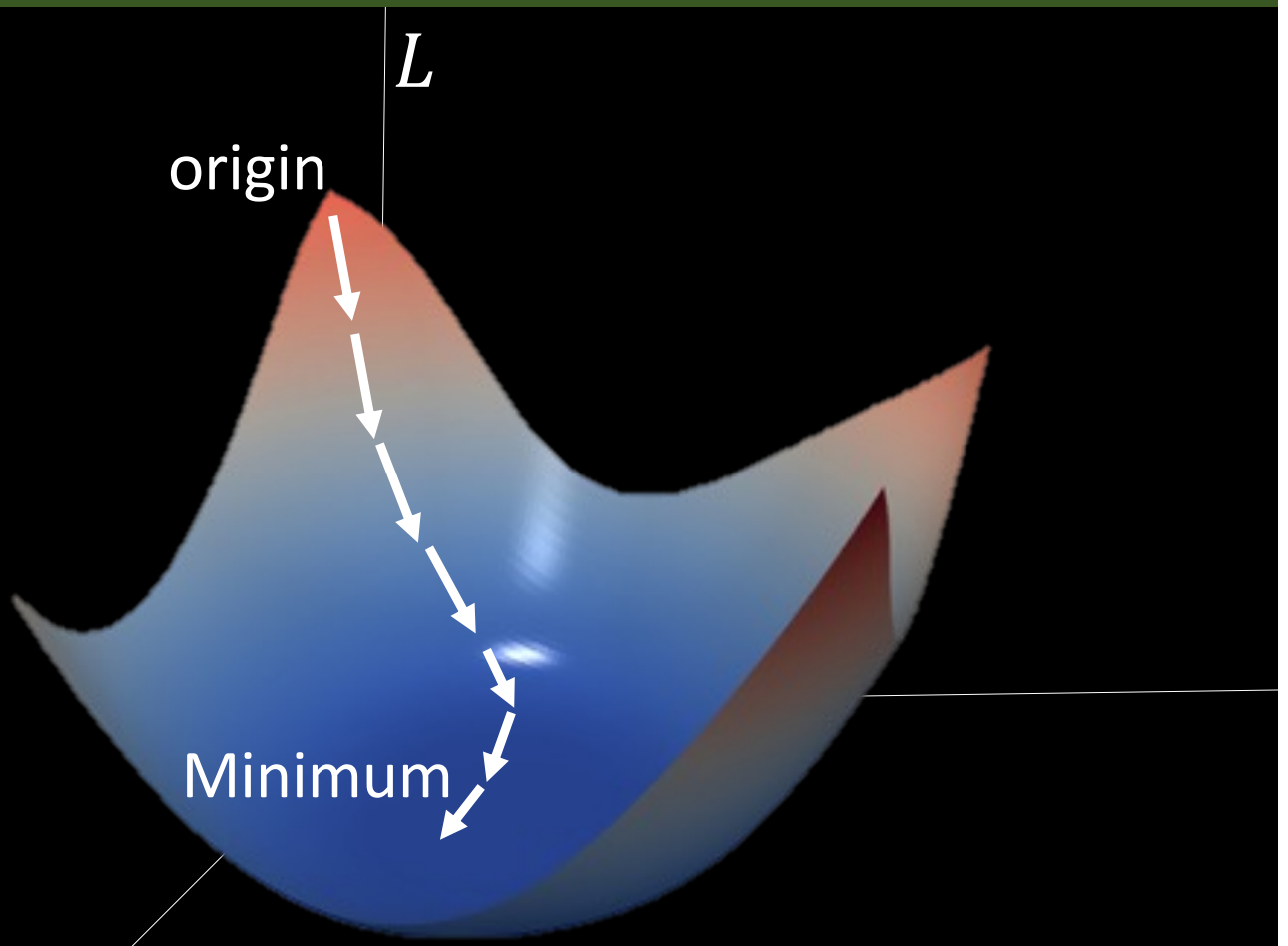
Optimization & Stochastic Gradient Descend (SGD)

Training Neural Networks: optimization

Optimizer

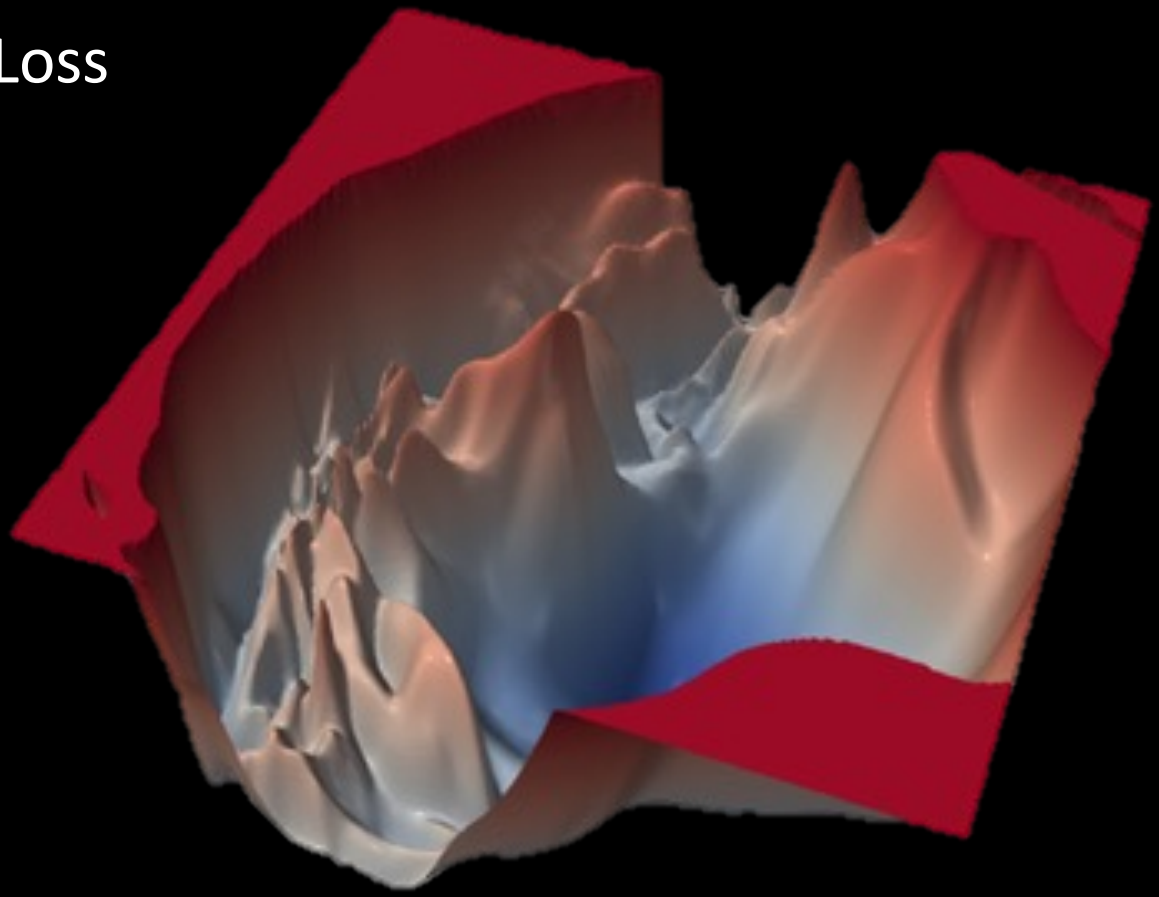
$argmin L(w_i, b)$

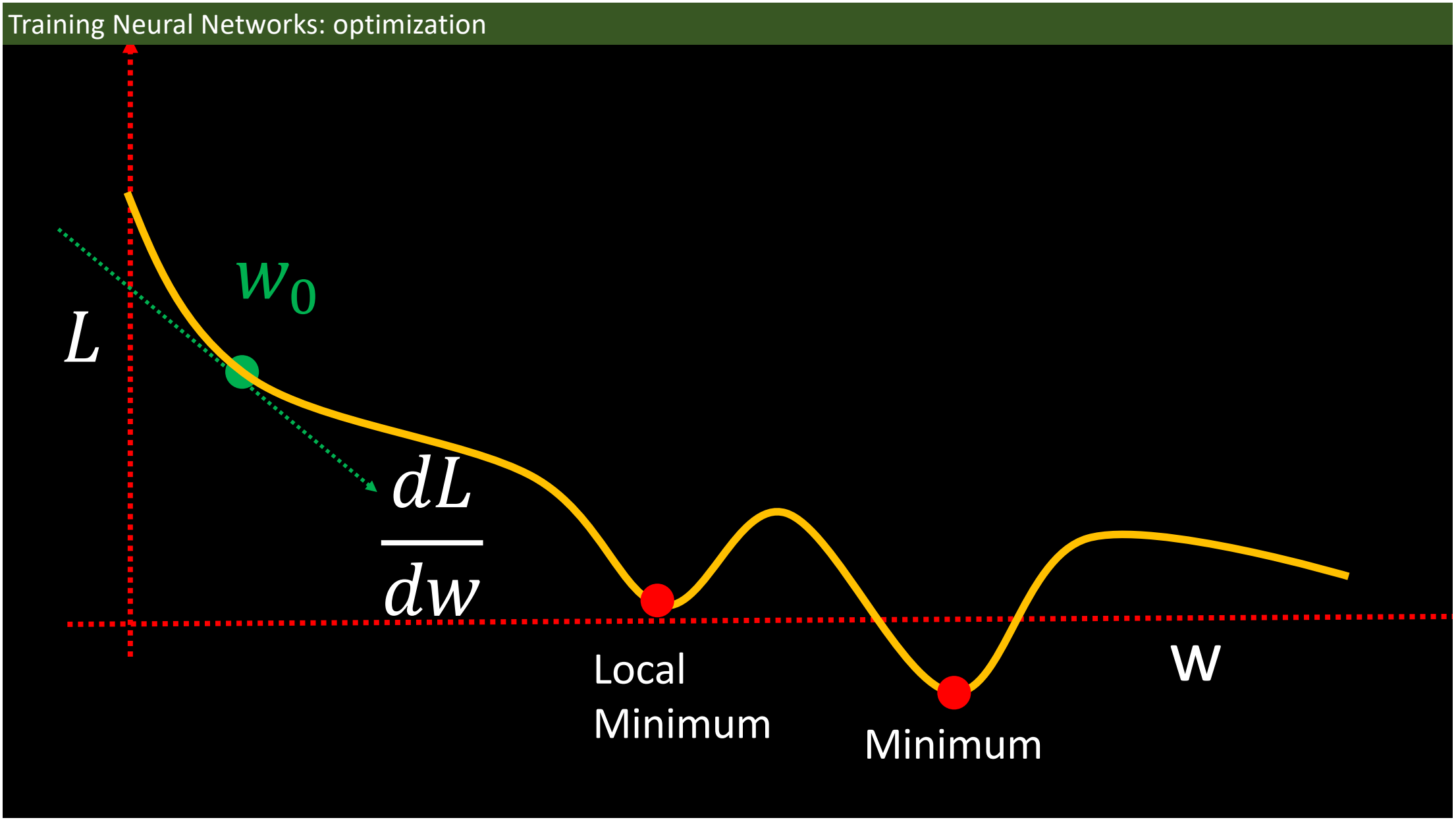




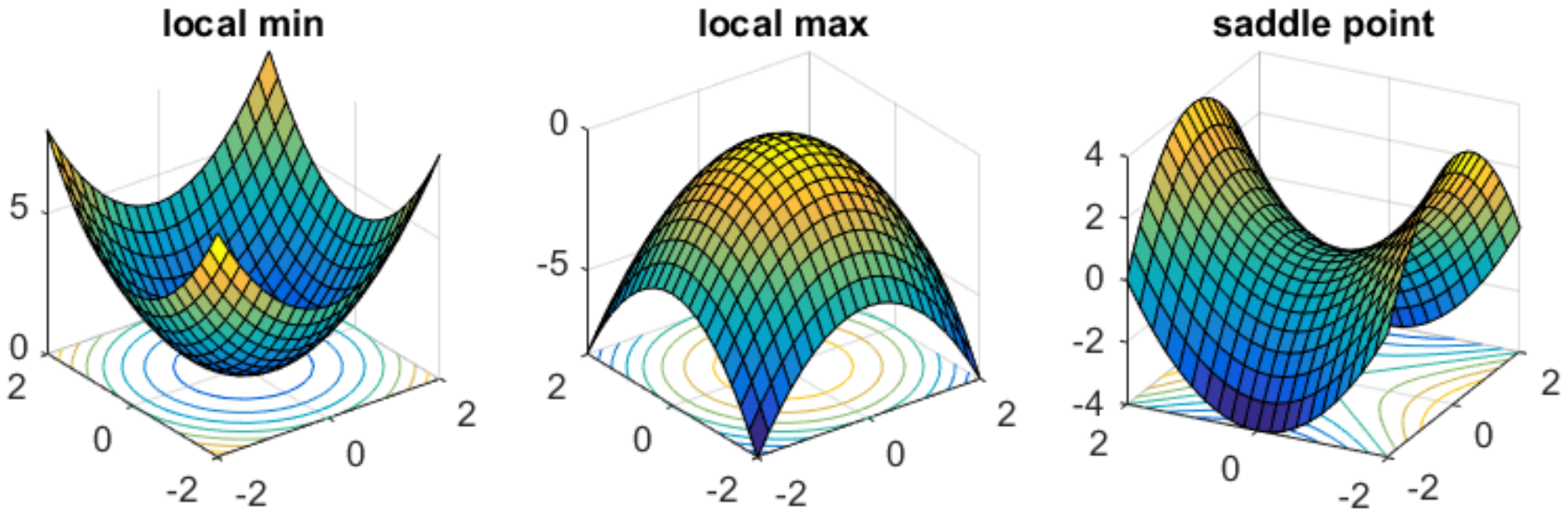
$$w_1 = w_0 - \frac{dL}{dw}\eta$$

Real Surface of Loss




Training Neural Networks: optimization
L
w_0
$\frac{dL}{dw}$
Local
Minimum
Minimum
W

Training Neural Networks: optimization



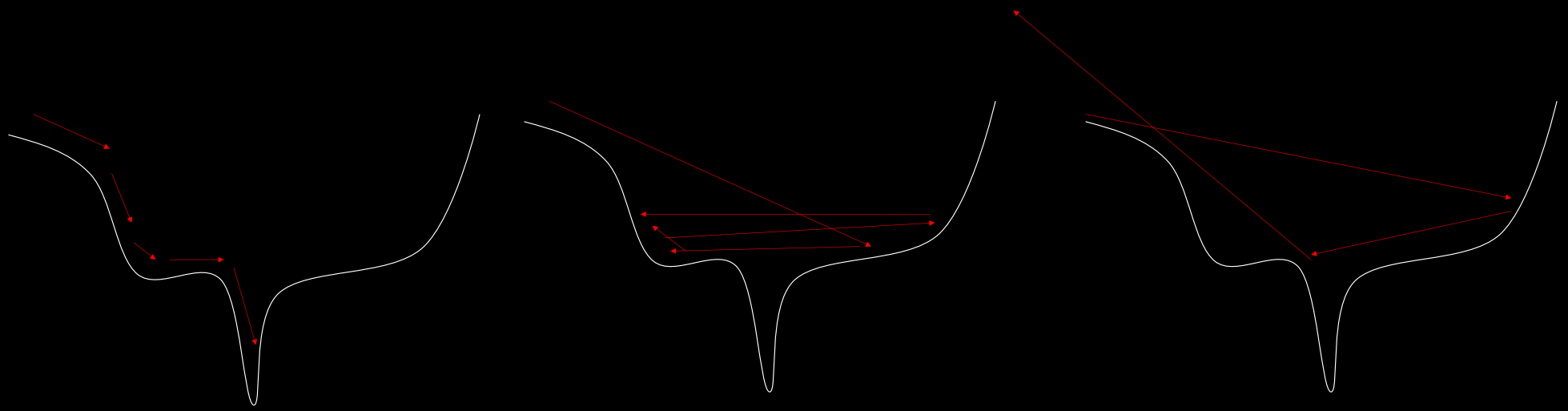


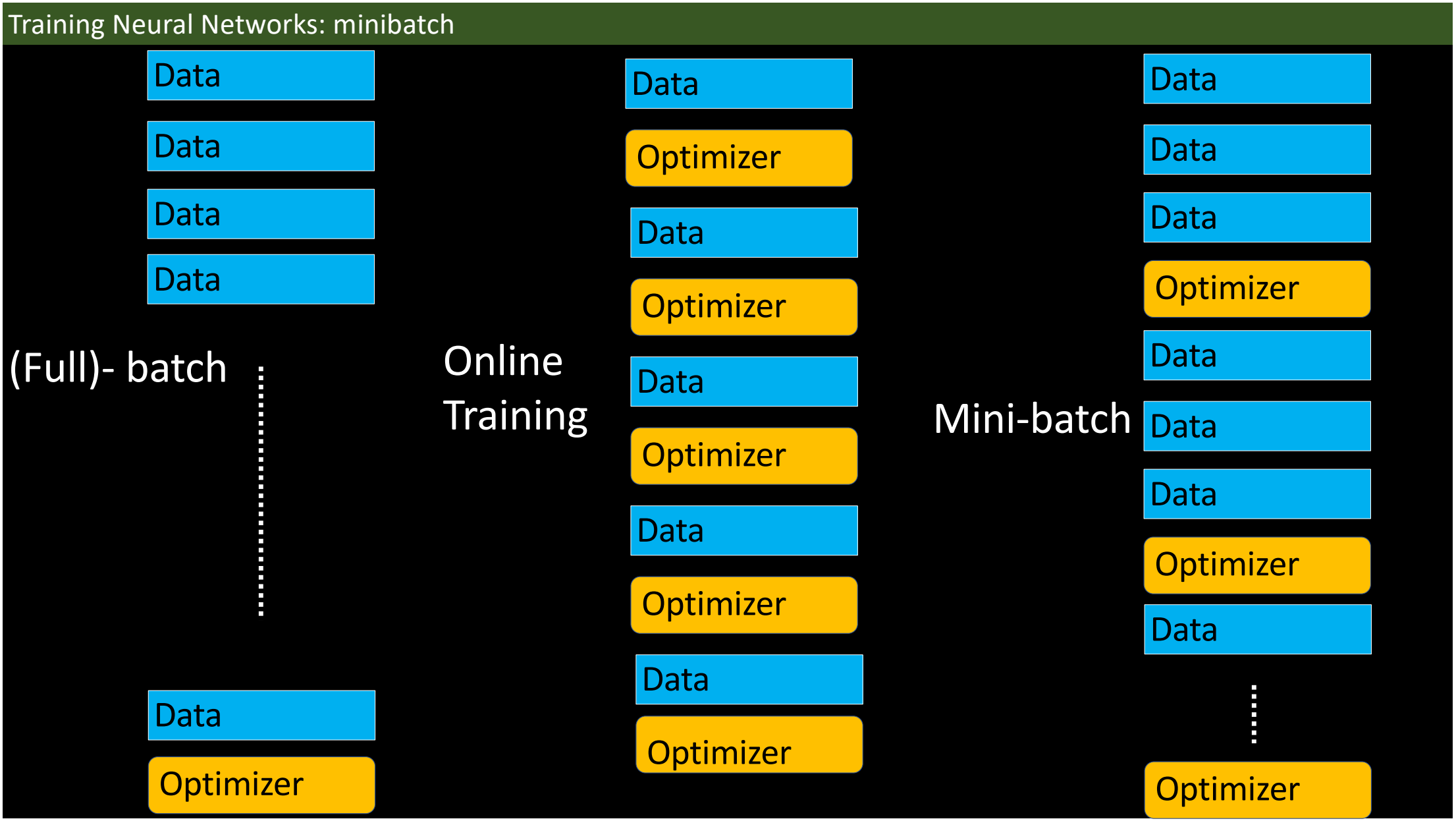


Fortunately
this is the most common



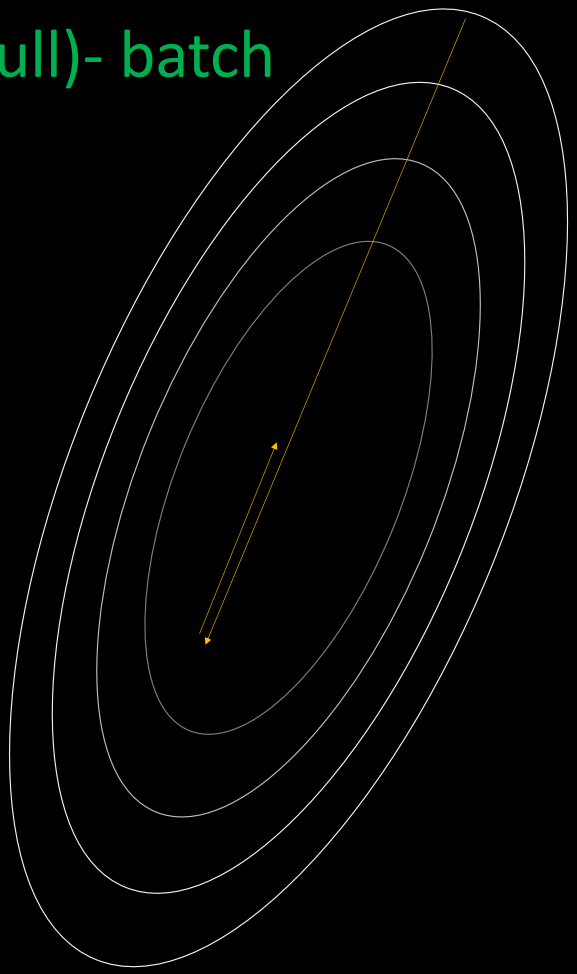
Training Neural Networks: optimization




Training Neural Networks: minibatch
Data
Data
Data
Data
(Full)- batch
Data
Optimizer
Online Training
Data
Optimizer
Data
Optimizer
Data
Optimizer
Data
Optimizer
Data
Optimizer
Mini-batch
Data
Data
Data
Optimizer
Data
Data
Data
Optimizer
Data
Optimizer

Training Neural Networks: minibatch

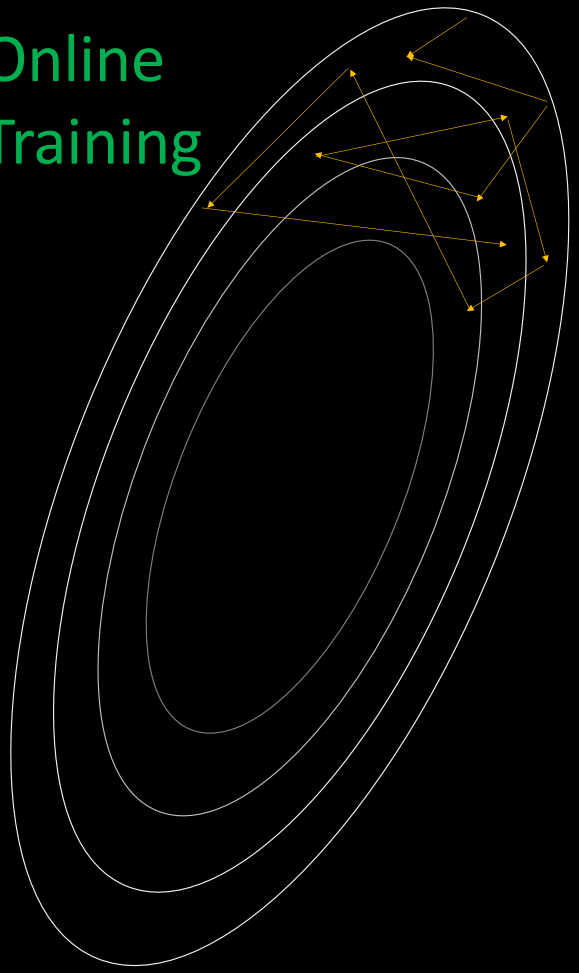
(Full)- batch



"deterministic"

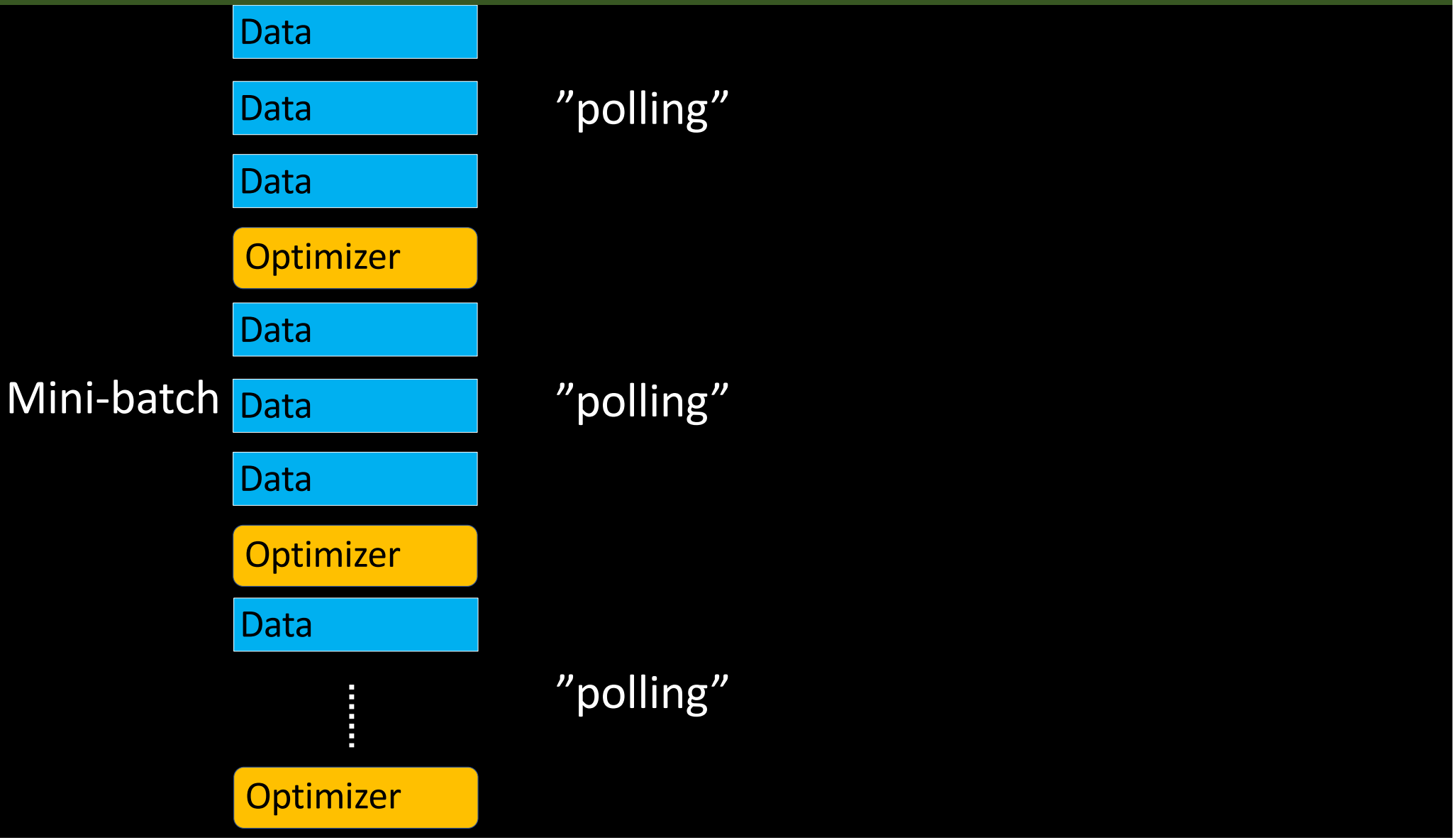
Training Neural Networks: minibatch

Online Training

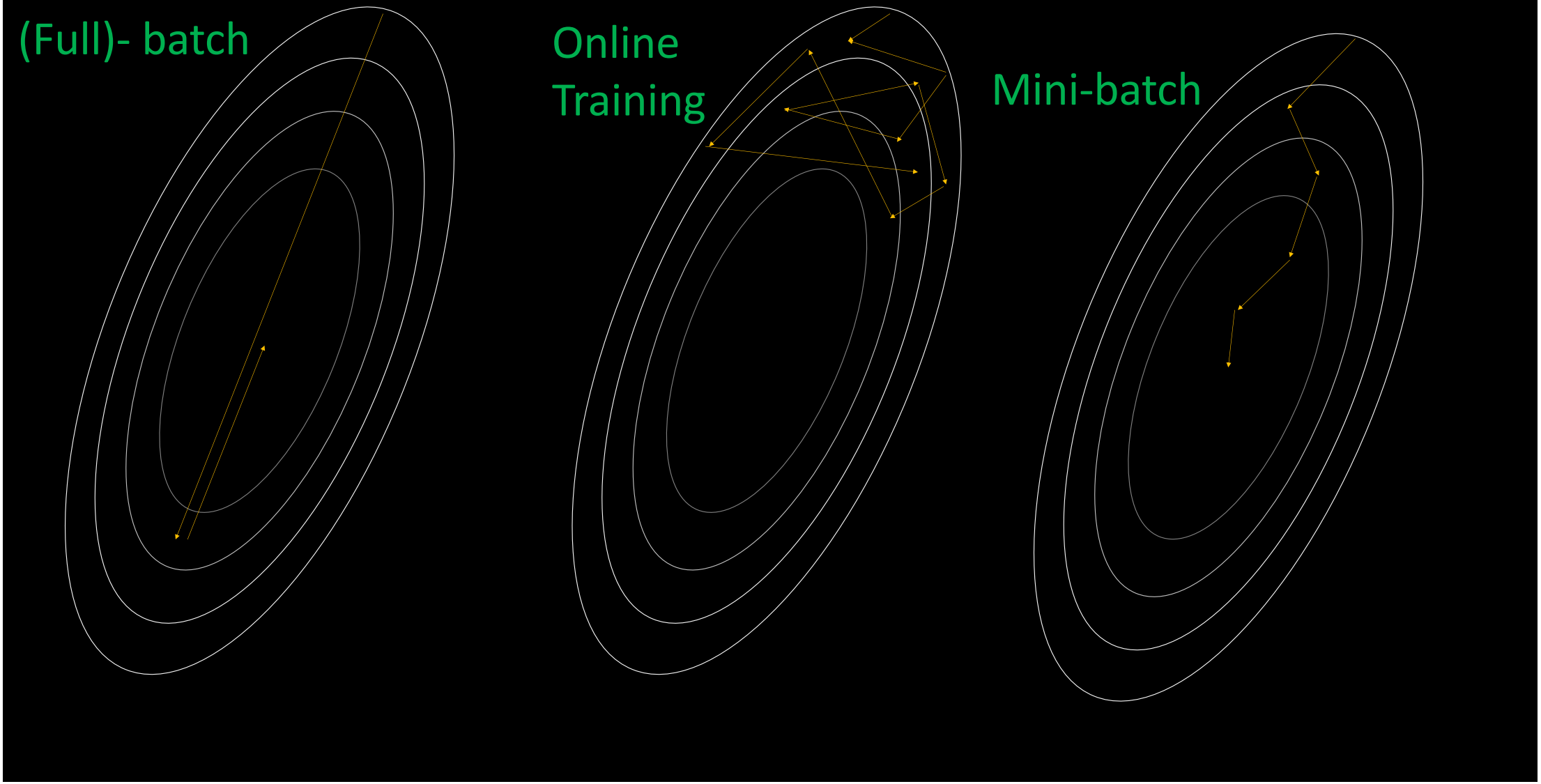


"tend to be crazy"

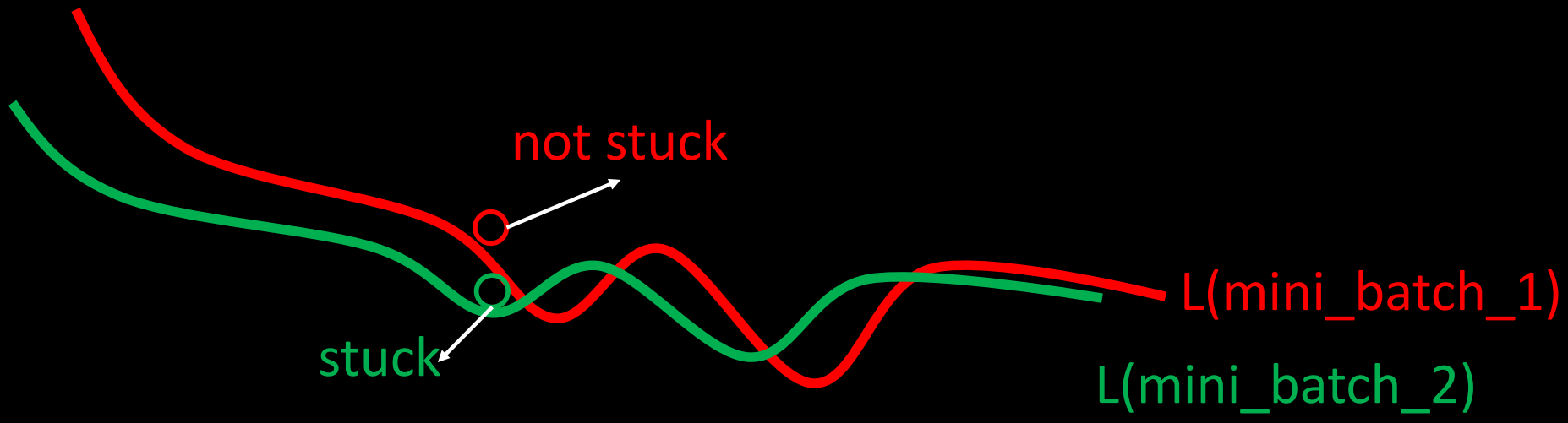
Training Neural Networks: minibatch

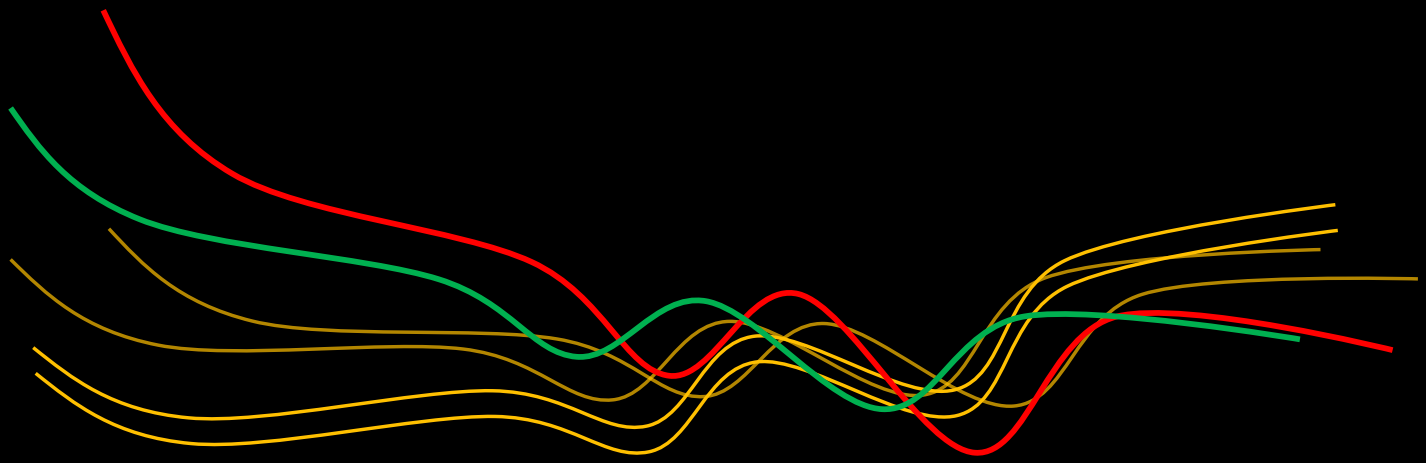


Training Neural Networks: minibatch

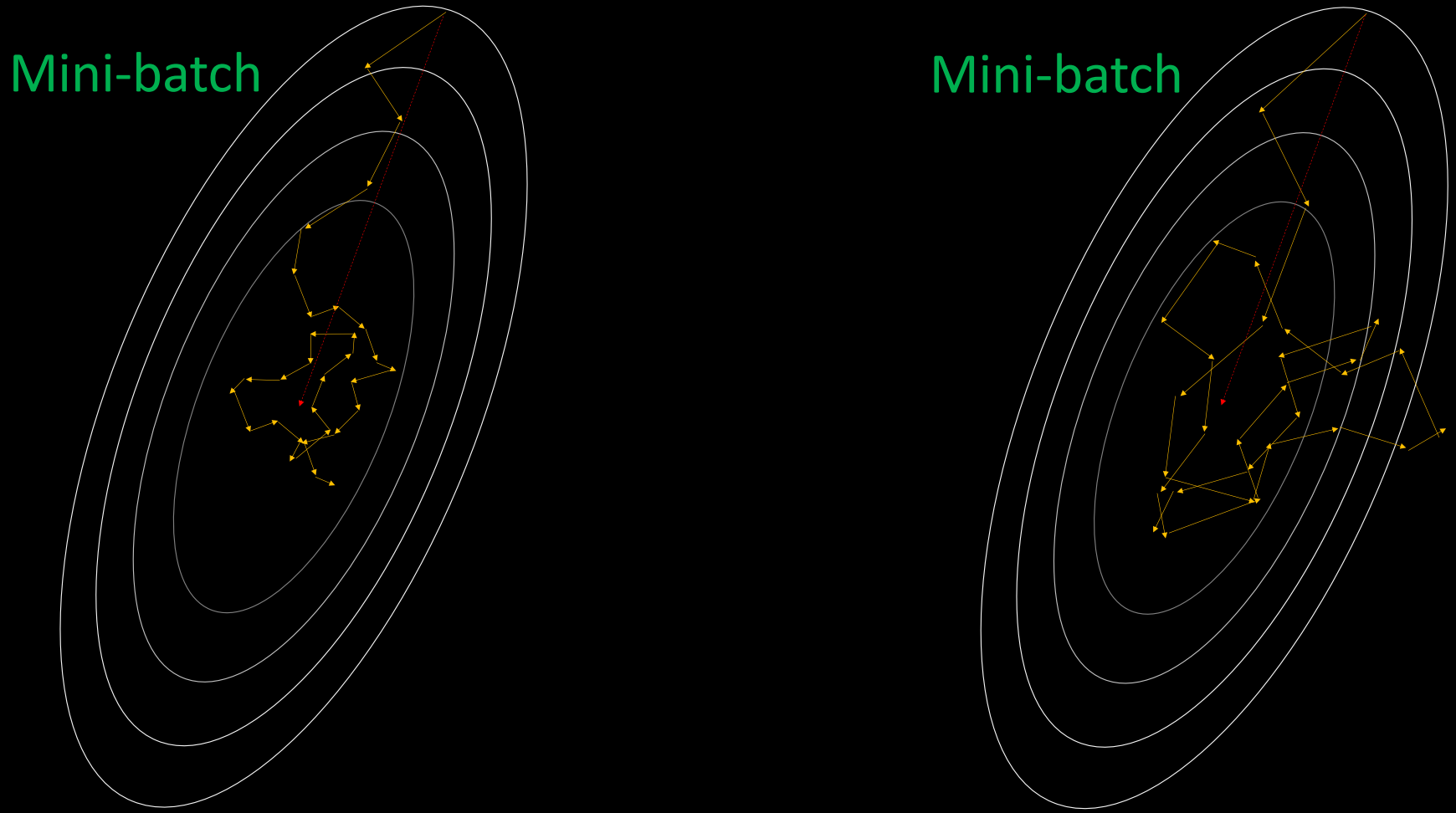


Training Neural Networks: minibatch



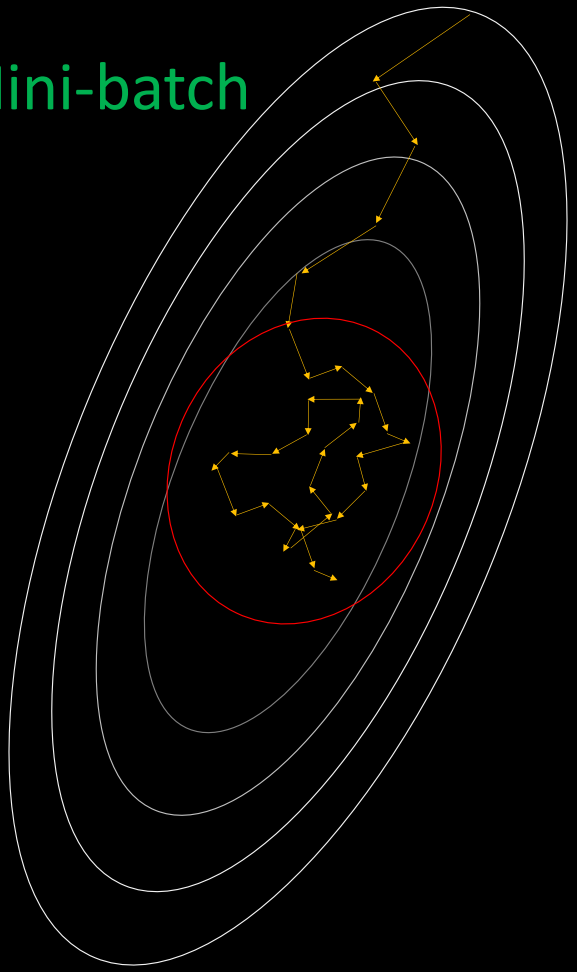


Training Neural Networks: minibatch



Training Neural Networks: minibatch

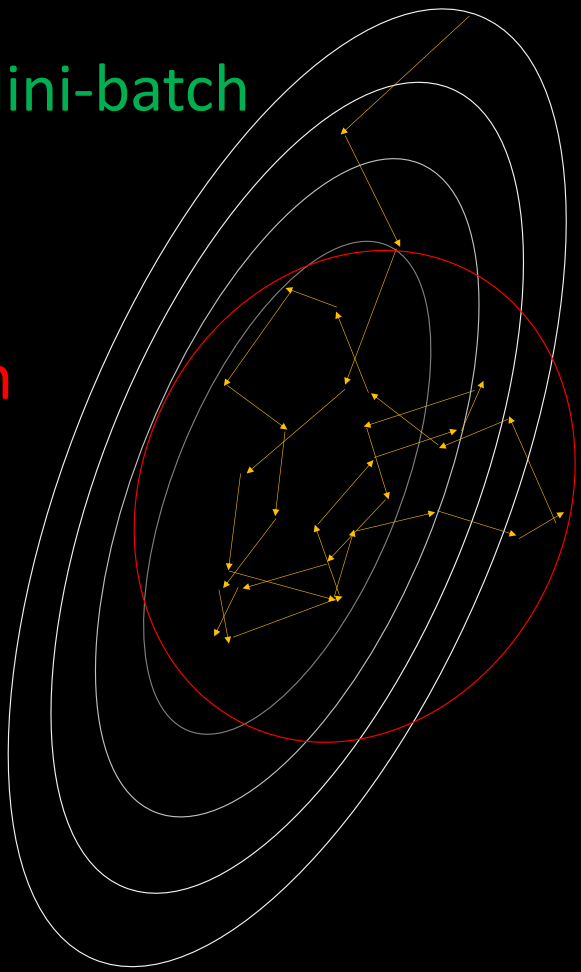
Mini-batch



Area of Confusion

(sometimes a
good thing
For ML!)

Mini-batch

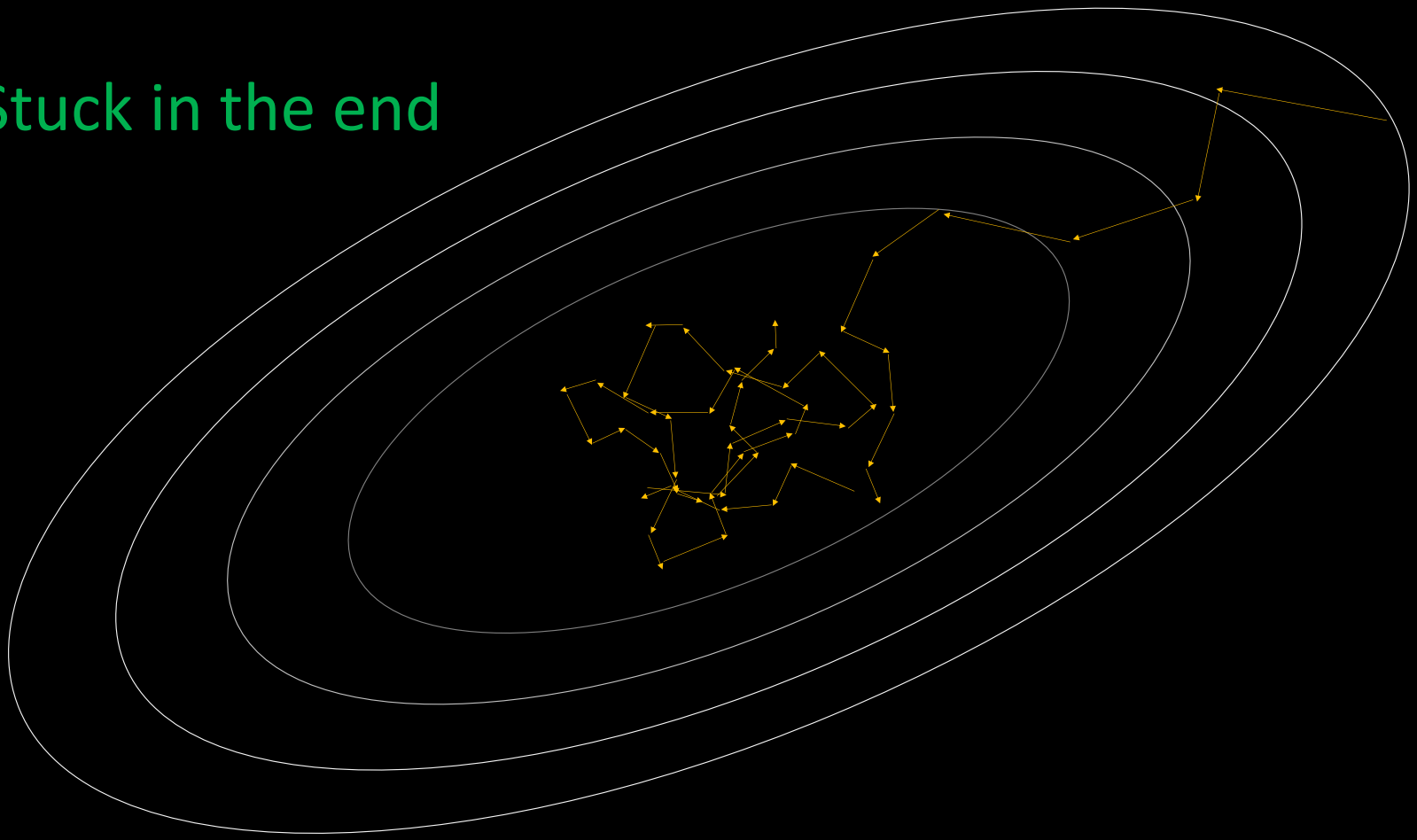


One example of modern deep learning training

	Hardware	Chips	Batch	Optimizer	BN	Accuracy	Time
Goyal et al. [6]	P100	256	8192	Momentum	Local	76.3%	1 hour
Smith et al. [16]	TPU v2	128	8192 → 16384	Momentum	Local	76.1%	30 mins.
Akiba et al. [2]	P100	1024	32768	RMS + Mom.	Local	74.9%	15 mins.
Jia et al. [10]	P40	1024	65536	LARS	Local	76.2%	8.7 mins.
Baseline	TPU v2	4	1024	Momentum	Local	76.3%	8.0 hours
Ours	TPU v2	256	16384	Momentum	Local	75.1%	10 mins.
Ours	TPU v2	256	32768	LARS	Local	76.3%	8.5 mins.
Ours	TPU v3	512	32768	LARS	Local	76.4%	3.3 mins.
Ours	TPU v3	1024	32768	LARS	Distributed	76.3%	**2.2 mins.**

Fast Progress in the Beginning

Crazy / Stuck in the end



Good for big data!

All Data

0	1	2	3	4	5	6	7	8	9

Random Shuffle

8	3	2	7	0	1	6	5	4	9

Sample without replacement…

8	3	2	7	0	1	6	5	4	9

Sample with replacement…

3	2	7	2	1	9	8	1	4	3

There are tones of research go into
how to fix crazy/stuck in the end

(not always matters in practice)

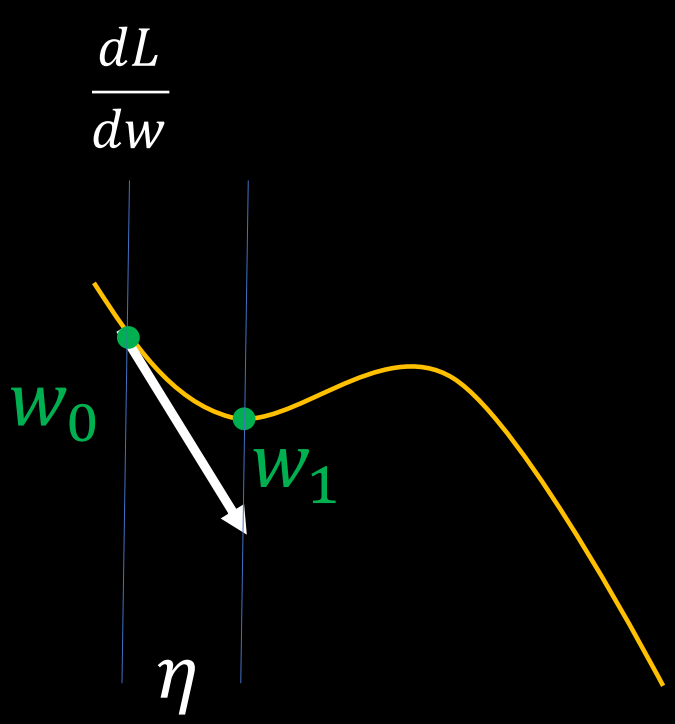
TIPS

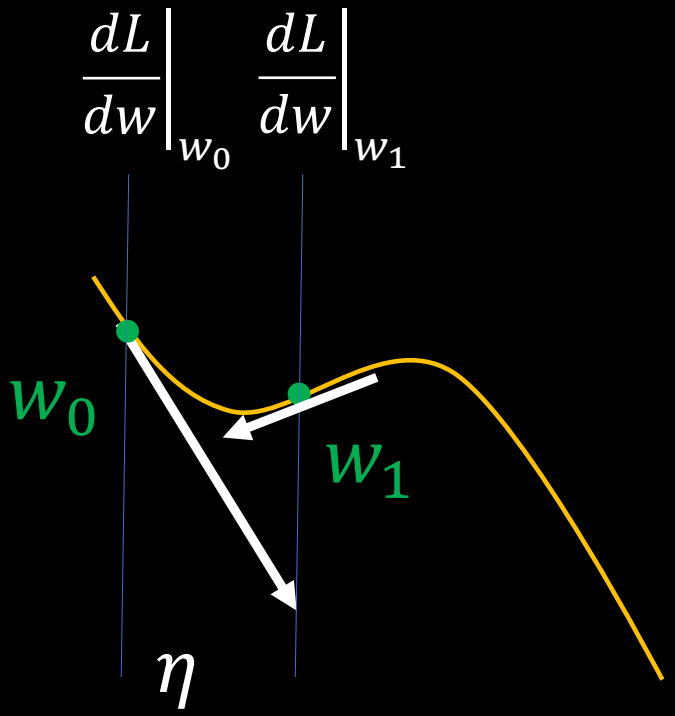
- schemes of gradient calculation

- find the right learning rate to start

- find the good way to change learning rate during training

Training Neural Networks: minibatch





$$w_1 = w_0 - \frac{dL}{dw}\eta$$

$$w_1 = w_0 - \left(\left.\frac{dL}{dw}\right|_0 + \beta \left.\frac{dL}{dw}\right|_{-1}\right)\eta$$

Momentum

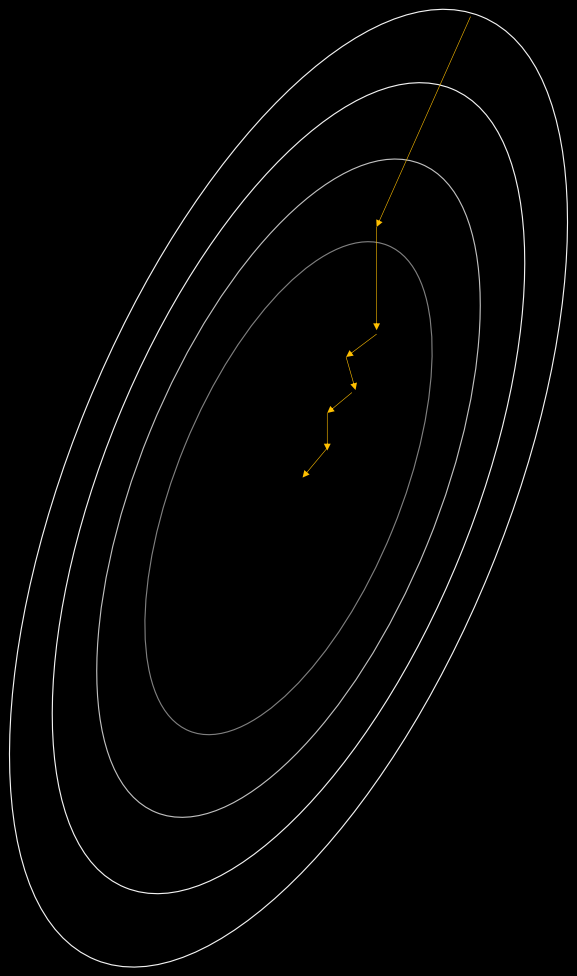

gettyimages
Credit: Ashley Jouhar
82214105



Adagrad (adaptive)

$$w_1 = w_0 - \eta \left.\frac{dL}{dw}\right|_0 \epsilon$$

$$\epsilon = \frac{1}{\sqrt{\sum_0^t (\frac{dL}{dw})^2 + \varepsilon}}$$



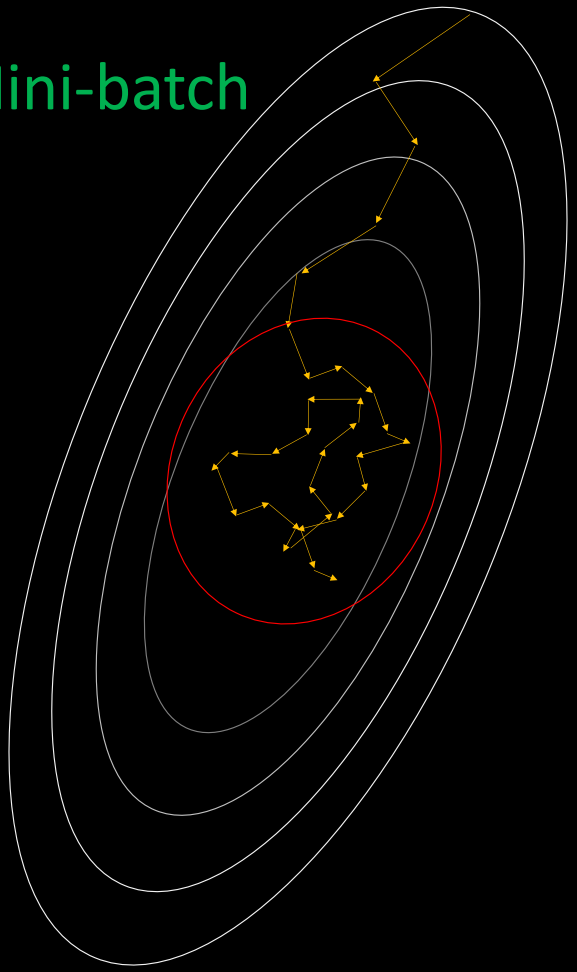
Adams

$$w_1 = w_0 - \eta \left(\left.\frac{dL}{dw}\right|_0 + \beta \left.\frac{dL}{dw}\right|_{-1}\right)\epsilon$$

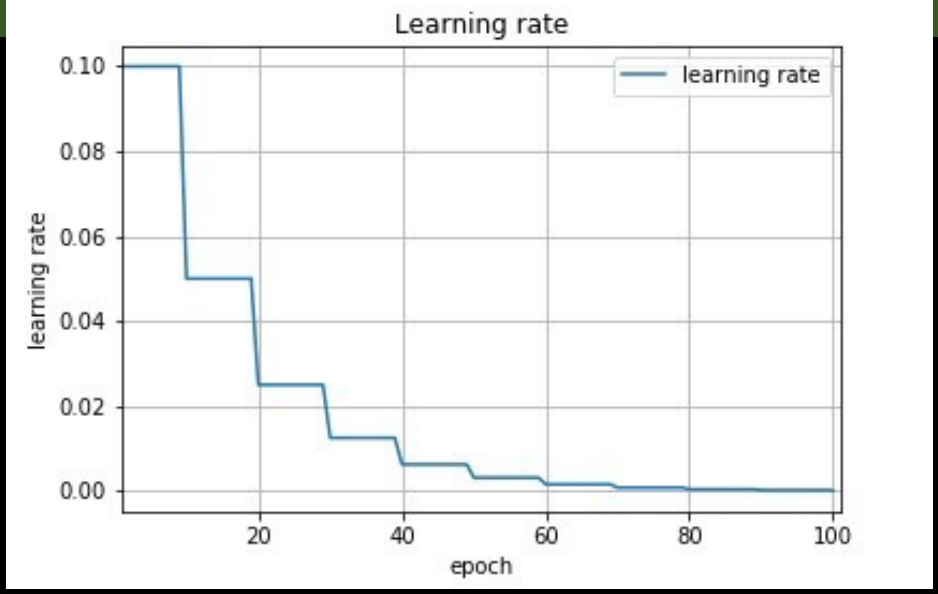
Momentum Ada

Training Neural Networks: minibatch

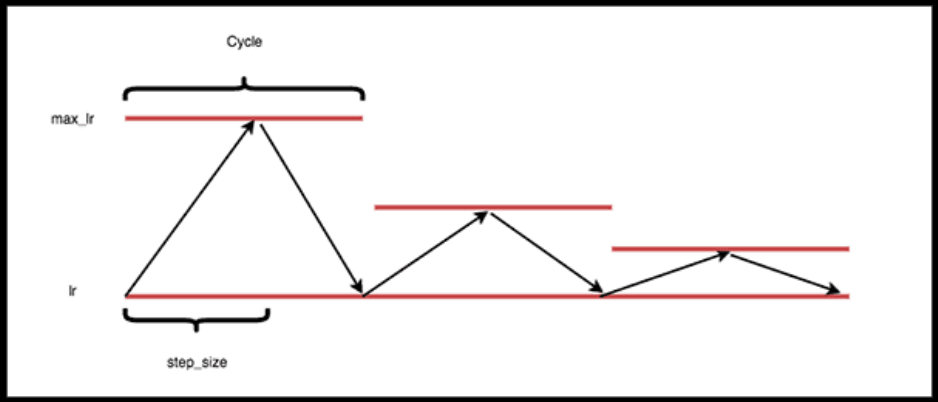
Mini-batch



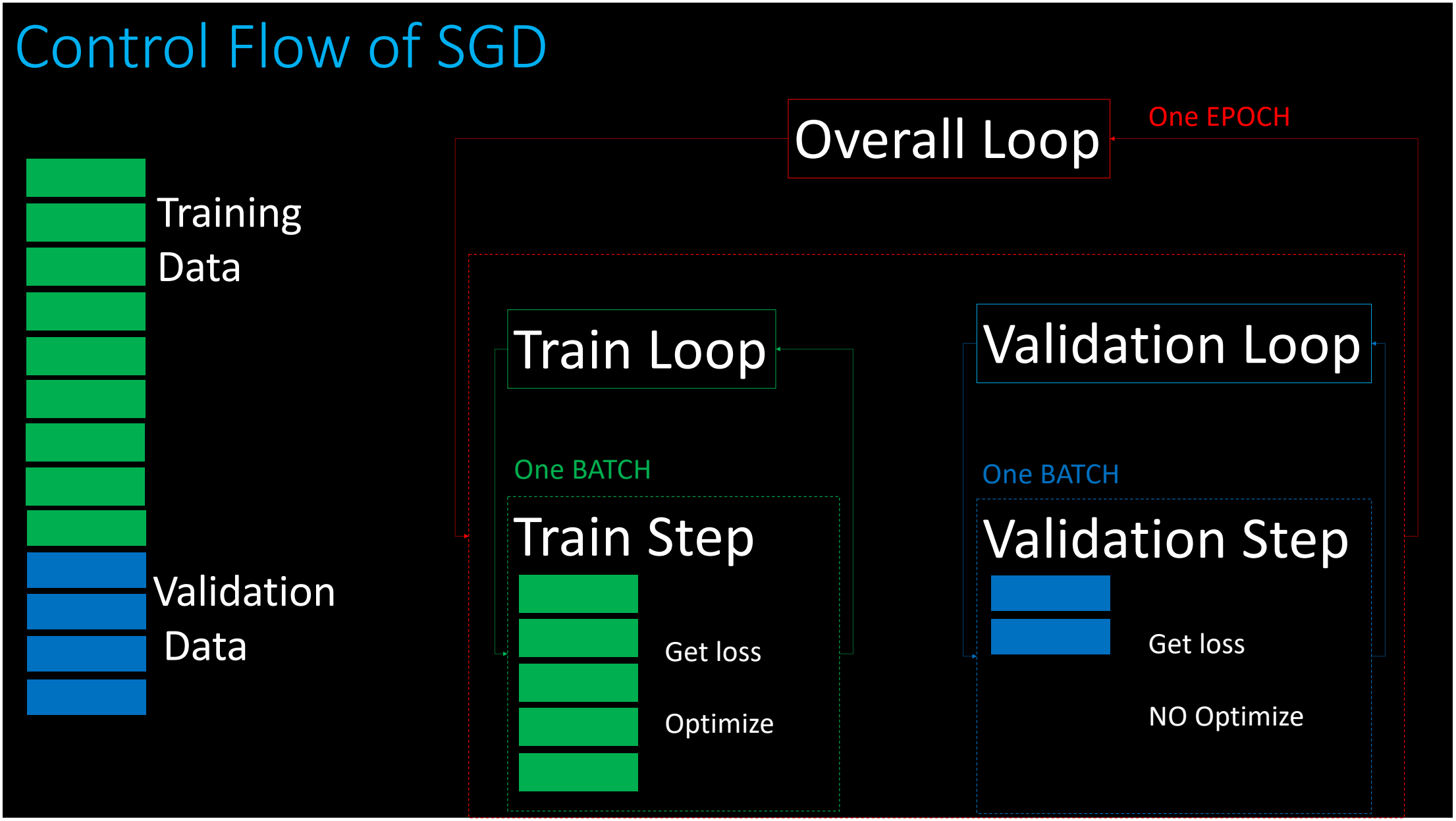
Learning rate decay



Cyclic learning rate



Control Flow of SGD



Control Flow of SGD

Overall Loop

For epoch = 1…..n_epoch:

Train Loop

for batch = 1 ….. n_batch_training:

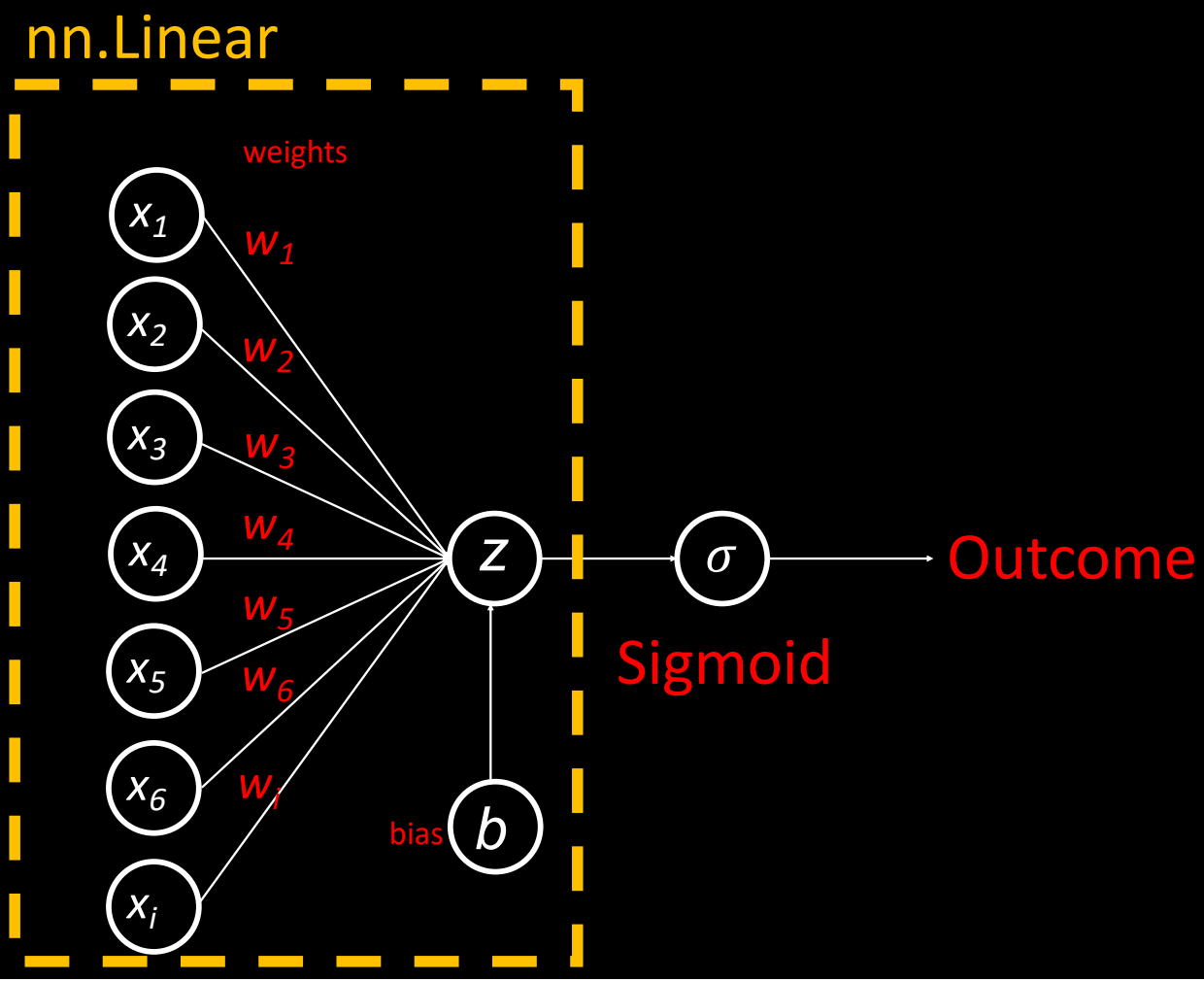
do training_step()
do optimize()

Validation Loop

for batch = 1 ….. n_batch_validation:

do validation_step()

Graph of Logistic Regression



```
>>> import torch
>>> import torch.nn as nn

>>> model = nn.Linear(100, 1)
>>> model

Linear(in_features=100, out_features=1, bias=True)
```

torch: basic things.
torch.nn: neural network components

a 100 features to 1 class linear model

```
>>> one_case = torch.rand(1, 100)
>>> print(one_case.shape)

torch.Size([1, 100])

>>> print(model(one_case).shape)

torch.Size([1, 1])
```

One data with 100 features

One data create one output

```
>>> three_case = torch.rand(3, 100)
>>> print(three_case.shape)

torch.Size([3, 100])

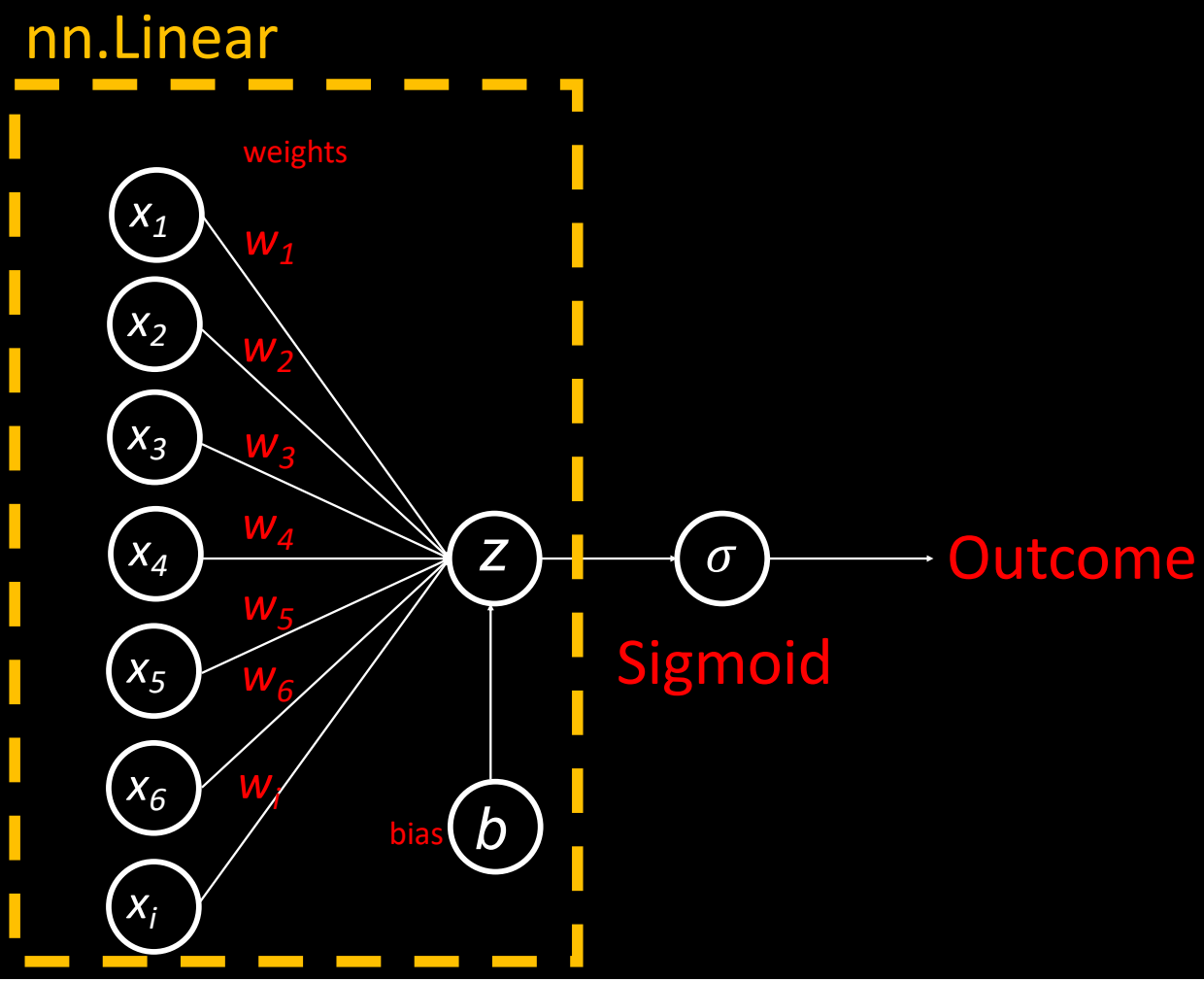
>>> print(model(three_case).shape)

torch.Size([3, 1])
```

three data with 100 features

three data create three output

Graph of Logistic Regression



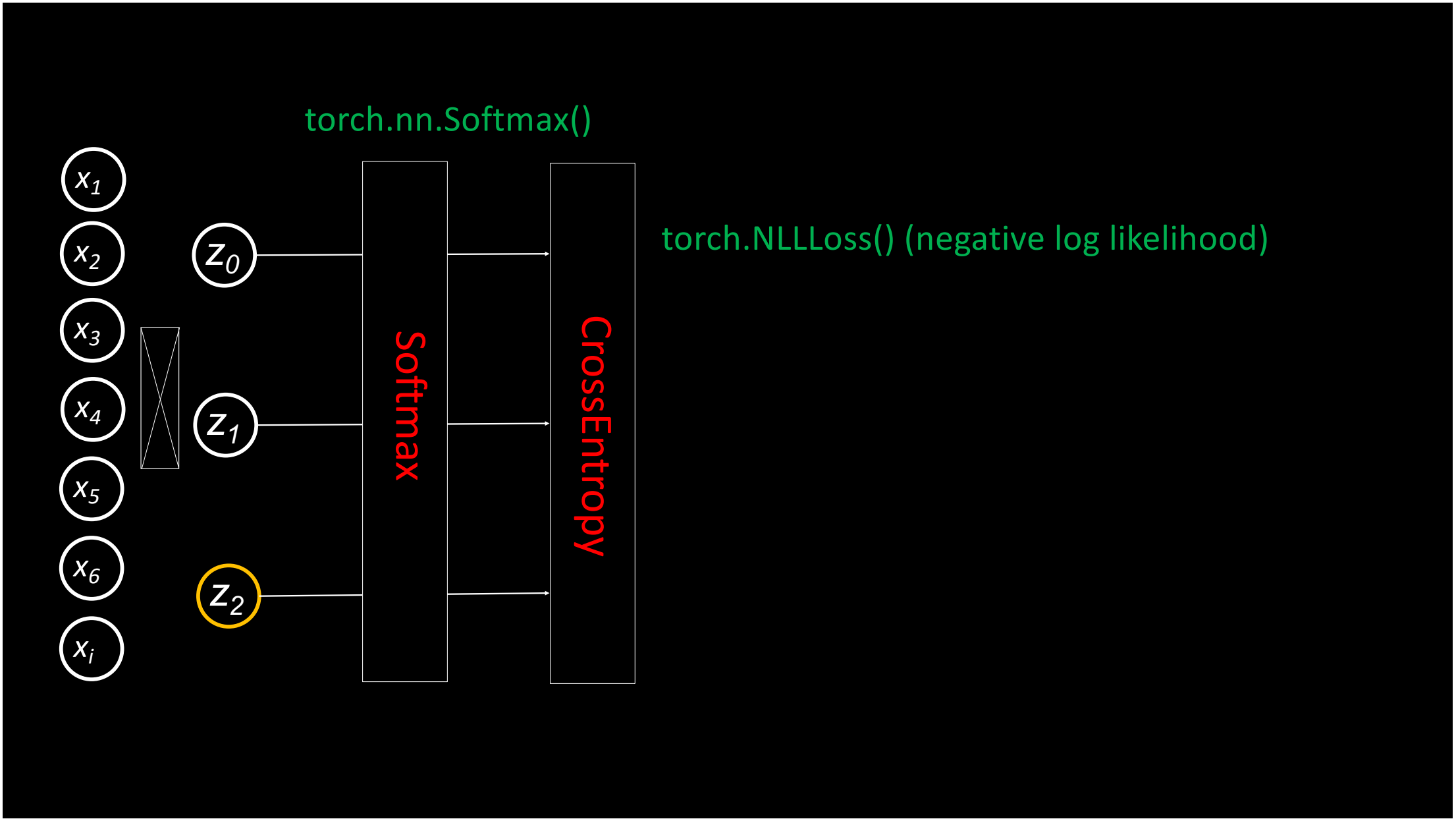
```
# Logistic regression model
model = nn.Linear(args['img_size'], args['num_classes'])
                          784 = 28 X 28        10

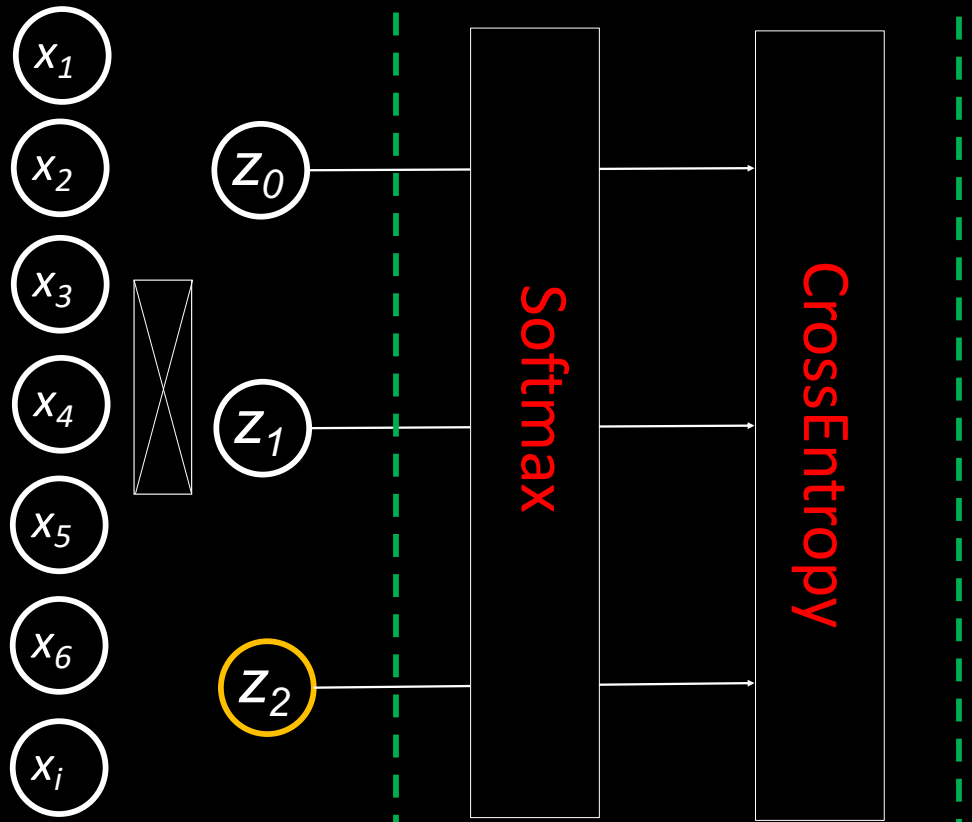

model
Linear(in_features=784, out_features=10, bias=True)

model.weight.shape
torch.Size([10, 784])

model.bias.shape
torch.Size([10])

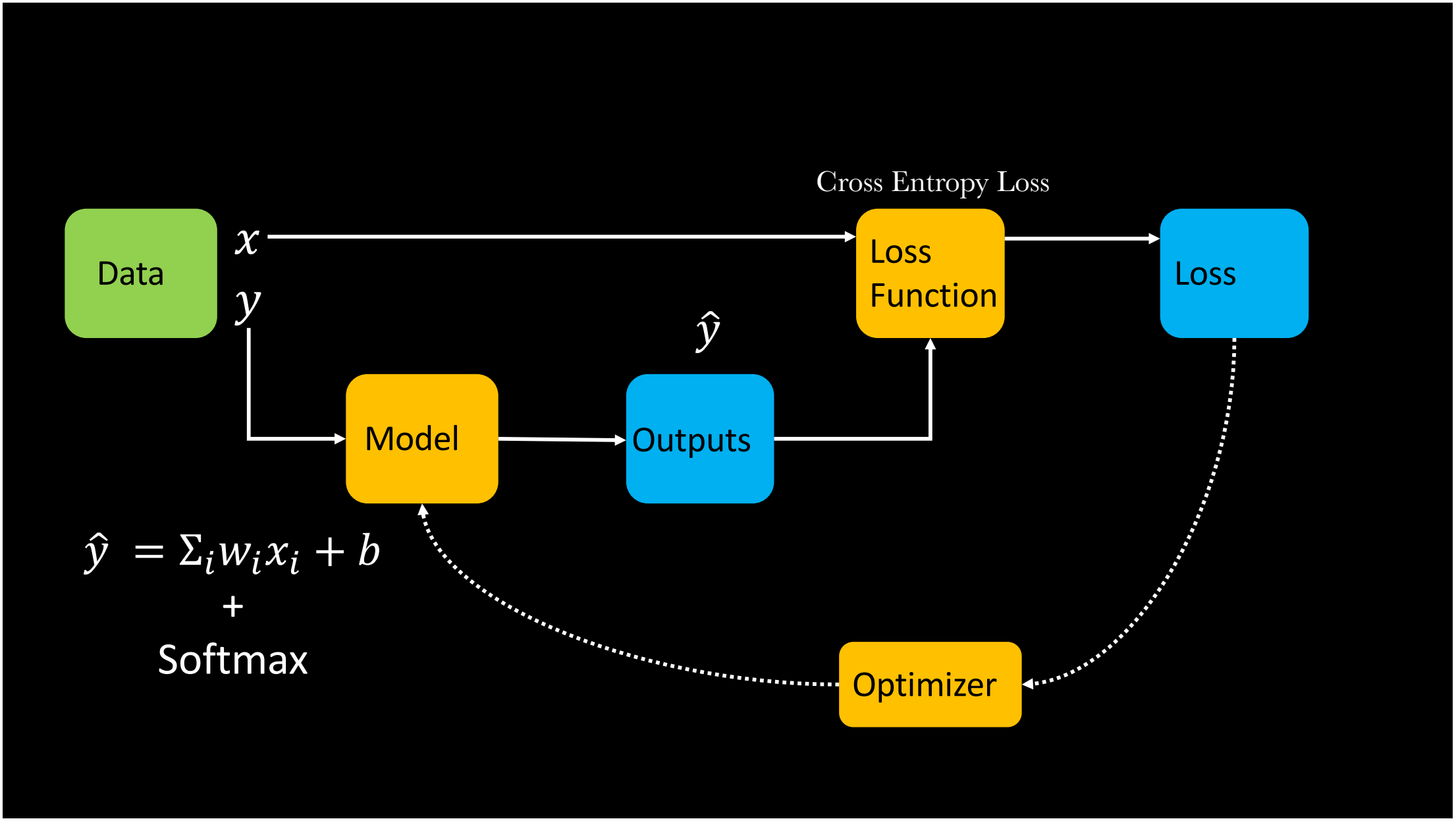
model(torch.rand(3, 28*28)).shape
torch.Size([3, 10])
```


torch.nn.Softmax()
x_1
x_2
x_3
x_4
x_5
x_6
x_i
z_0
z_1
z_2
Softmax
CrossEntropy
torch.NLLLoss() (negative log likelihood)




```
>>> loss = nn.CrossEntropyLoss()
>>> label =  torch.LongTensor([0])
>>> loss(torch.FloatTensor([5, 1, -2]).unsqueeze(0), label)
tensor(0.0190)

>>> loss(torch.FloatTensor([1, 5, -2]).unsqueeze(0), label)
tensor(4.0190)
```


Cross Entropy Loss
Data
x
y
Loss Function
Loss
$\hat{y}$
Model
Outputs
$\hat{y} = \Sigma_i w_i x_i + b$
+
Softmax
Optimizer

```
# Hyper-parameters
args = {'img_size': 28 * 28,

        'num_classes': 10,

        'num_epochs': 50,          How many training steps until finished?

        'batch_size’: 16,          How many cases the model see each time?

        'learning_rate': 0.001}    How fast we are moving in training?
```

```
# MNIST dataset (images and labels)
train_dataset = torchvision.datasets.MNIST(root='../../data',
                                           train=True,
                                           transform=transforms.ToTensor(),
                                           download=True)

test_dataset = torchvision.datasets.MNIST(root='../../data',
                                          train=False,
                                          transform=transforms.ToTensor())
```

Location

Training or testing

Download if not avalible

```
len(train_dataset)
60000
x = train_dataset.__getitem__(10)
len(x)
2
x[0].shape
torch.Size([1, 28, 28])
x[1]
3
```

```
# Data loader (input pipeline)
train_loader =
torch.utils.data.DataLoader(dataset=train_dataset,        Using training data
                            batch_size=args['batch_size'],
                             shuffle=True)

test_loader =
torch.utils.data.DataLoader(dataset=test_dataset,
                            batch_size=args['batch_size'],
                            shuffle=False)



len(train_loader)
3750
i, x=next(enumerate(train_loader))
x[0].shape
torch.Size([16, 1, 28, 28])
x[1]
tensor([7, 5, 1, 0, 8, 7, 4, 5, 4, 1, 2, 6, 1, 6, 1, 2])
```

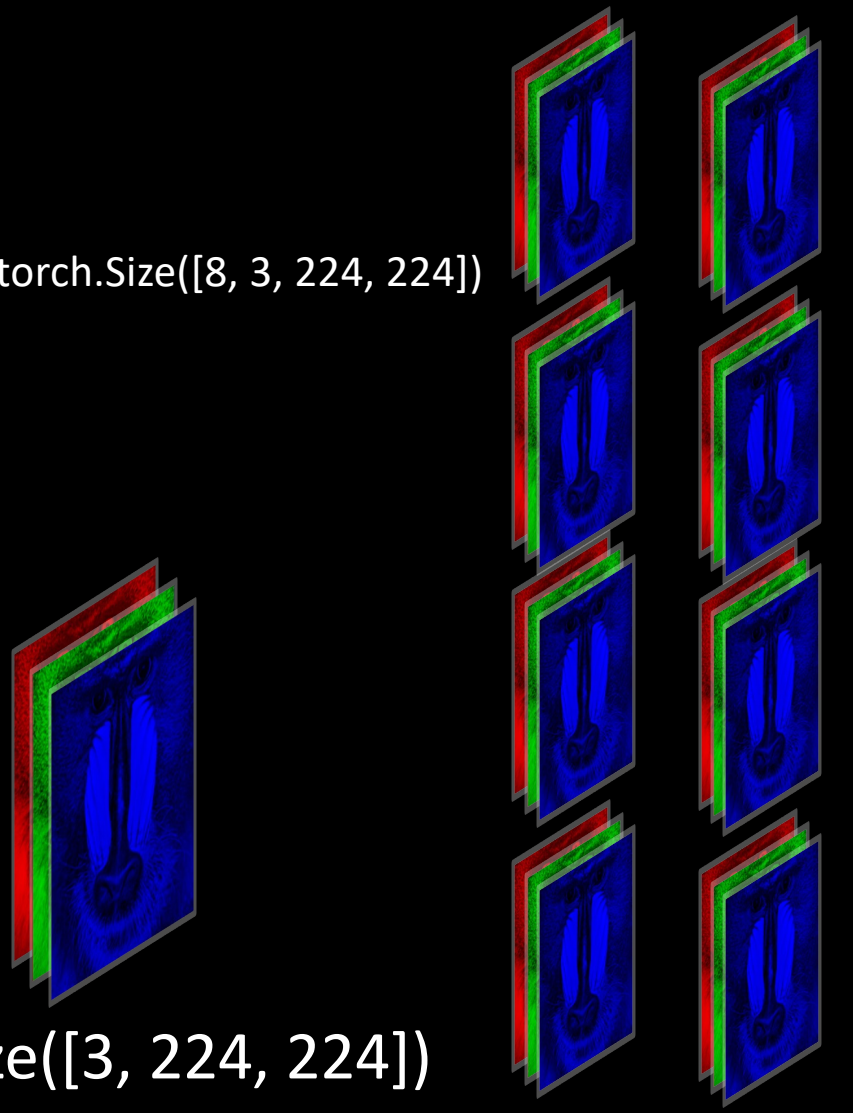
Tensor Shapes

```
>>> torch.from_numpy(np.ones((8, 3, 224, 224)).astype(np.float32)).shape
torch.Size([8, 3, 224, 224])
```

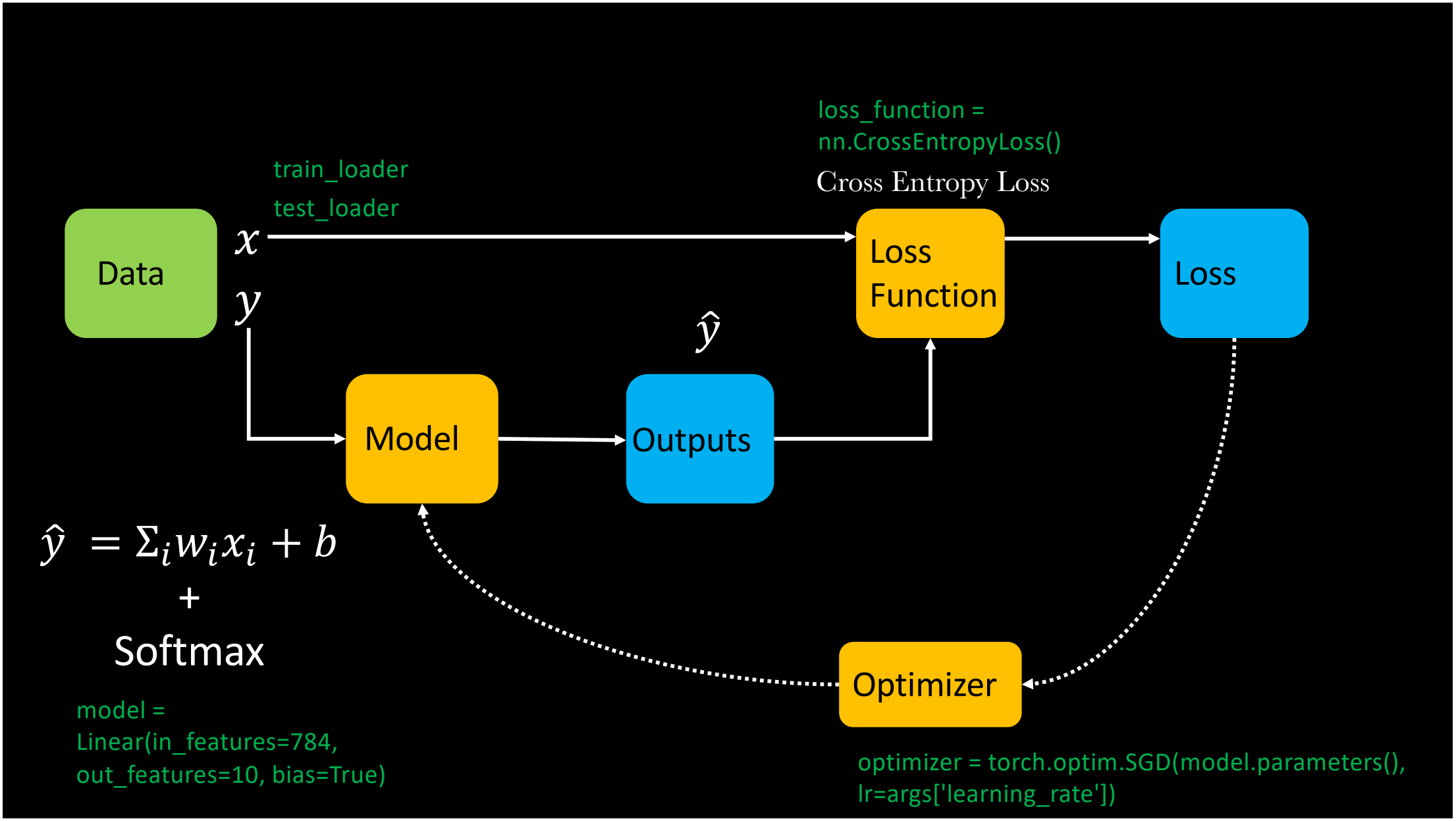
X.shape
torch.Size([8, 3, 224, 224])

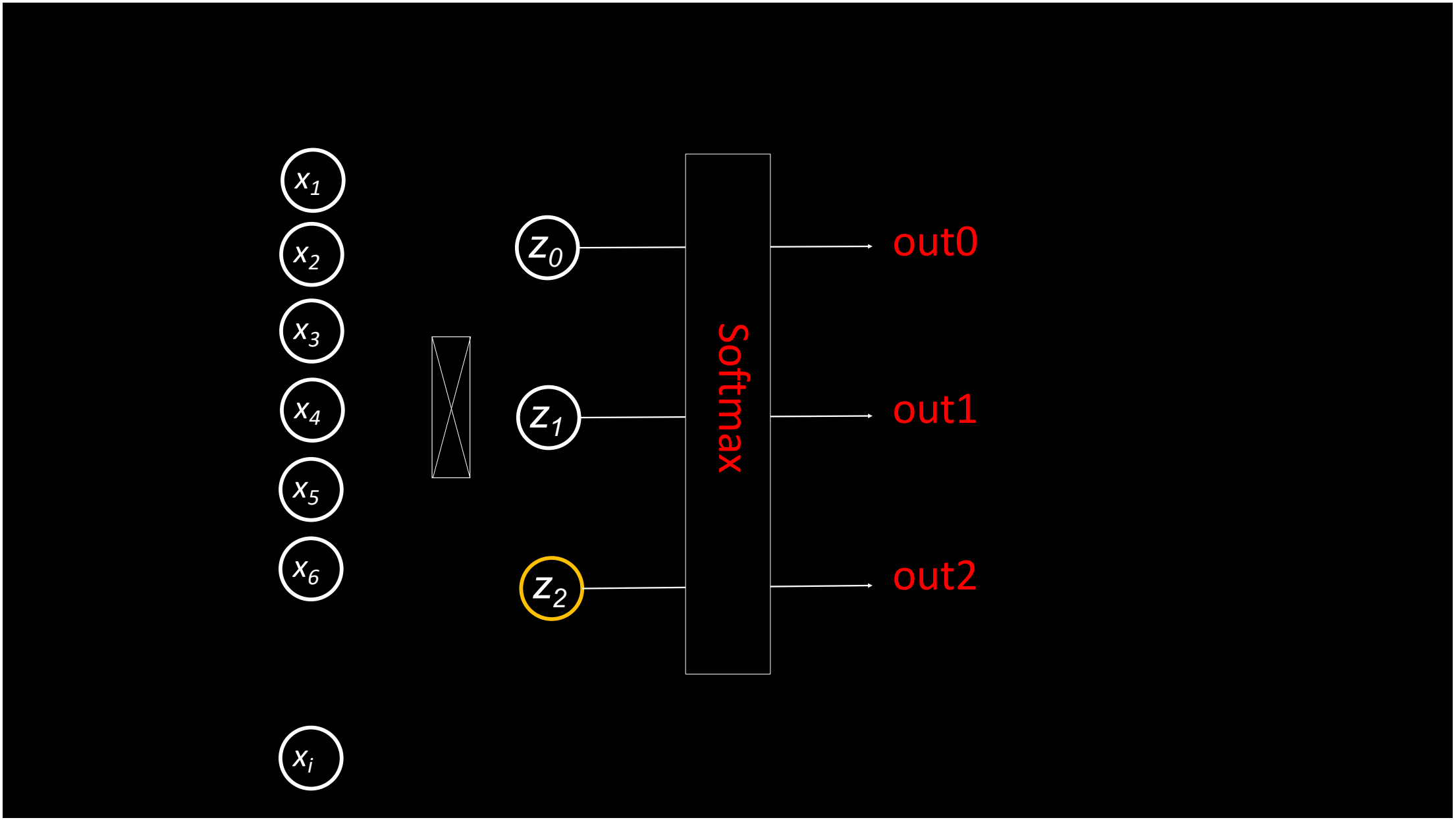
Batch size
Channel #
Height
Width

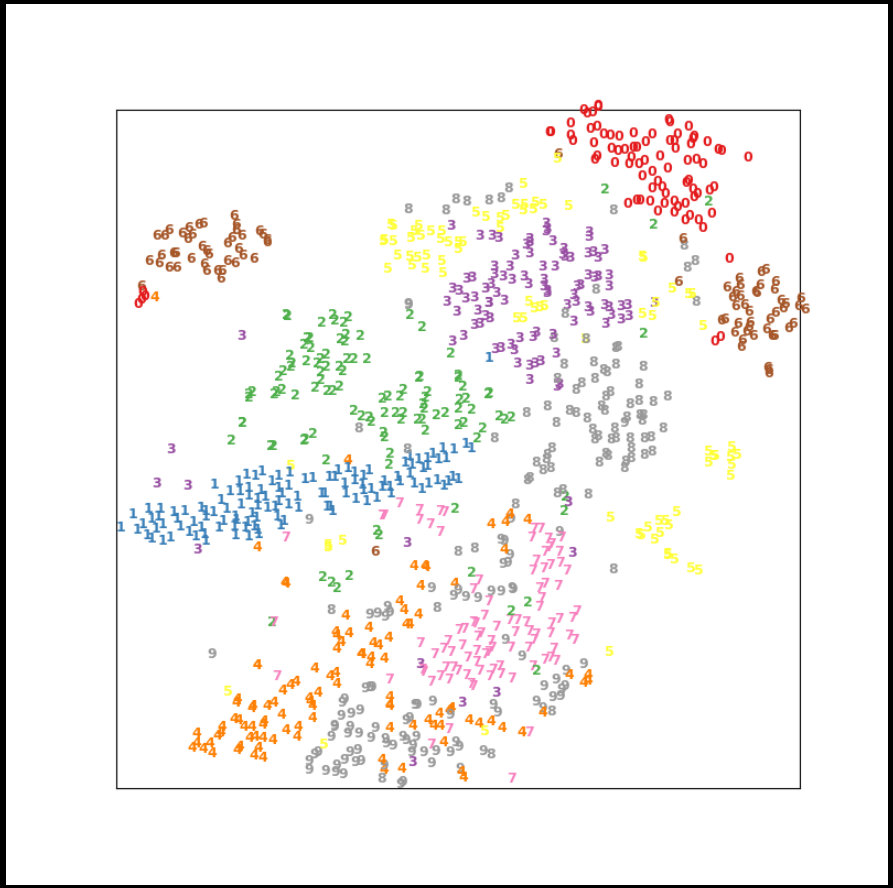
torch.Size([8, 3, 224, 224])



torch.Size([3, 224, 224])


loss_function =
nn.CrossEntropyLoss()
Cross Entropy Loss
train_loader
test_loader
Data
x
y
Loss Function
Loss
$\hat{y}$
Model
Outputs
$\hat{y} = \Sigma_i w_i x_i + b$
+
Softmax
Optimizer
model =
Linear(in_features=784,
out_features=10, bias=True)
optimizer = torch.optim.SGD(model.parameters(),
lr=args['learning_rate'])


x_1
x_2
x_3
x_4
x_5
x_6
x_i
z_0
z_1
z_2
Softmax
out0
out1
out2

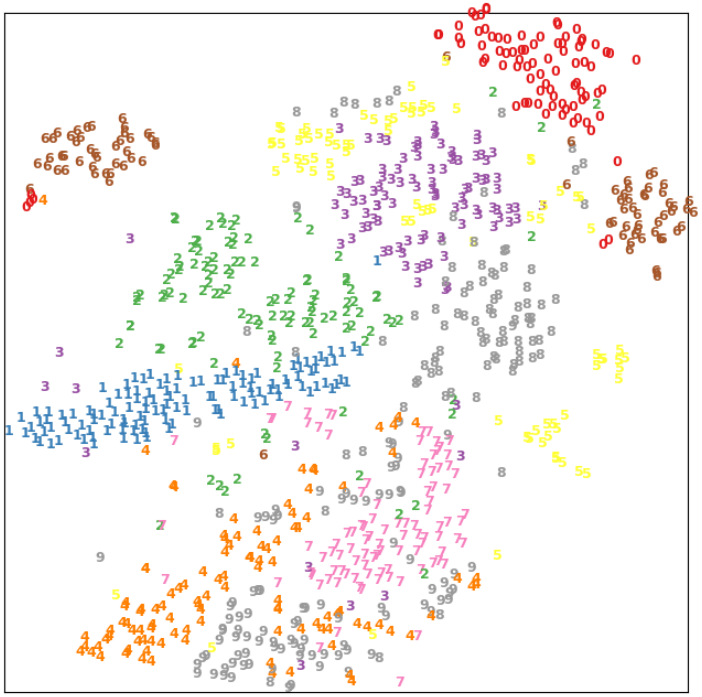


Visualize the results by tsne

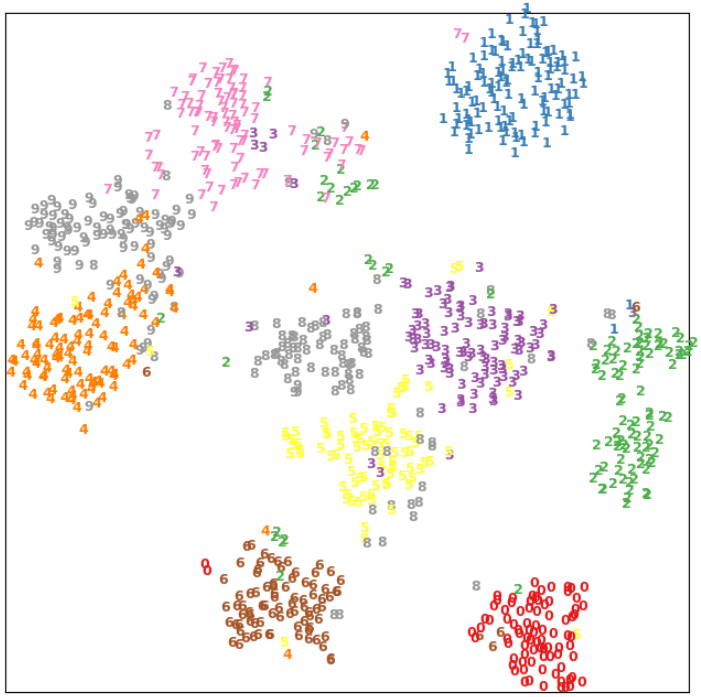
784

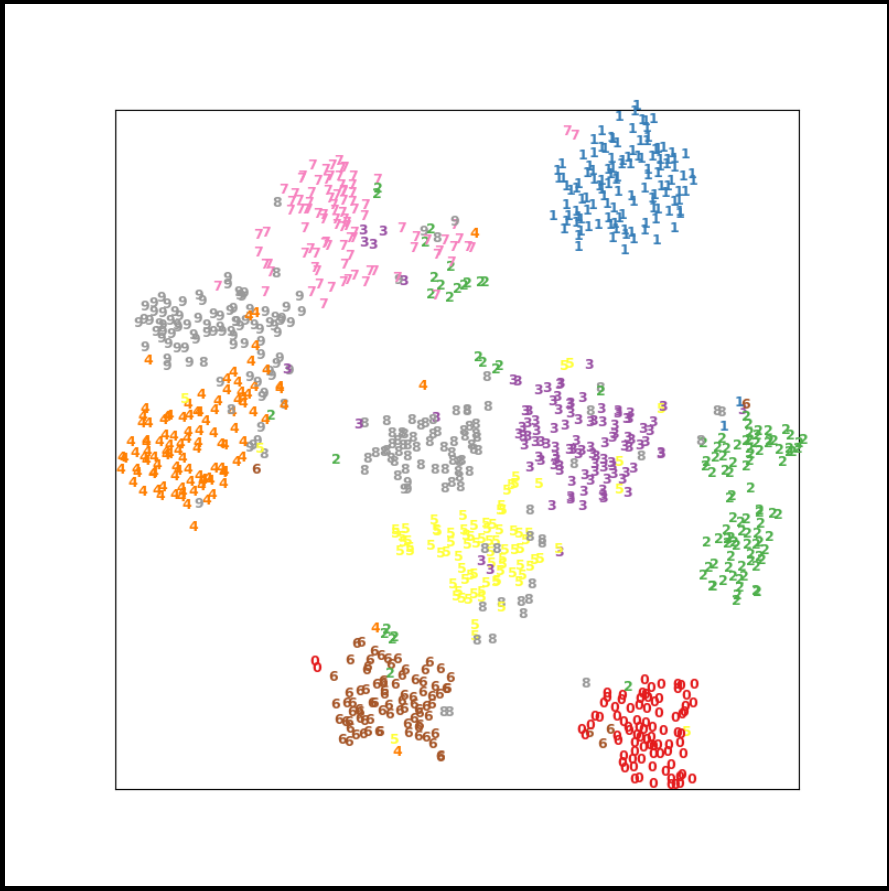
10

Before logistic regression



After logistic regression





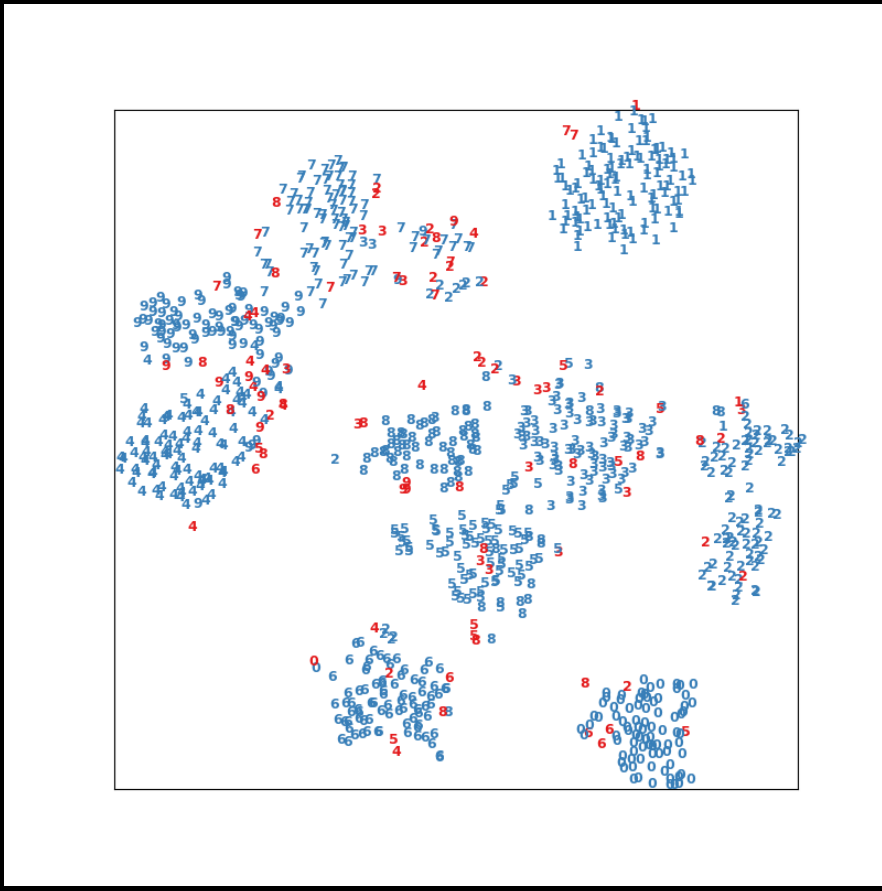
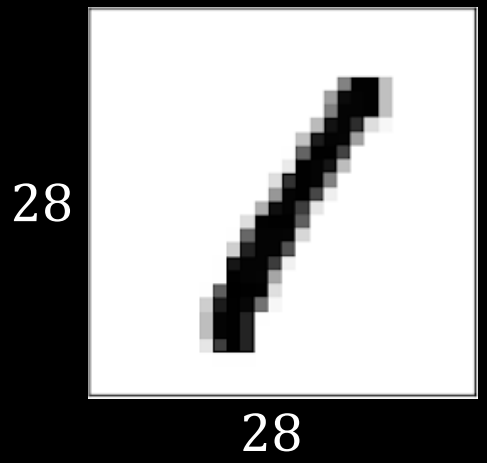


Image Classification by Logistic Regression



MNIST dataset



LARXEL · UPDATED 4 YEARS AGO

125

New Notebook

Download (89 MB)

Medical MNIST

58954 medical images of 6 classes



Data Card Code (41) Discussion (0) Suggestions (0)

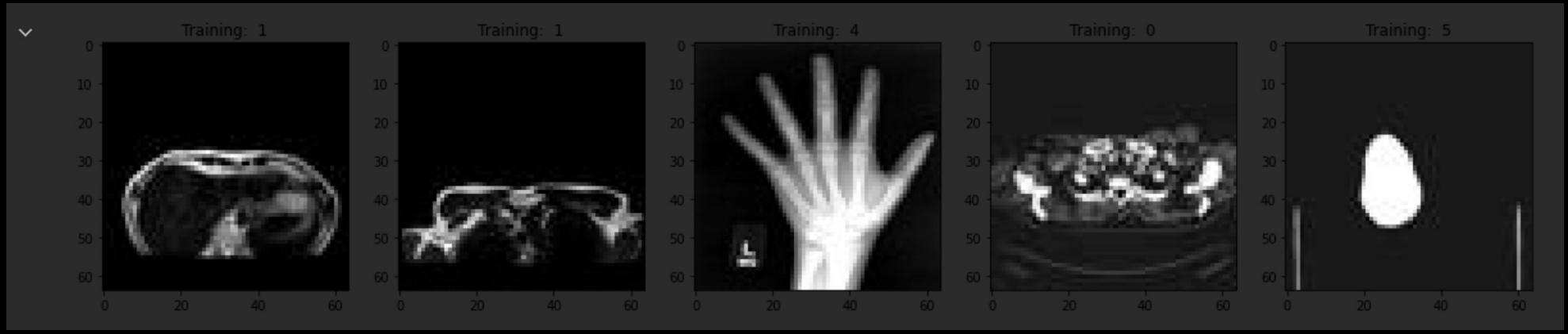
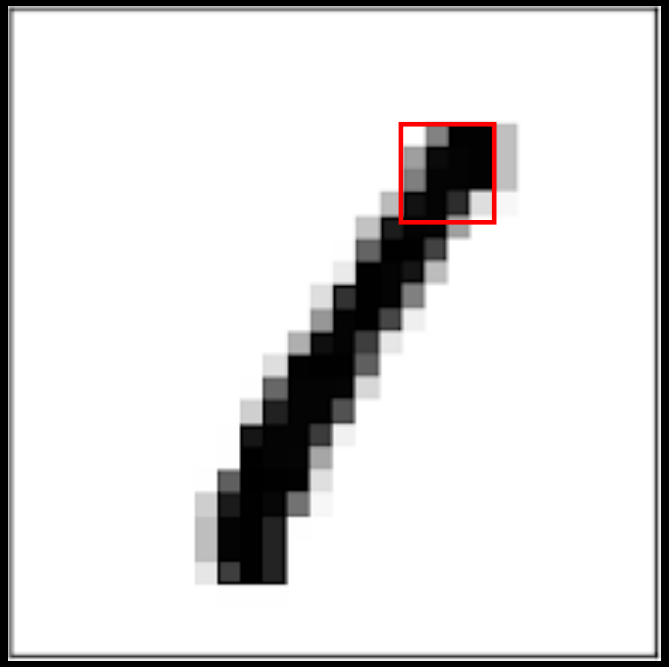
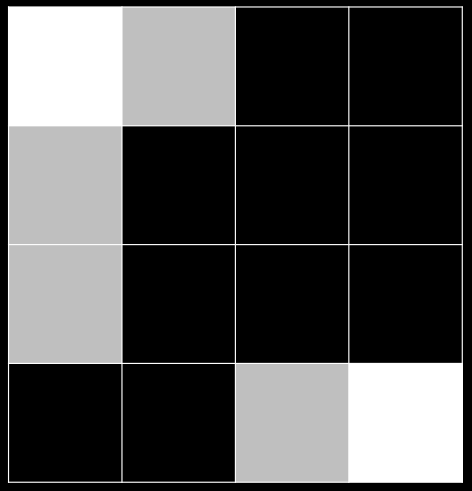


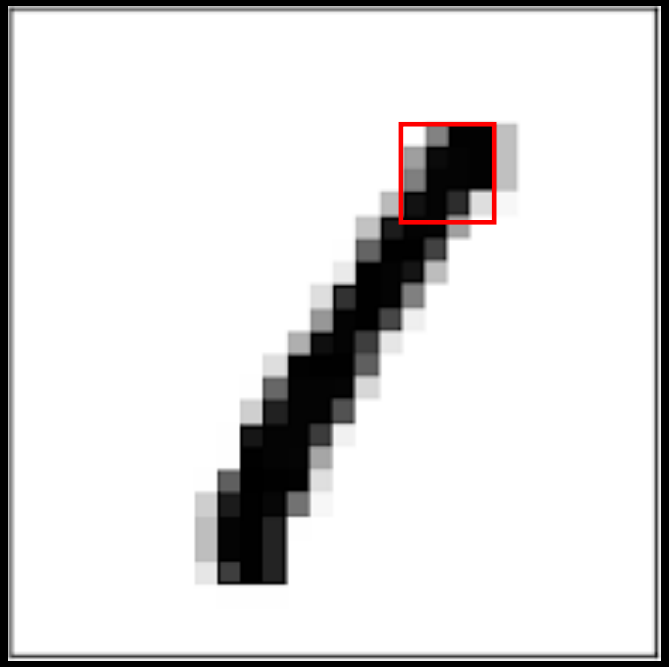
Image to grayscale pixel value





1	0.5	0	0
0.5	0	0	0
0.5	0	0	0
0	0	0.5	1

Image to grayscale pixel value

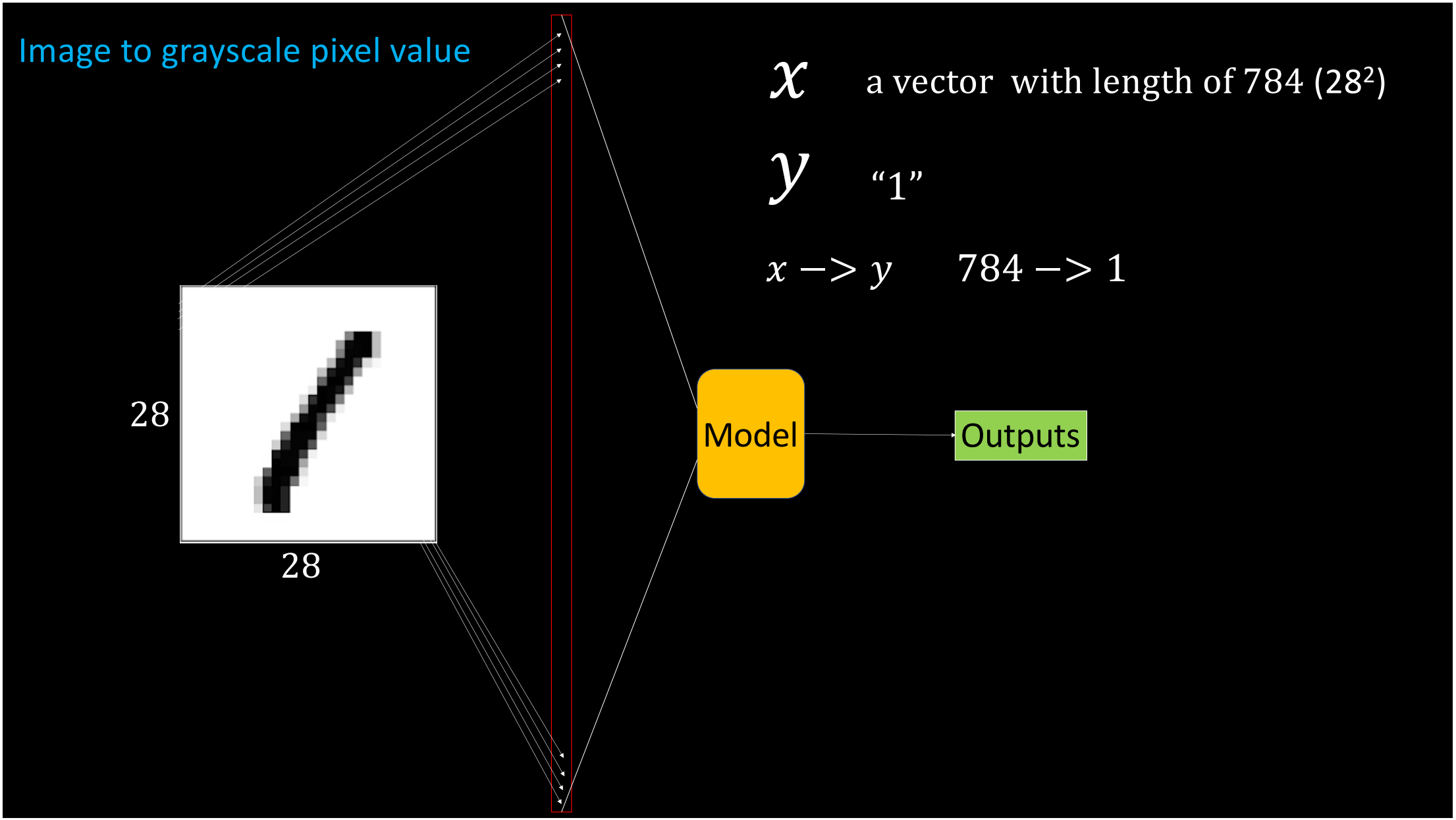


1	0.5	0	0
0.5	0	0	0
0.5	0	0	0
0	0	0.5	1

1
0.5
0
0
0.5
0
0
0
0.

numpy.ndarray.flatten

torch.array.view(-1)


Image to grayscale pixel value
28
28
x a vector with length of 784 (28^2)
y "1"
$x -> y$ $784 -> 1$
Model
Outputs

```
First, download medical_mnist.zip from Kaggle
https://www.kaggle.com/datasets/andrewmvd/medical-mnist
or from dropbox
https://www.dropbox.com/scl/fi/puo3gd1wmnat5ggz2naih/mmnist
 .zip?rlkey=uhiyemgjg6xl67qt36fdkyuny&e=2&dl=0
unzip the data to Exercise2 / mmnist/
```



LARXEL · UPDATED 4 YEARS AGO

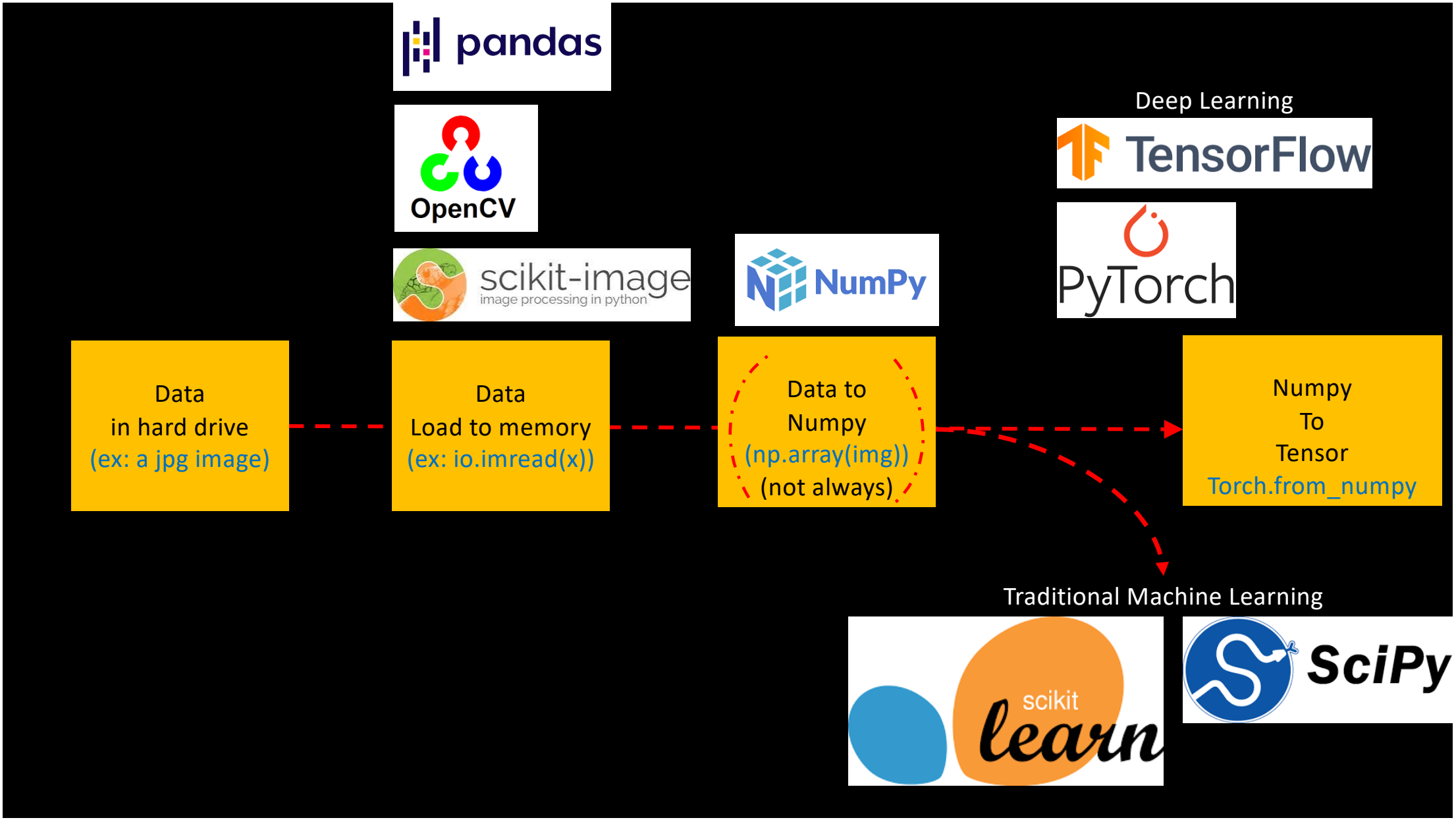
125 | New Notebook | Download (89 MB)

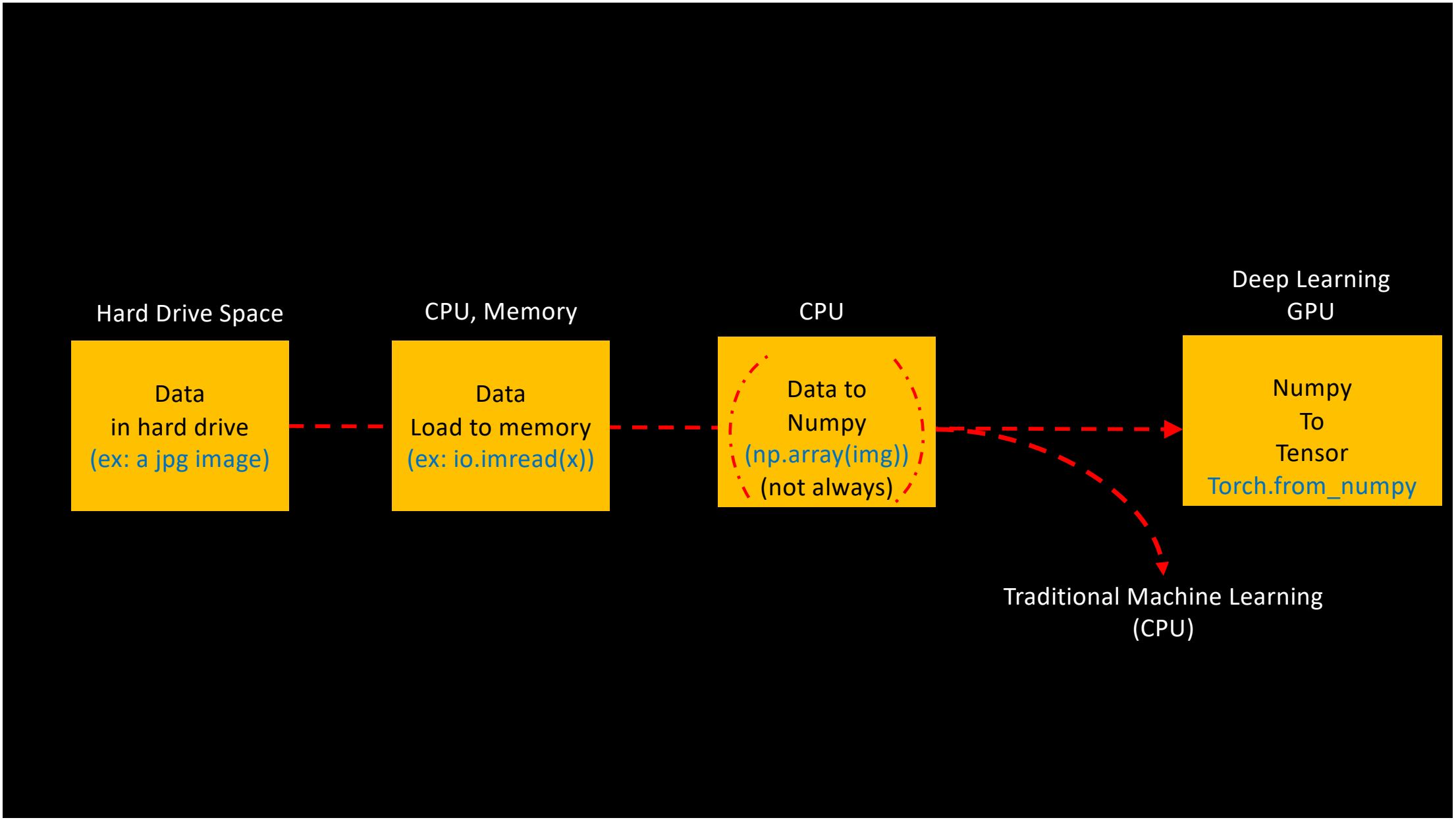
Medical MNIST

58954 medical images of 6 classes



Data Card | Code (41) | Discussion (0) | Suggestions (0)


pandas
OpenCV
scikit-image
image processing in python
NumPy
Deep Learning
TensorFlow
PyTorch
Data
in hard drive
(ex: a jpg image)
Data
Load to memory
(ex: io.imread(x))
Data to
Numpy
(np.array(img))
(not always)
Numpy
To
Tensor
Torch.from_numpy
Traditional Machine Learning
scikit
learn
SciPy


Hard Drive Space
Data
in hard drive
(ex: a jpg image)
CPU, Memory
Data
Load to memory
(ex: io.imread(x))
CPU
Data to
Numpy
(np.array(img))
(not always)
Deep Learning
GPU
Numpy
To
Tensor
Torch.from_numpy
Traditional Machine Learning
(CPU)

Pytorch Installation Pytorch.org

START LOCALLY

Select your preferences and run the install command. Stable represents the most currently tested and supported version of PyTorch. This should be suitable for many users. Preview is available if you want the latest, not fully tested and supported, 1.12 builds that are generated nightly. Please ensure that you have **met the prerequisites below (e.g., numpy)**, depending on your package manager. Anaconda is our recommended package manager since it installs all dependencies. You can also install previous versions of PyTorch. Note that LibTorch is only available for C++.

PyTorch Build	Stable (1.12.1)	Preview (Nightly)	LTS (1.8.2)	
Your OS	Linux	Mac	Windows	
Package	Conda	Pip	LibTorch	Source
Language	Python	C++ / Java		
Compute Platform	~~CUDA 10.2~~	~~CUDA 11.3~~	~~CUDA 11.6~~	~~ROCm 5.1.1~~	Default
Run this Command:	`conda install pytorch torchvision torchaudio -c pytorch`			

Data

```
# Medical MNIST dataset (images and labels)
train_loader, validation_loader = get_medical_mnist(args=args)
print('Done with data preparation')
```

```
data folder:  mmnist/
length of all images: 42000
length of all images: 16954
Done with data preparation
```

Model

```
# get model
if args['model'] == 'logistic_regression':
    print('Using logistic regression')
    model = nn.Linear(args['img_size'], args['num_classes'])
elif args['model'] == 'MLP':
    print('MLP')
    model = MLP(dropout=0, hidden_1=512, hidden_2=512)
```

Loss

Optimizer

```
# Loss and optimizer
loss_function = nn.CrossEntropyLoss() # this combined the Log
optimizer = torch.optim.SGD(model.parameters(), lr=args['lear
```

Hyper-parameters

```python
# arguments
def get_arguments():
    # Hyper-parameters
    args = {'img_size': 64 * 64,
            'num_classes': 10,
            'num_epochs': 50,
            'batch_size': 16,
            'learning_rate': 0.001,
            'model': 'logistic_regression'} # MLP or logistic_regression
    return args

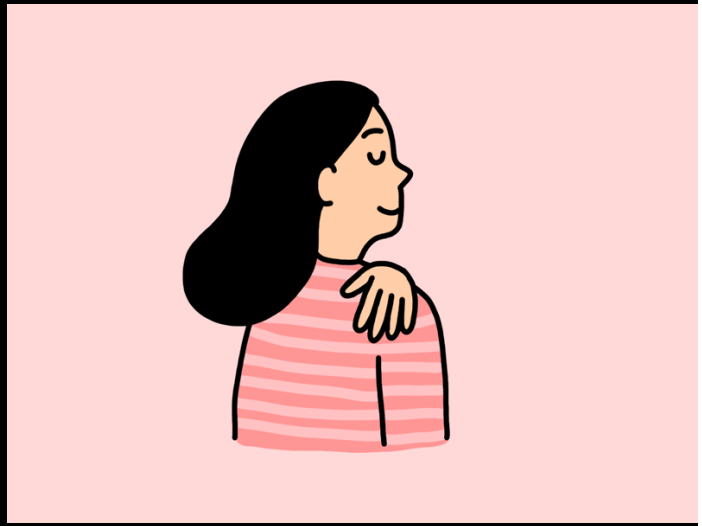
args = get_arguments()
print(args)
```

Trainer

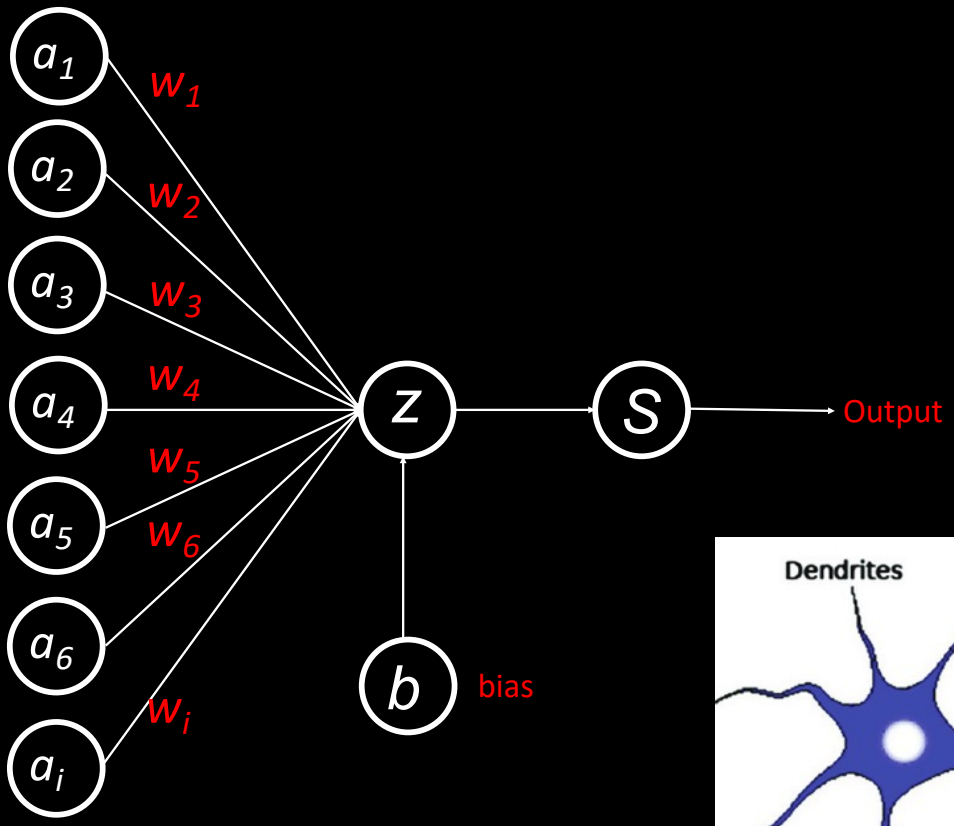
```
class Trainer():
    def __init__(self, args, train_loader, validation_loader, model, loss_function, optimizer):
        pass

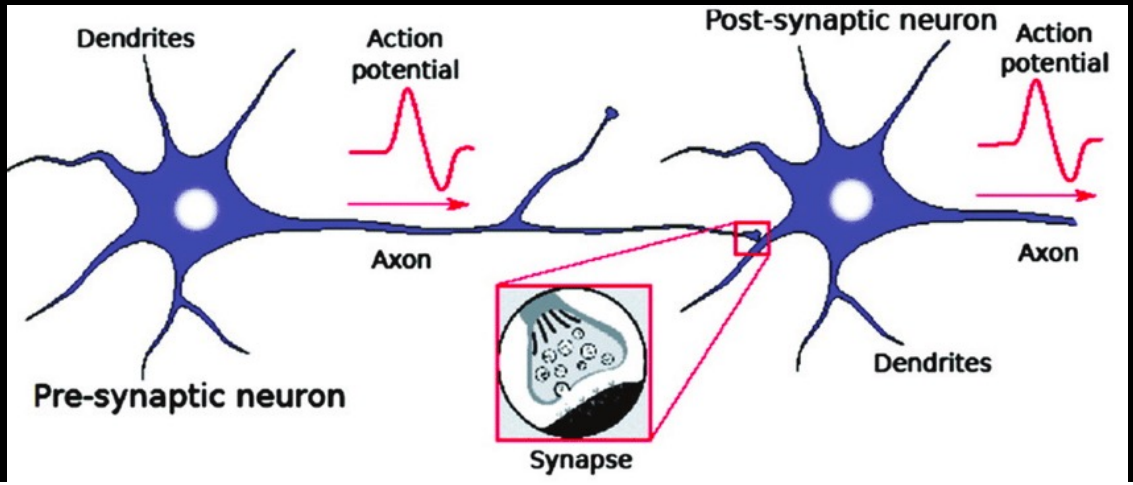
    def overall_loop(self):
        for epoch in range(args['num_epochs']):
            train_loss = self.training_loop(train_loader)  # do the training look
            validation_loss = self.validation_loop(validation_loader) # do the validation loop
            # print out the training and validation loss per epoch
            print('Epoch [{}/{}], Train Loss: {:.4f}, Validation Loss: {:.4f}'
                  .format(epoch + 1, args['num_epochs'],
                          sum(train_loss) / len(train_loss), sum(validation_loss) / len(validation_loss)))
```

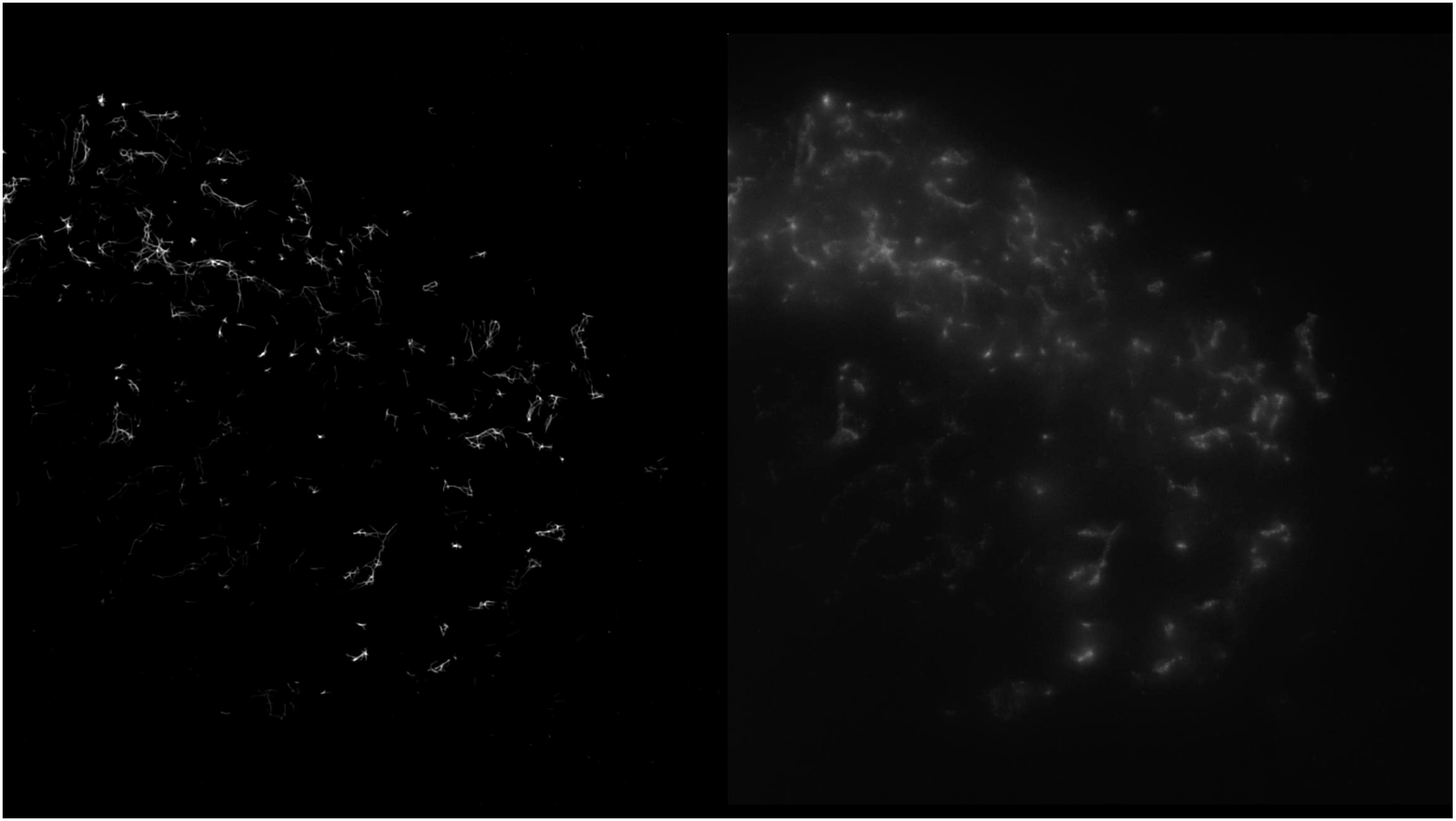
```
$ python mnist_logistic.py
Epoch [42/50], Step [100/600], Train Loss: 0.4635, Test Loss: 0.4393
Epoch [43/50], Step [100/600], Train Loss: 0.4607, Test Loss: 0.4365
Epoch [44/50], Step [100/600], Train Loss: 0.4579, Test Loss: 0.4339
Epoch [45/50], Step [100/600], Train Loss: 0.4552, Test Loss: 0.4313
Epoch [46/50], Step [100/600], Train Loss: 0.4527, Test Loss: 0.4289
Epoch [47/50], Step [100/600], Train Loss: 0.4502, Test Loss: 0.4265
Epoch [48/50], Step [100/600], Train Loss: 0.4478, Test Loss: 0.4243
Epoch [49/50], Step [100/600], Train Loss: 0.4455, Test Loss: 0.4221
Epoch [50/50], Step [100/600], Train Loss: 0.4433, Test Loss: 0.4200
Accuracy of the model on the 10000 test images: 89.110 %
```



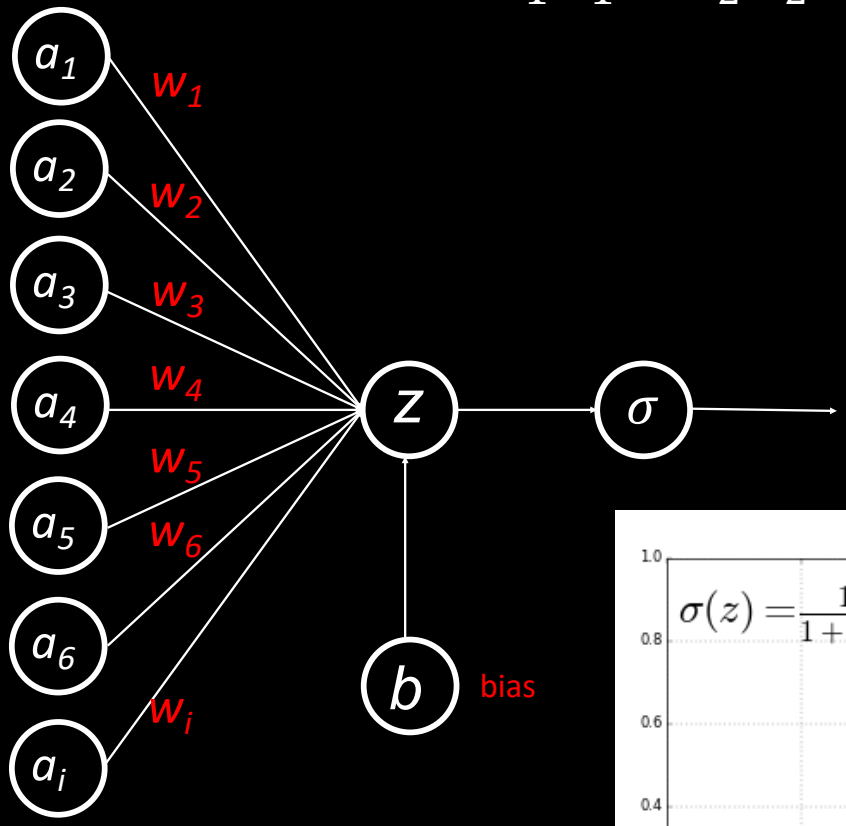
Neurons and Neural Networks


a_1
a_2
a_3
a_4
a_5
a_6
a_i
w_1
w_2
w_3
w_4
w_5
w_6
w_i
z
S
Output
b
bias

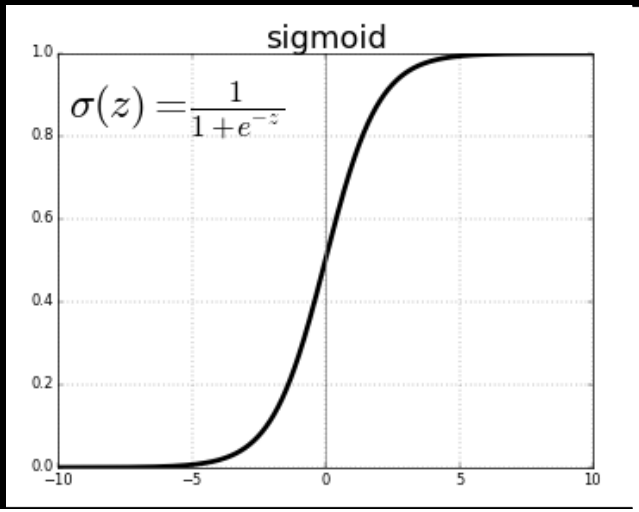

Dendrites
Action potential
Post-synaptic neuron
Action potential
Axon
Axon
Dendrites
Pre-synaptic neuron
Synapse

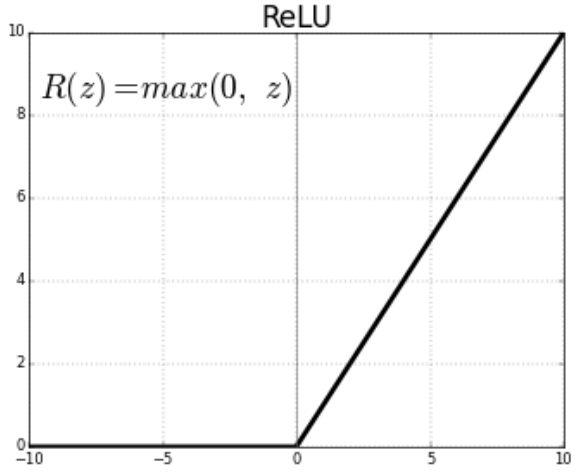


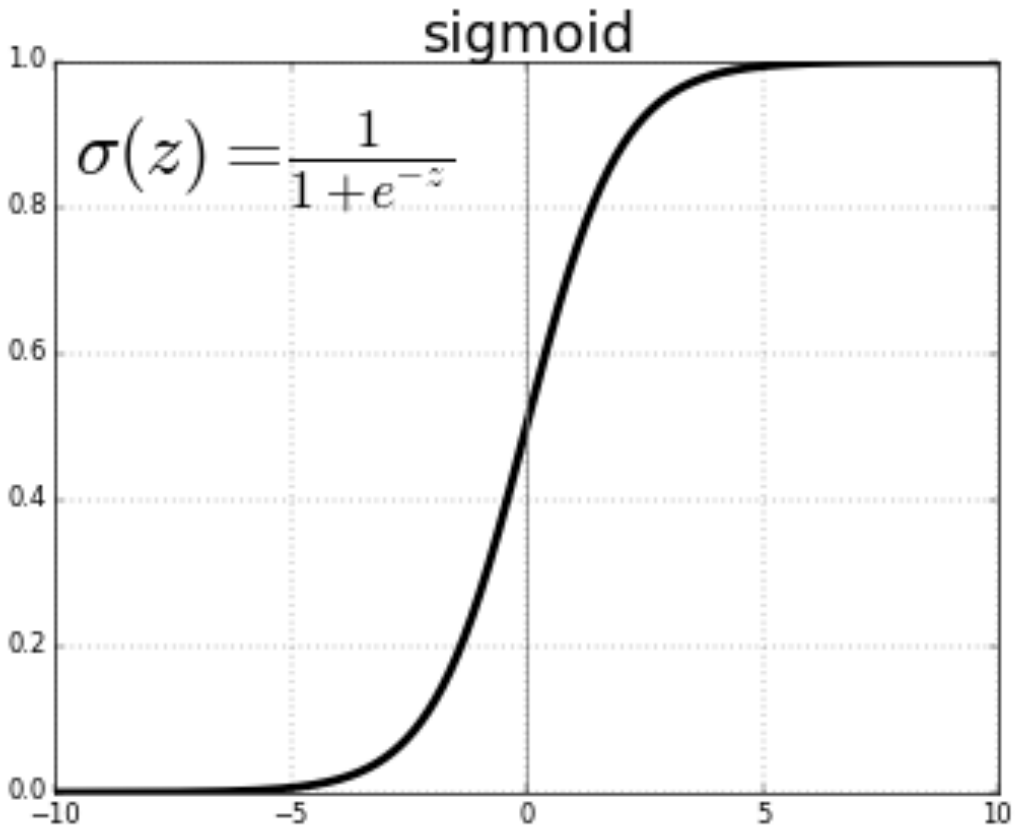
$$z = a_1 w_1 + a_2 w_2 + \cdots + a_i w_i + b$$



$$out = \sigma(z)$$

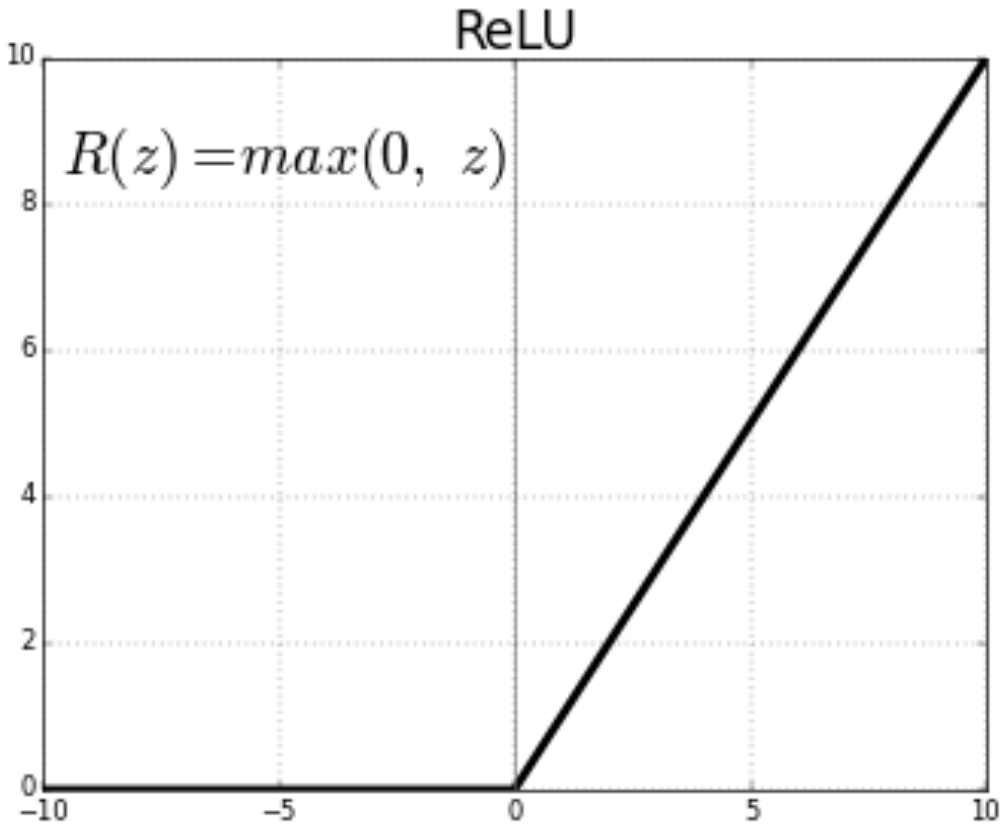




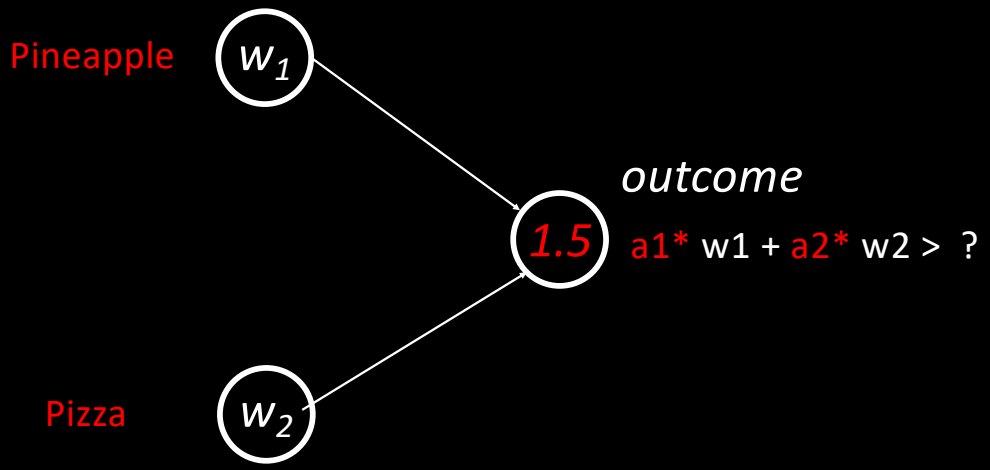


pizza

happiness

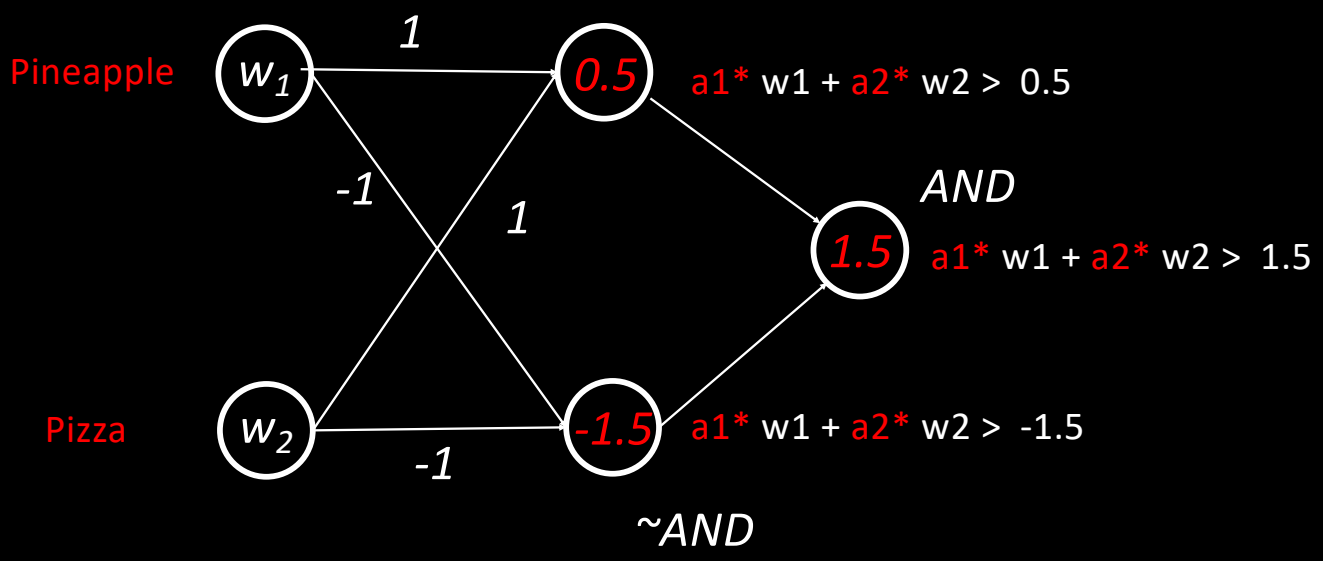


pizza



	Pizza	Pineapple	Happiness
XOR	**A**	**B**	**Out**
	0	0	0
	1	0	1
	0	1	1
	1	1	0

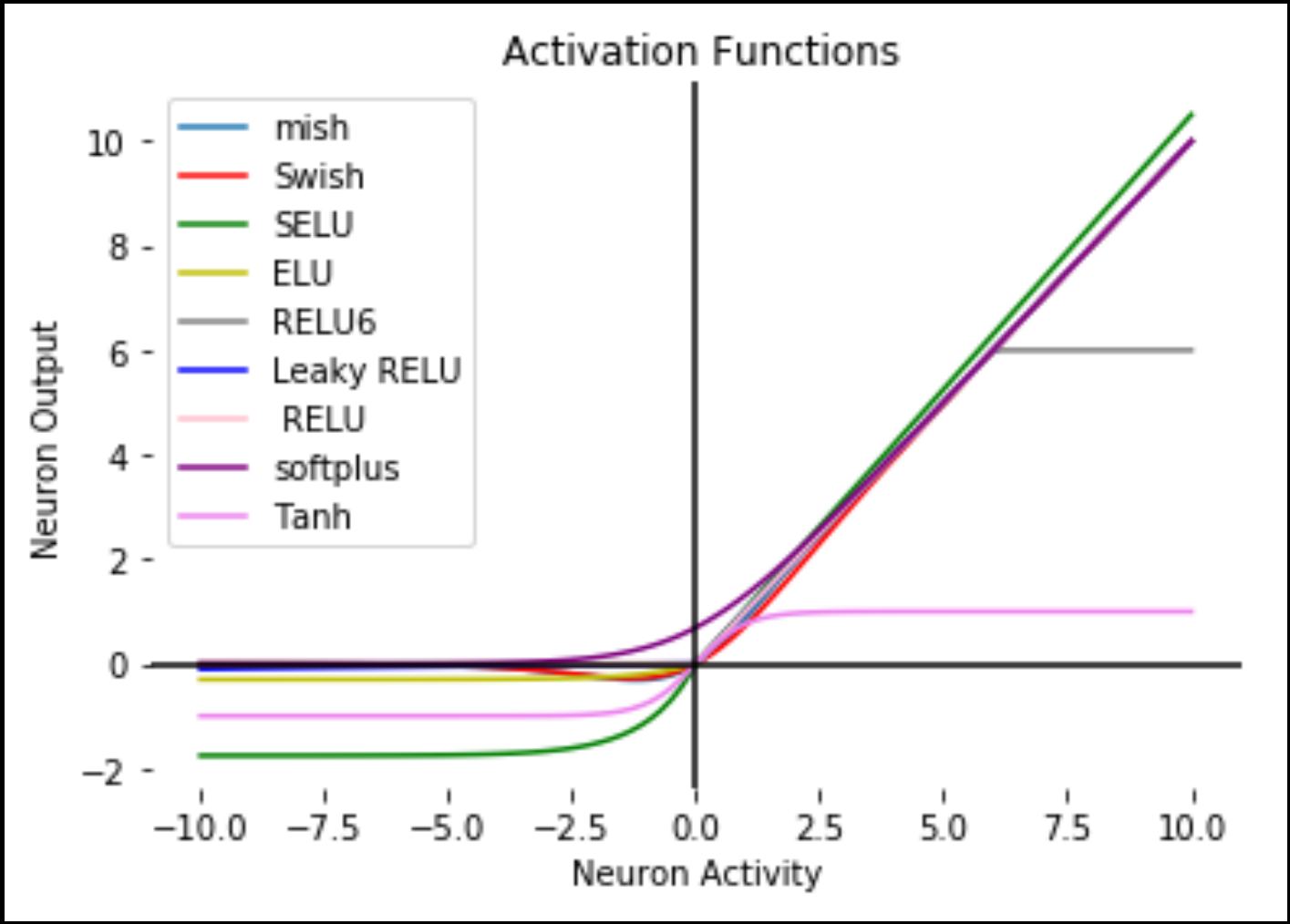
A	B	Out
0	0	0
1	0	1
0	1	1
1	1	1



XOR

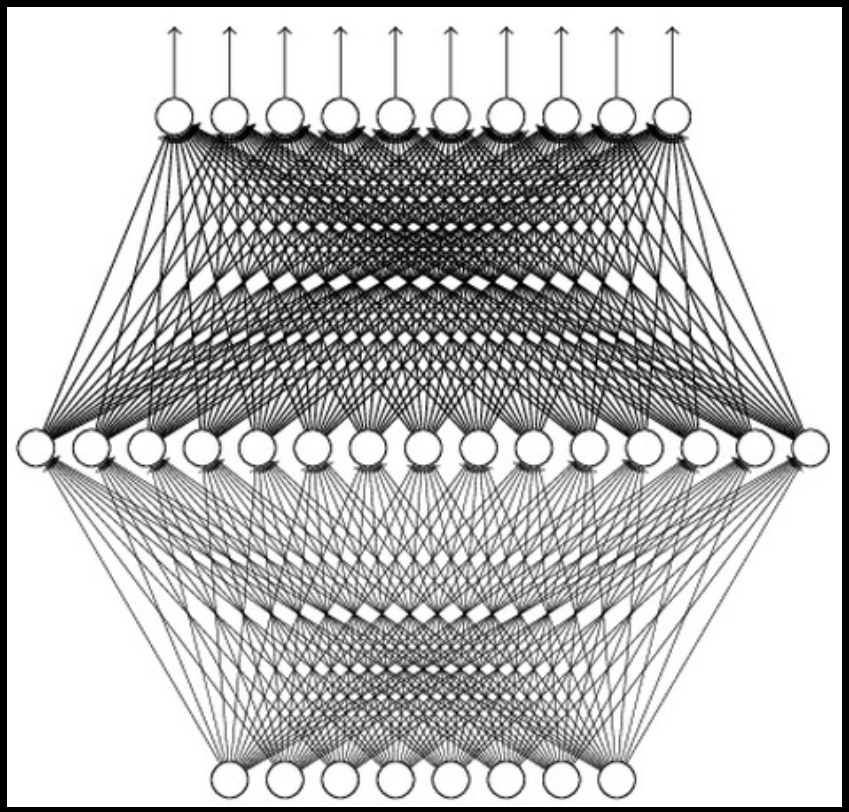
A	B	Out
0	0	0
1	0	1
0	1	1
1	1	0

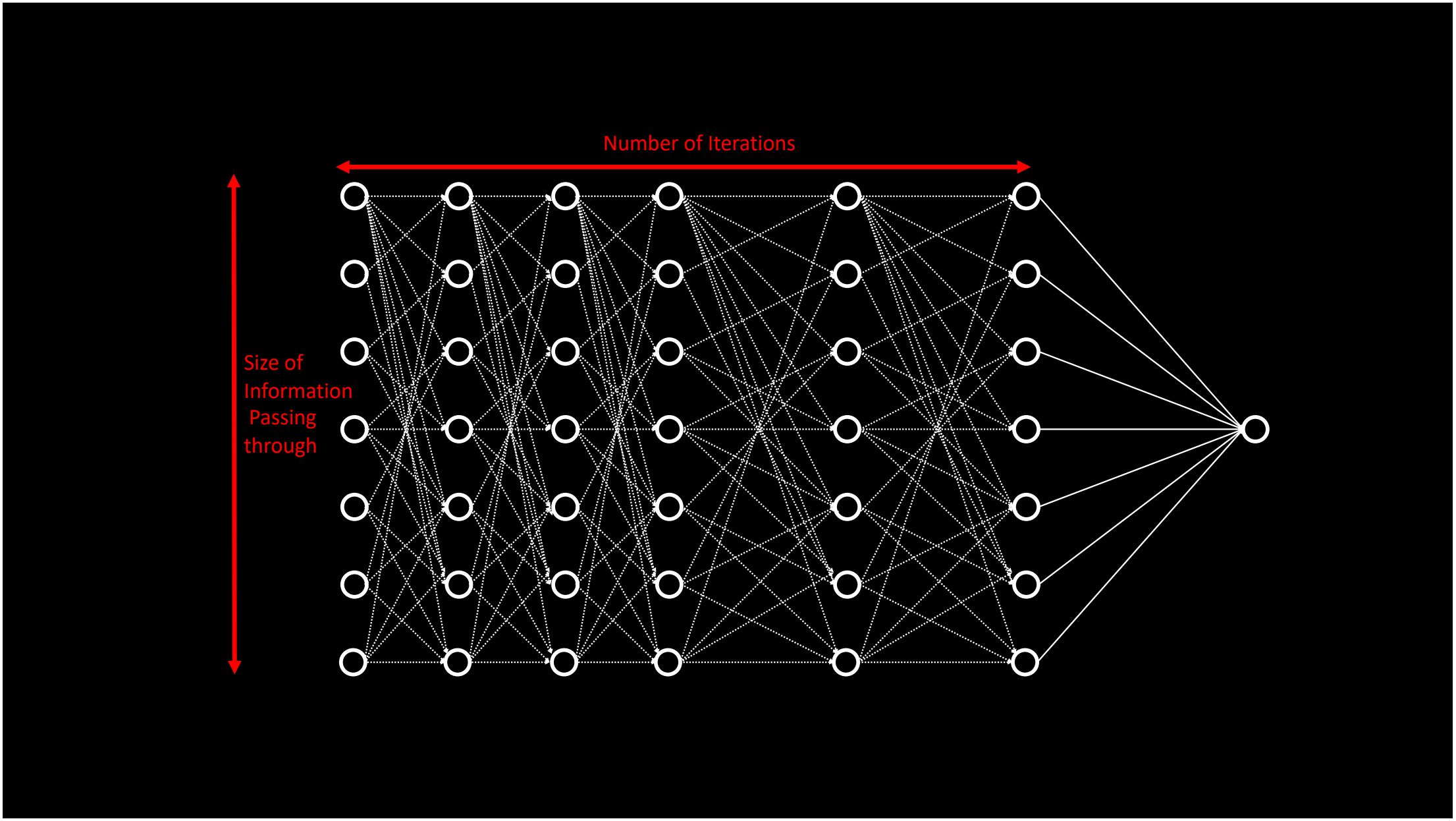
A	B	Out
0	0	1
1	0	1
0	1	1
1	1	0

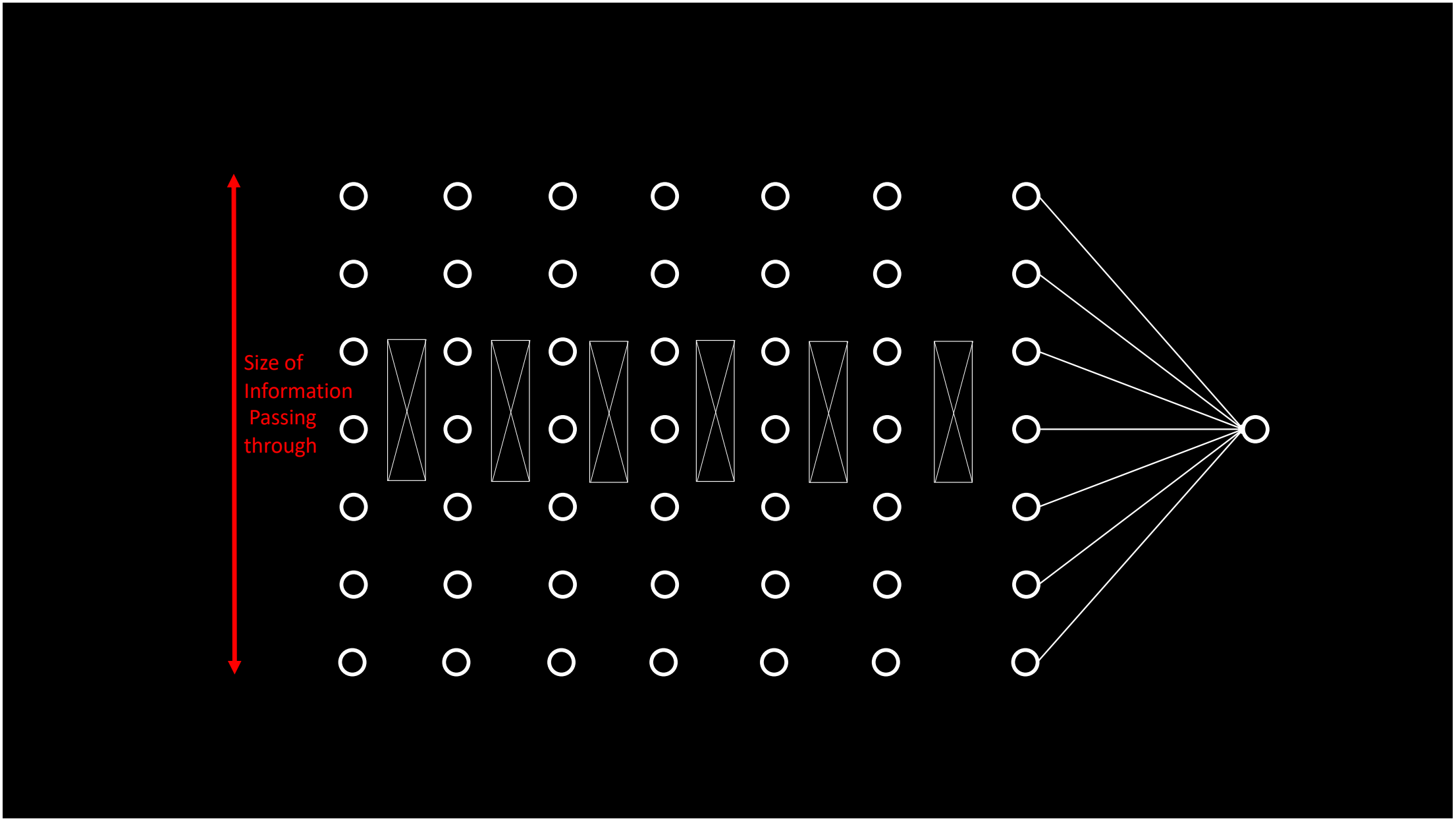

Activation Functions
mish
Swish
SELU
ELU
RELU6
Leaky RELU
RELU
softplus
Tanh
Neuron Output
10
8
6
4
2
0
−2
−10.0
−7.5
−5.0
−2.5
0.0
2.5
5.0
7.5
10.0
Neuron Activity

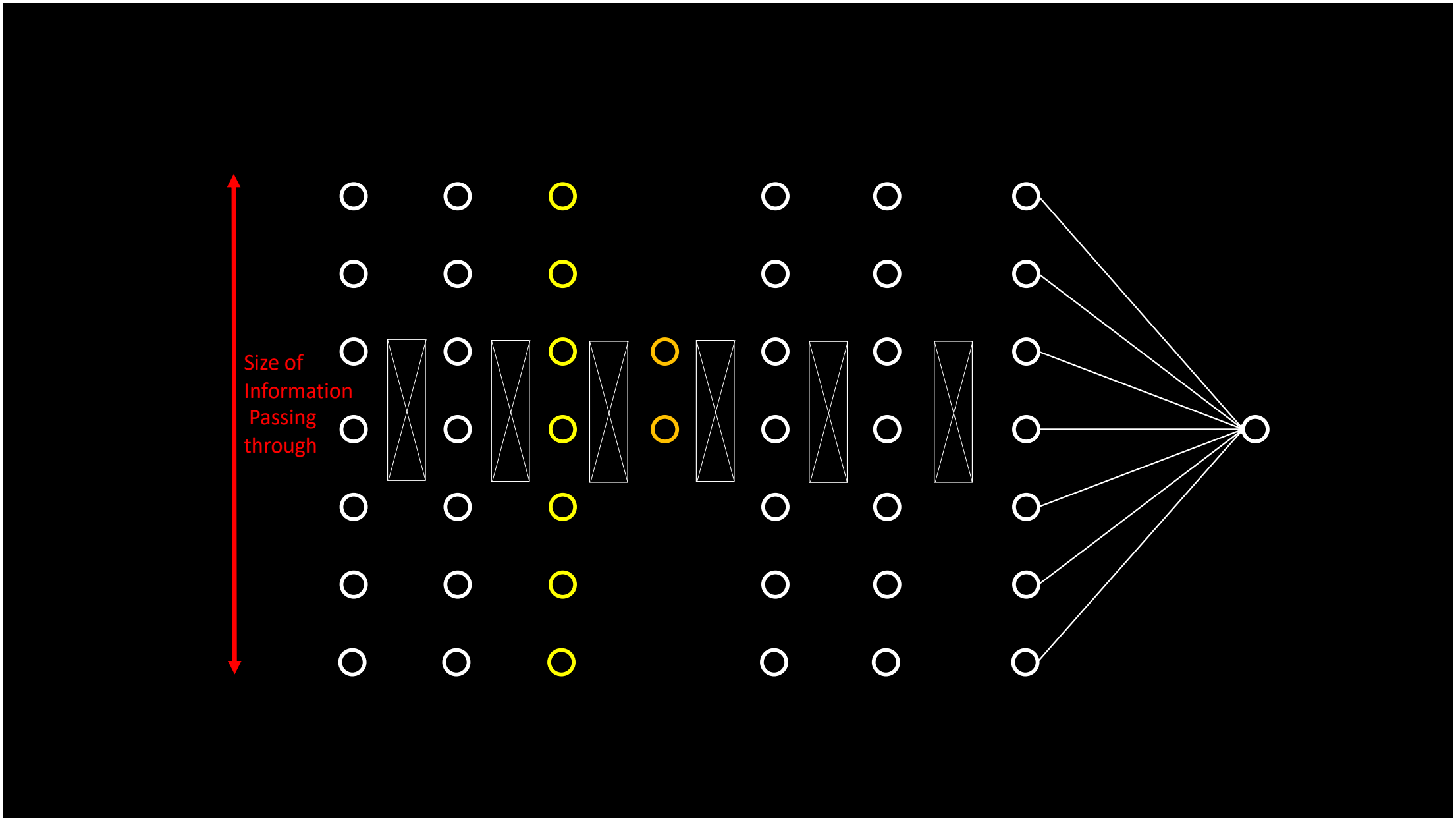
Universality Approximation Theorem

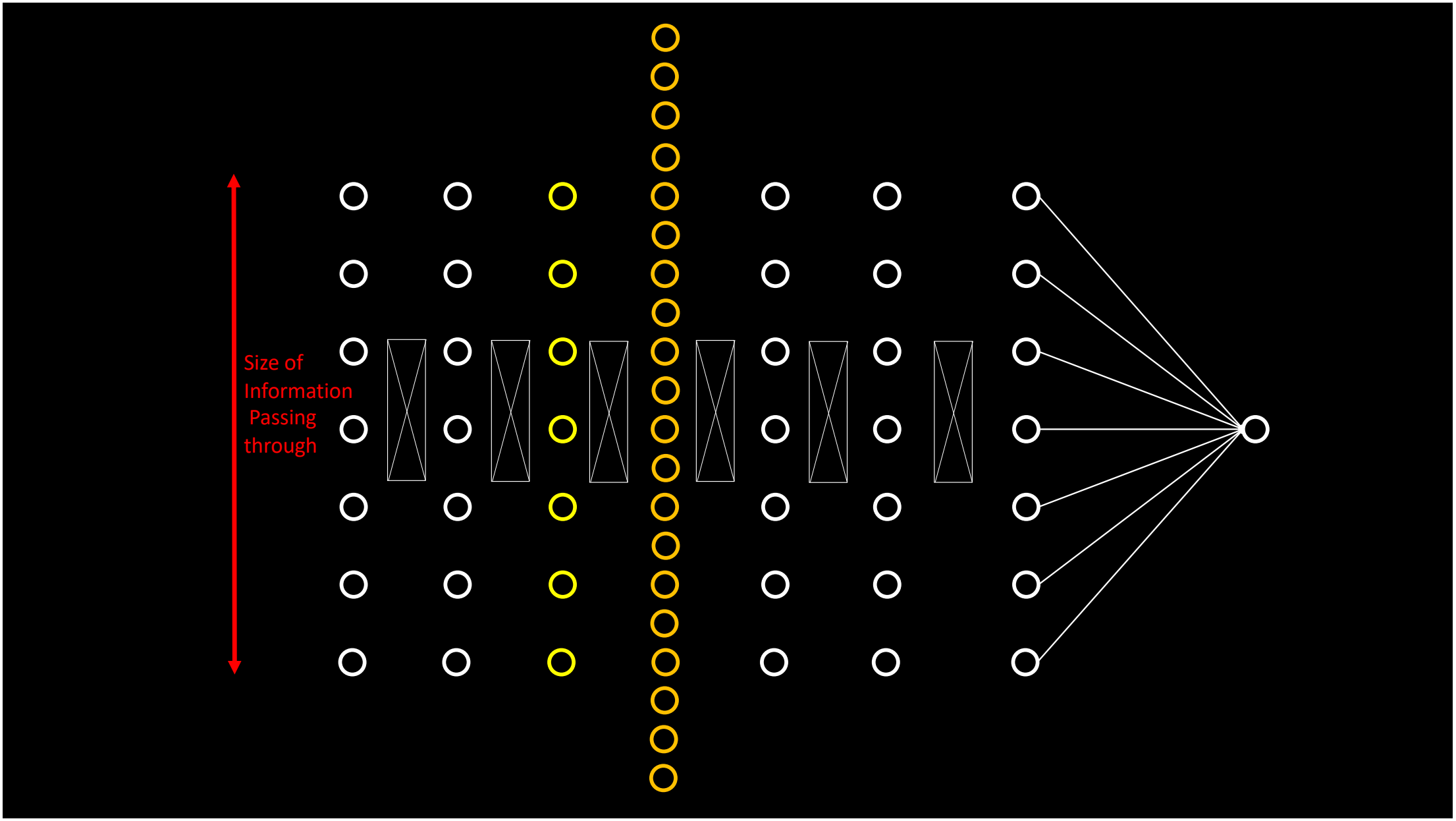
$$f: R^N > R^M$$

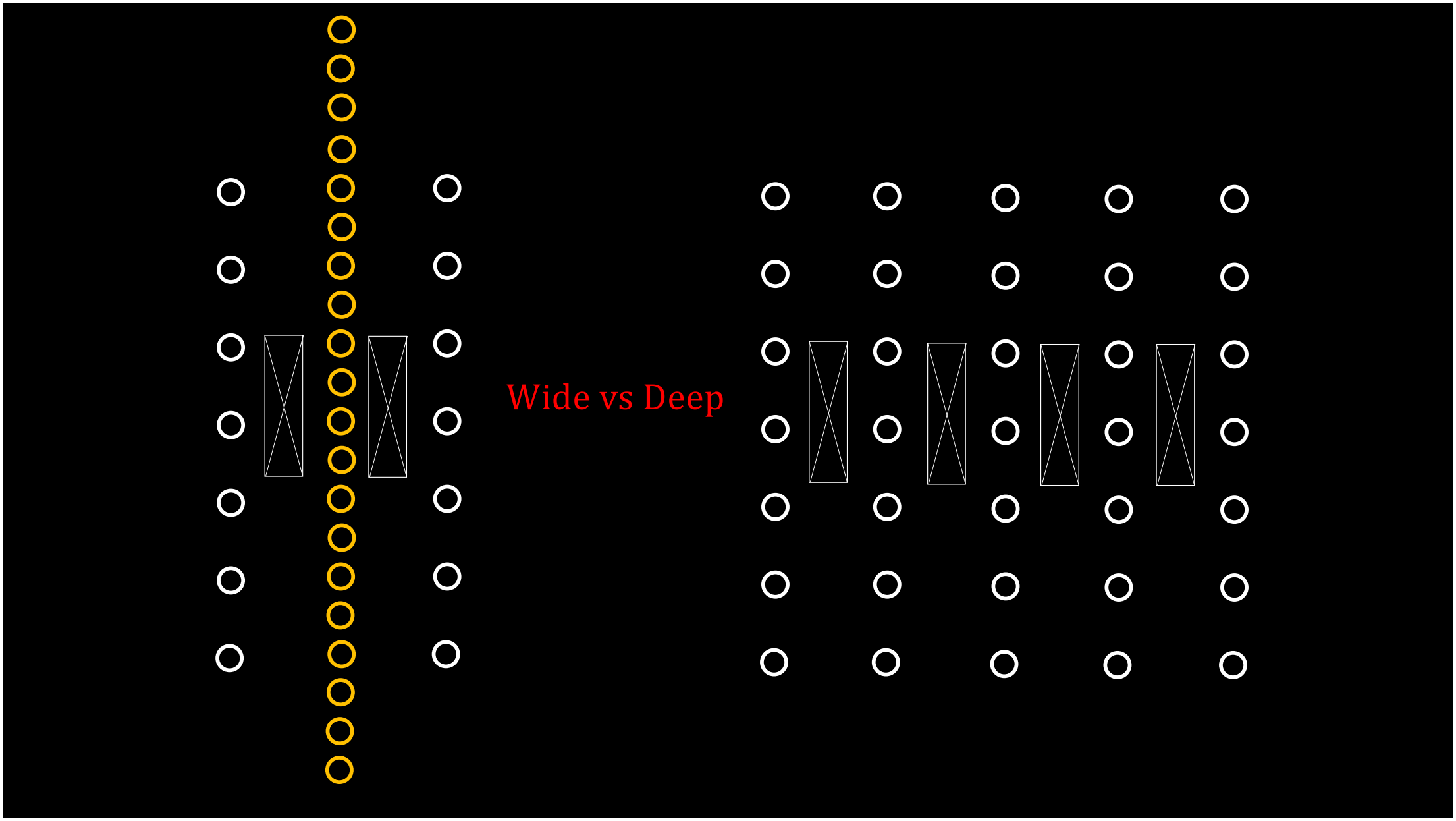



Number of Iterations
Size of Information Passing through


Size of
Information
Passing
through


Size of
Information
Passing
through


Size of
Information
Passing
through


Wide vs Deep

VGG16

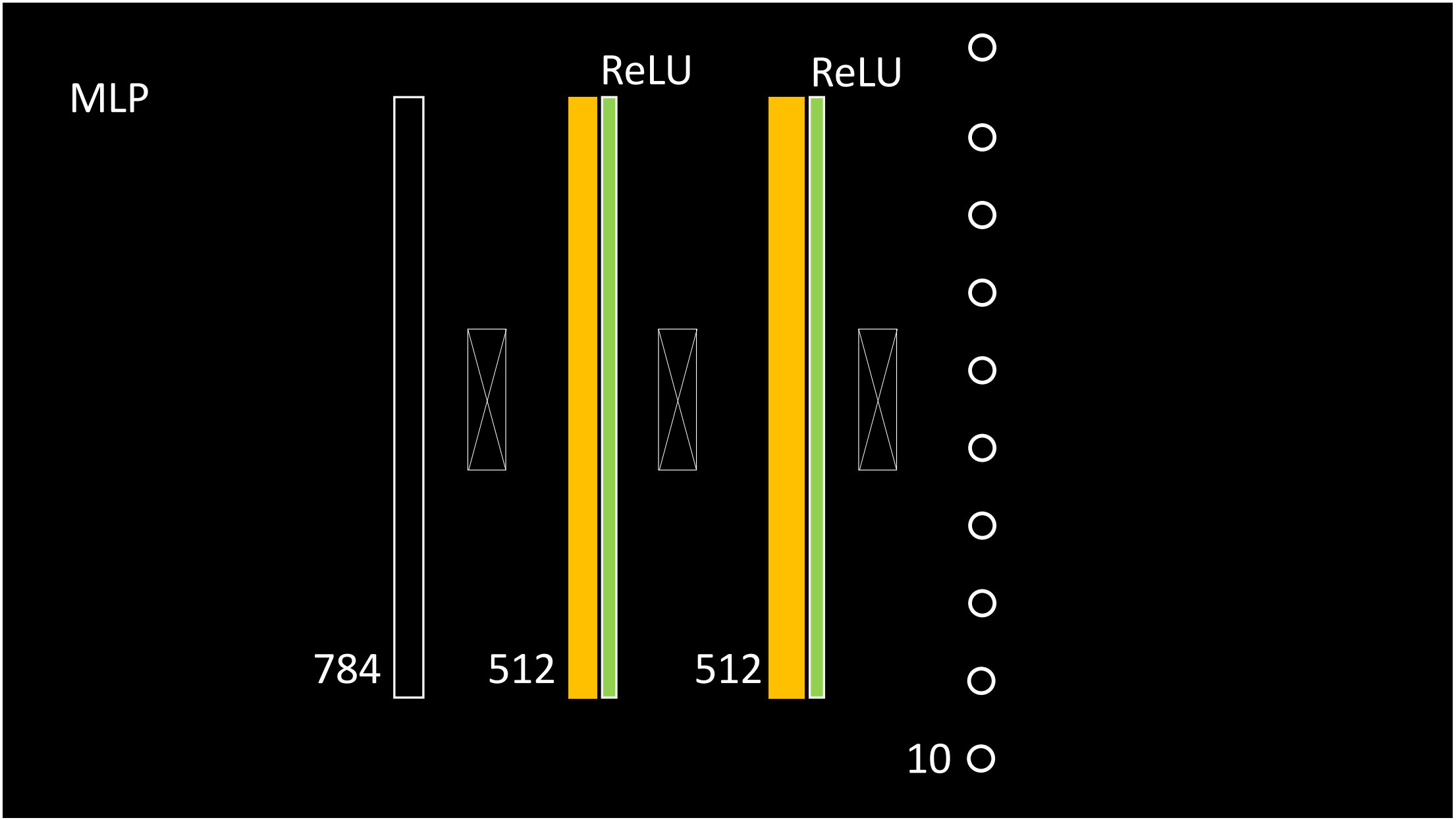
VGG19

Inception

Xception

ResNet

Pytorch Example:
MNIST w/ Multi-layer Perceptron


MLP
ReLU
ReLU
784
512
512
10

```python
# define NN architecture
class MLP(nn.Module):
    def __init__(self):
        super(MLP, self).__init__()
        # number of hidden nodes in each layer (512)

        # linear layer (784 -> hidden_1)
        self.fc1 = nn.Linear(28 * 28, 512)

        # linear layer (n_hidden -> hidden_2)
        self.fc2 = nn.Linear(512, 512)

        # linear layer (n_hidden -> 10)
        self.fc3 = nn.Linear(512, 10)
```

```python
def forward(self, x):
        # flatten image input
        x = x.view(-1, 28 * 28)

        # add hidden layer, with relu activation function
        x = F.relu(self.fc1(x))

        # add hidden layer, with relu activation function
        x = F.relu(self.fc2(x))

        # add output layer
        x = self.fc3(x)
        return x
```

```
model
Net(
  (fc1): Linear(in_features=784, out_features=512, bias=True)
  (fc2): Linear(in_features=512, out_features=512, bias=True)
  (fc3): Linear(in_features=512, out_features=10, bias=True))
)
```

```
model.fc1.weight.shape
torch.Size([512, 784])

model.fc1.bias.shape
torch.Size([512])

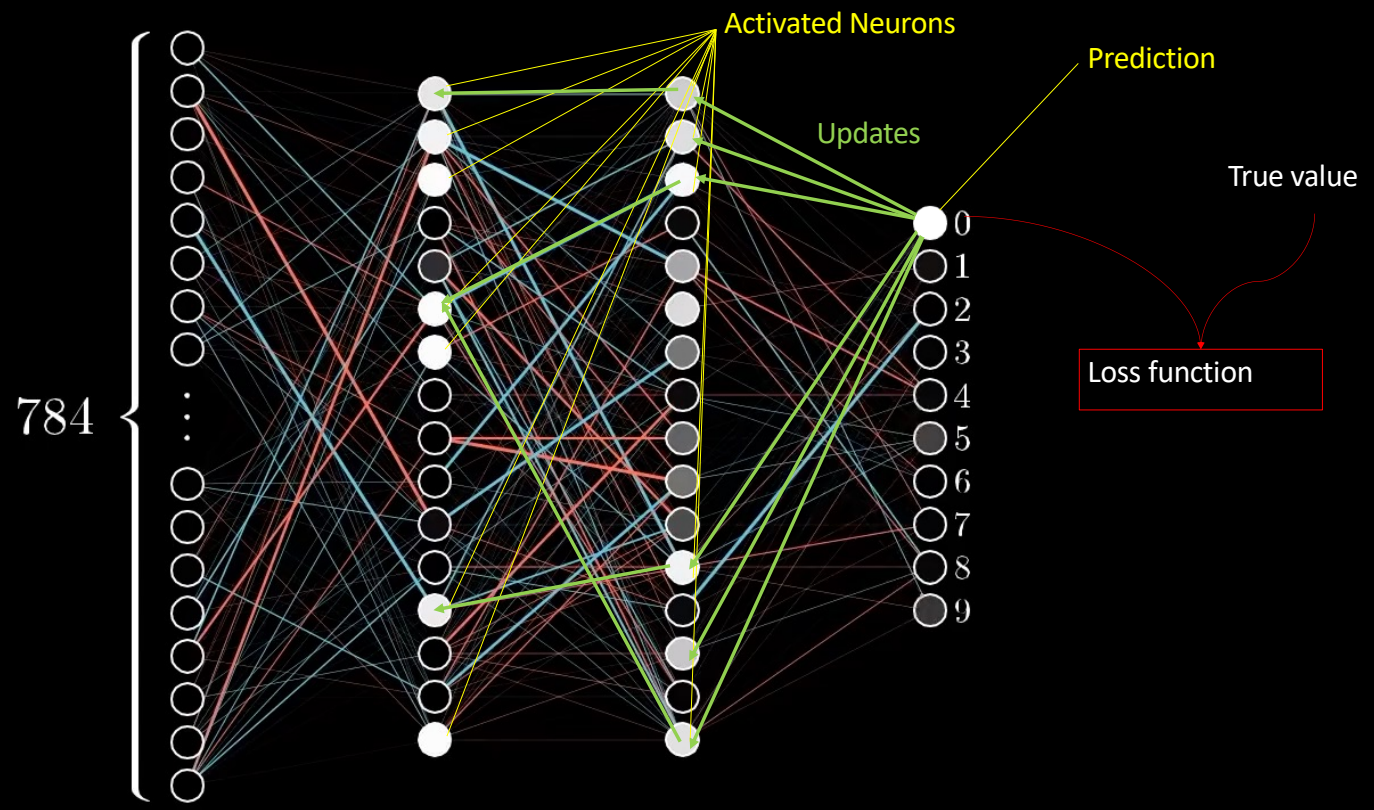
model.fc2.weight.shape
torch.Size([512, 512])

model.fc2.bias.shape
torch.Size([512])

model.fc3.weight.shape
torch.Size([10, 512])

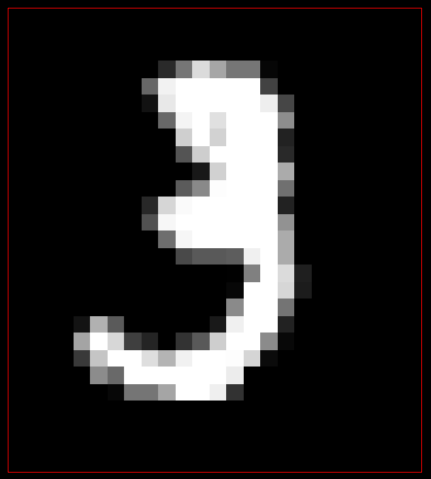
model.fc3.bias.shape
torch.Size([10])
```

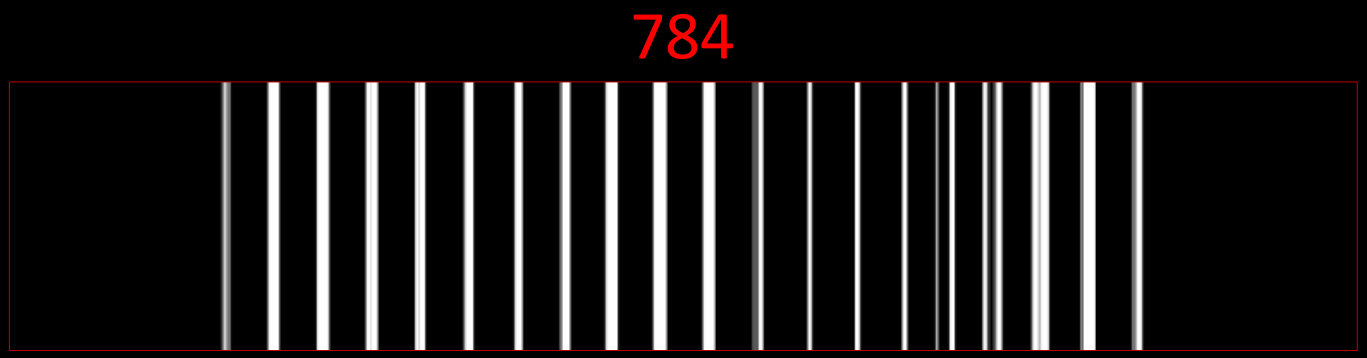
https://www.3blue1brown.com



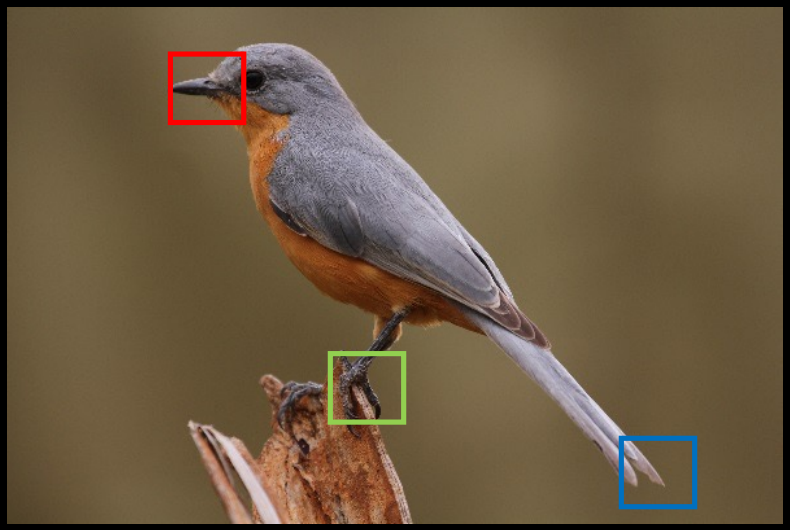
Convolutional Neural Networks

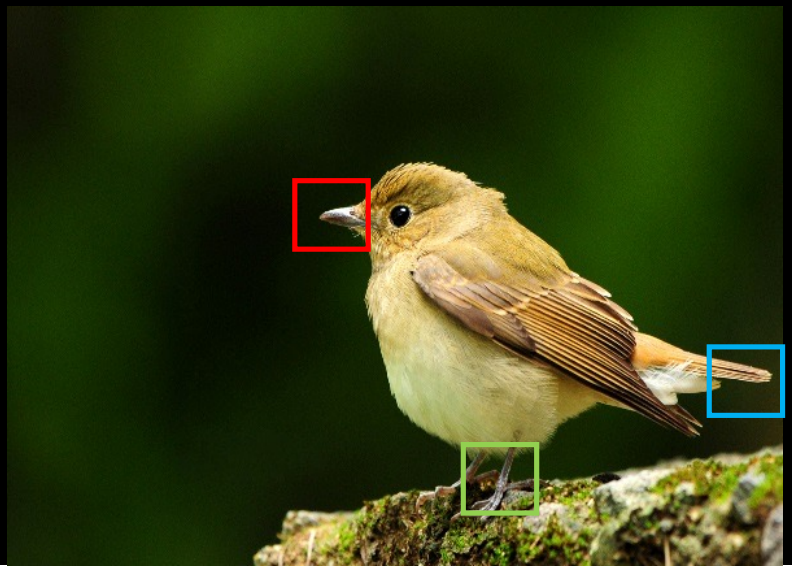
Convolutional Neural Networks





Convolutional Neural Networks





"Convolutional"

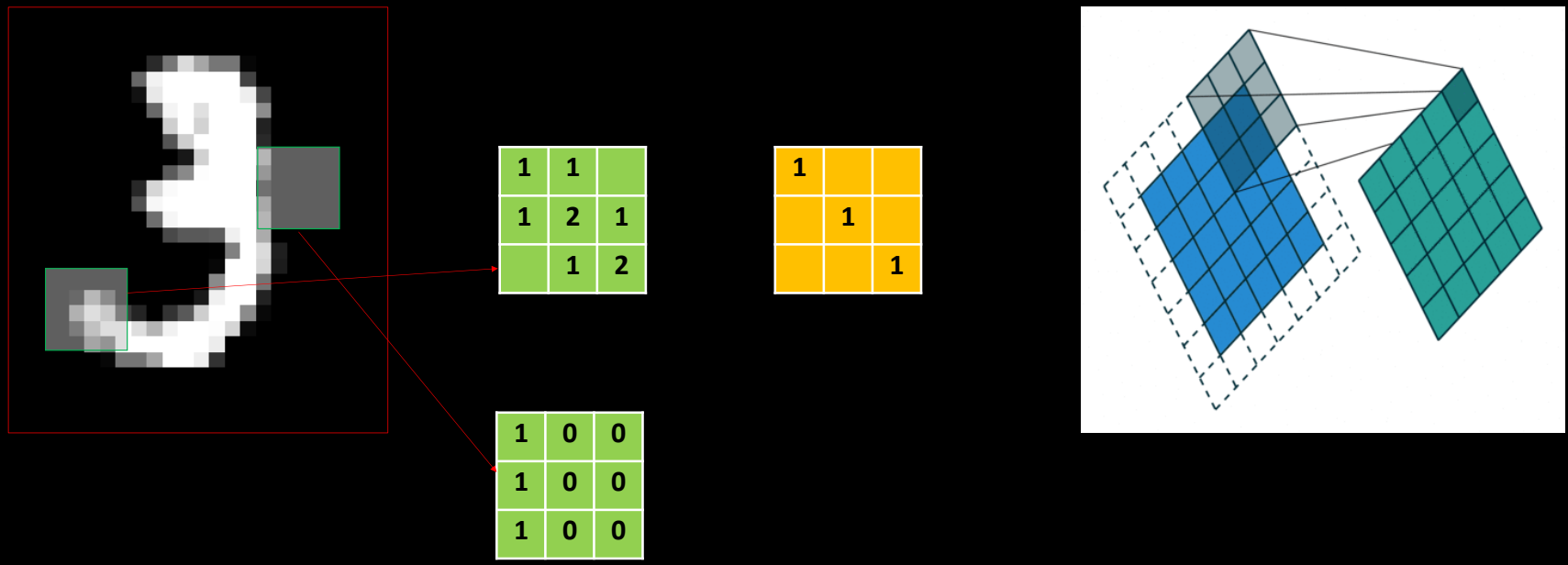
Beak (left, right, middle….)

Feet (left, right, middle….)

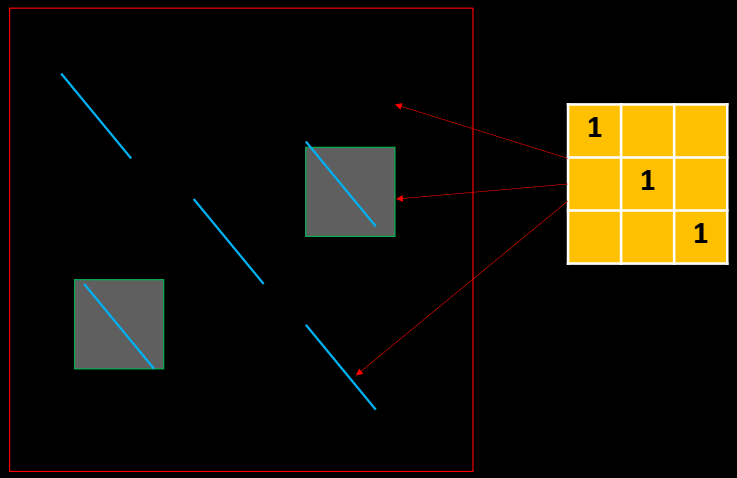
Tail (left, right, middle….)

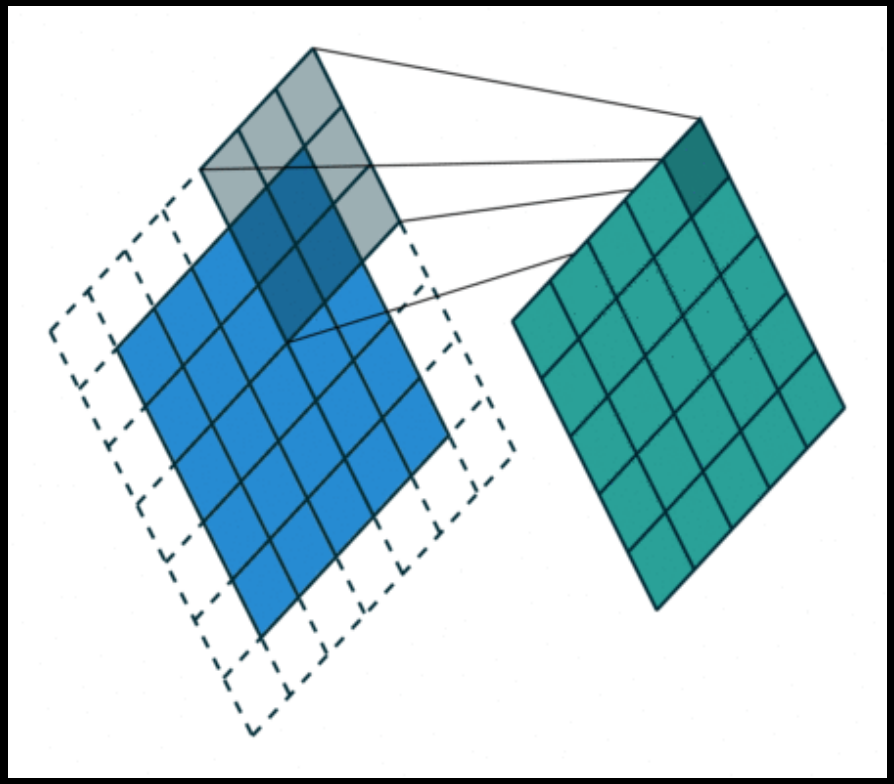
Should share the same parameters!

Convolutional Neural Networks

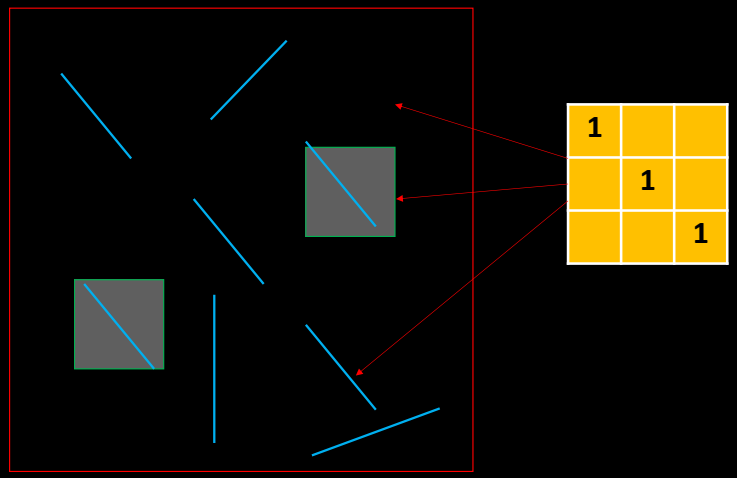


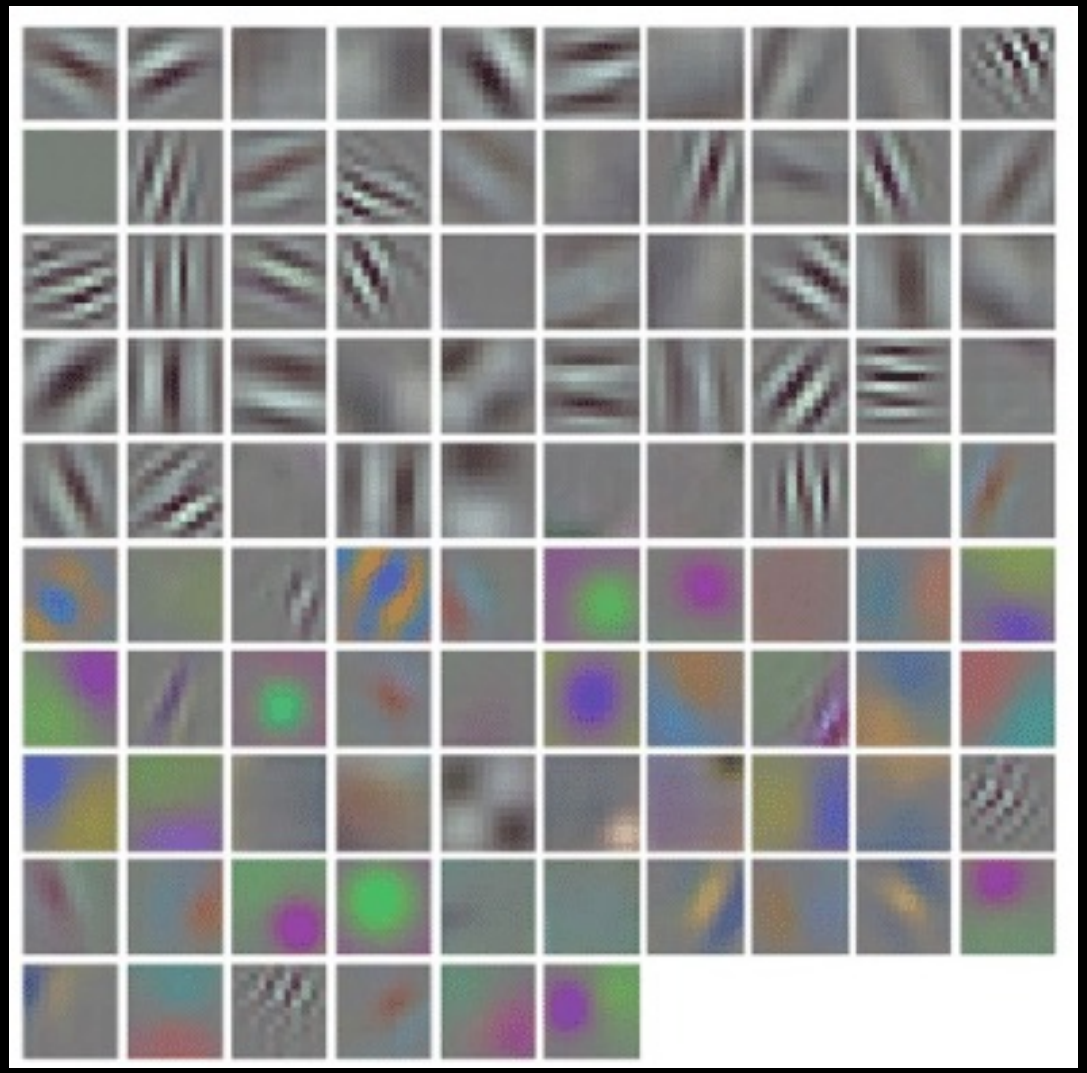
Convolutional Neural Networks



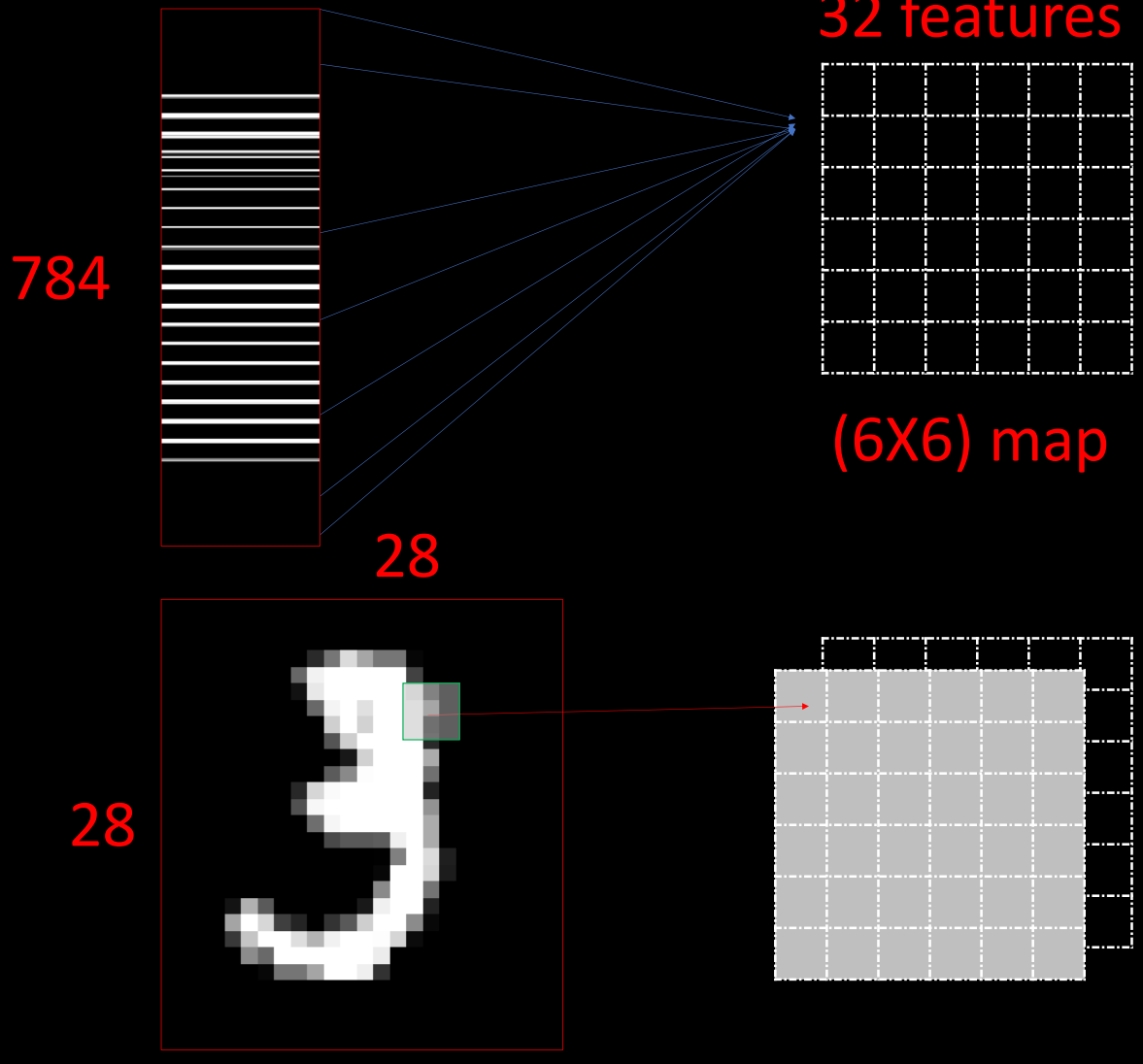


Convolutional Neural Networks

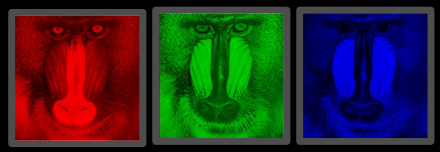




Convolutional Neural Networks



Convolutional Neural Networks



3

256X256

256X256

128X128

64

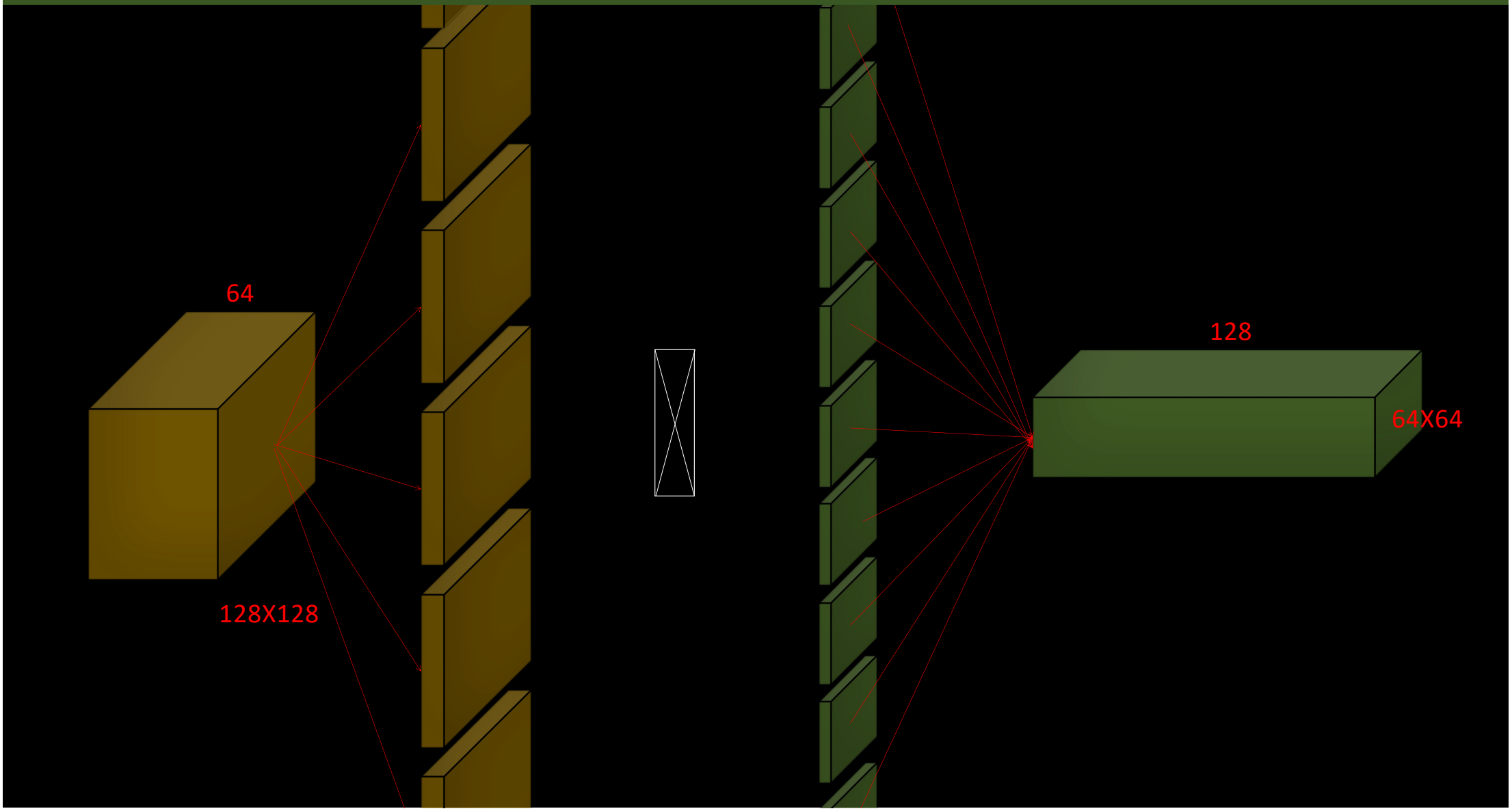
128X128

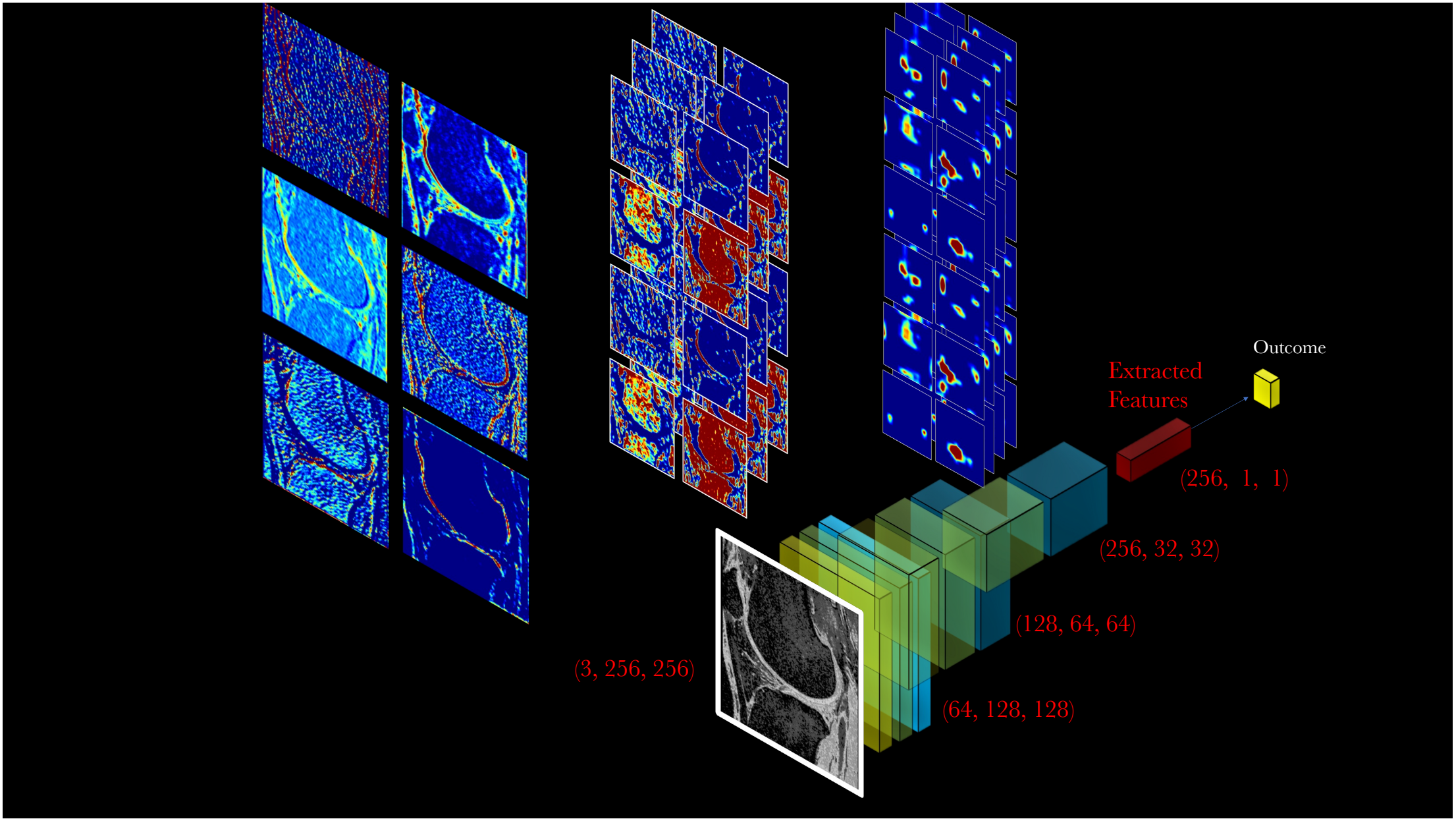
```
model=torchvision.models.vgg11()

model.features[0]
Conv2d(3, 64, kernel_size=(3, 3), stride=(1, 1), padding=(1, 1)

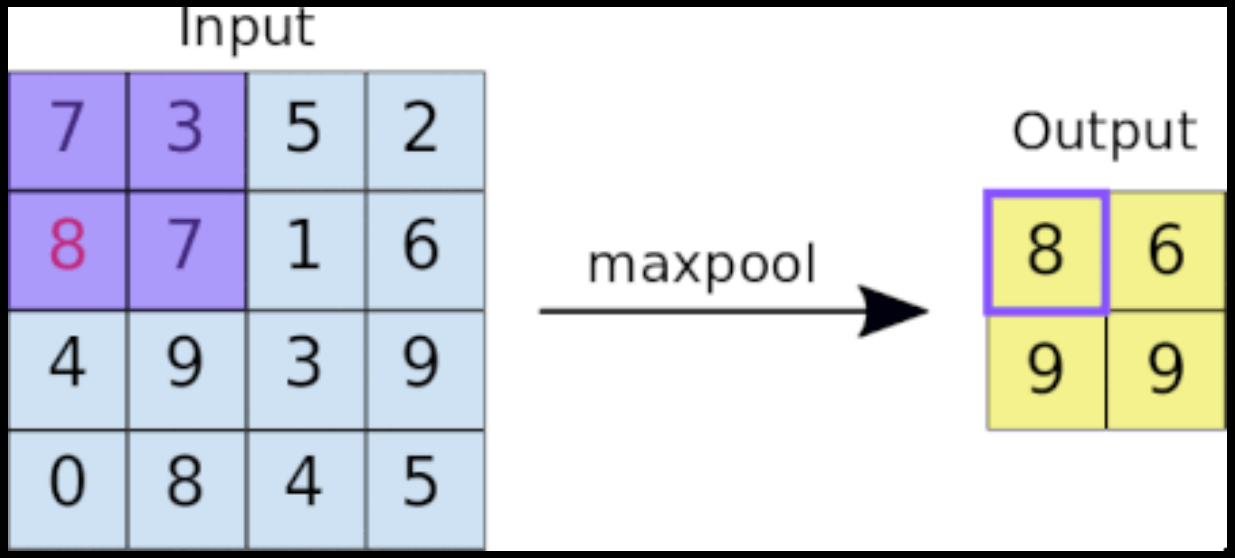
model.features[0].weight.shape
torch.Size([64, 3, 3, 3])
```

Convolutional Neural Networks



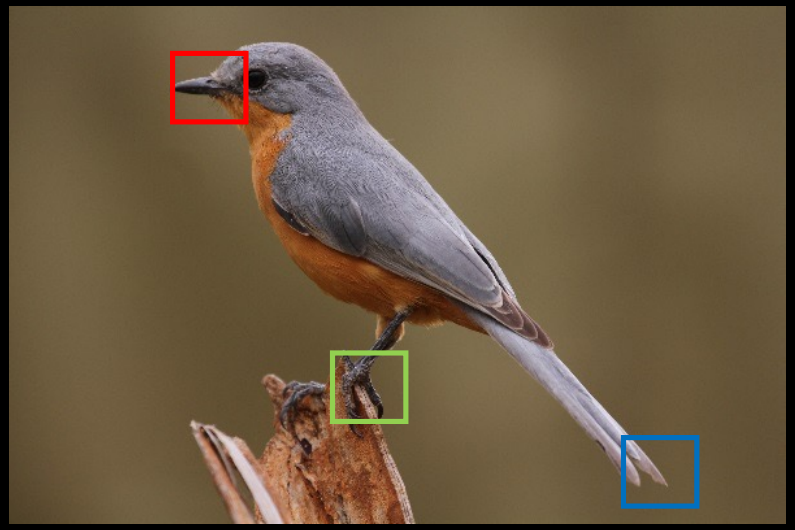

Outcome
Extracted
Features
(256, 1, 1)
(256, 32, 32)
(128, 64, 64)
(64, 128, 128)
(3, 256, 256)

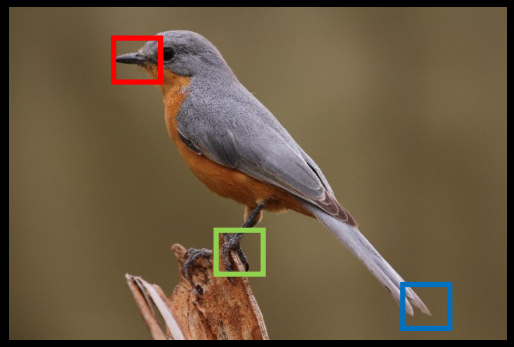
Layers: pooling

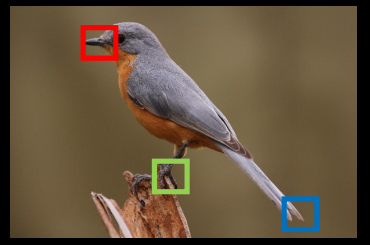


nn.MaxPool2d(kernel_size=2, stride=2, padding=0, dilation=1, ceil_mode=False)

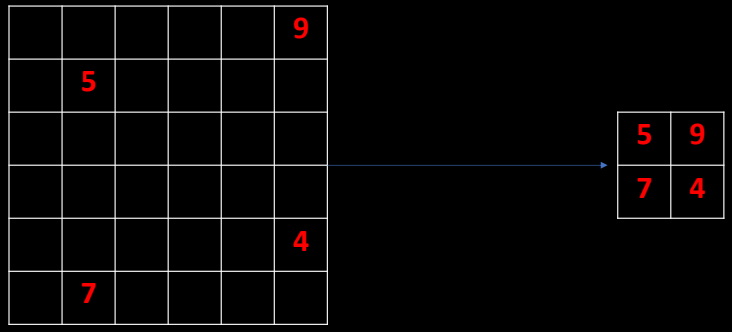
Layers: pooling





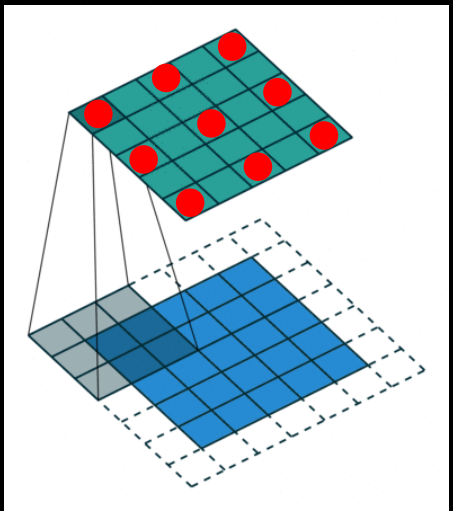


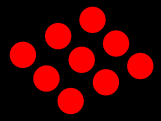
Important features should be independent of scaling!



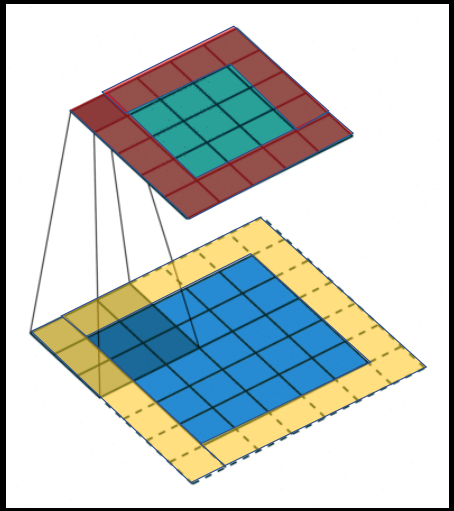
Layers

Stride

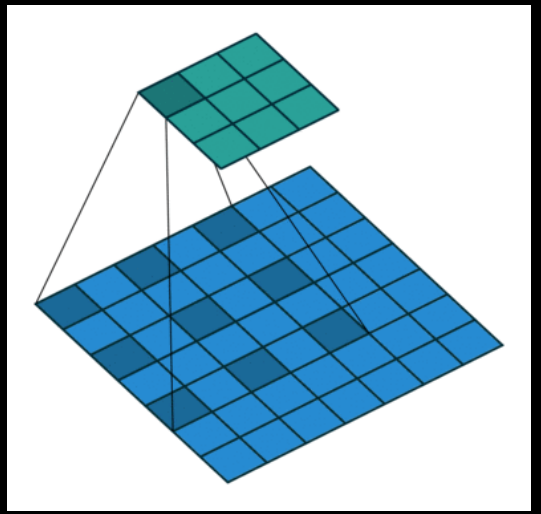


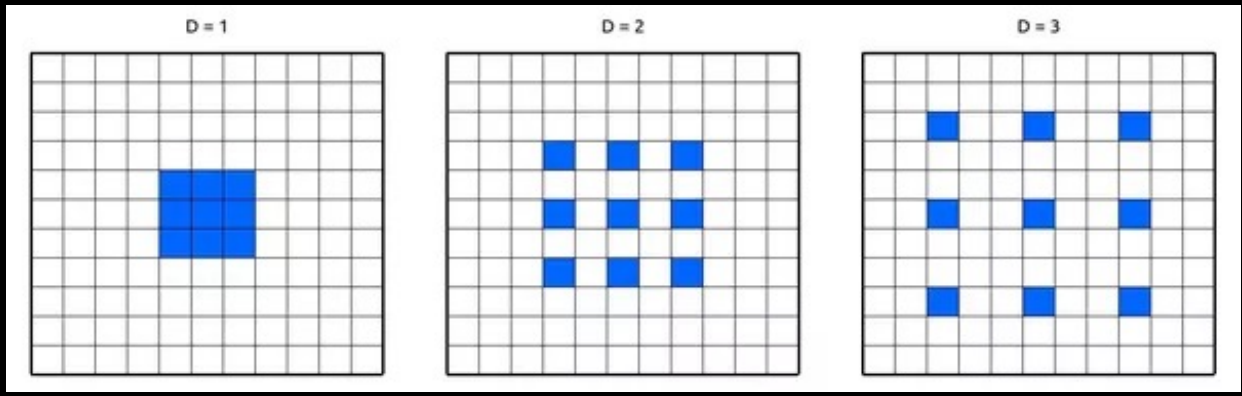


Padding

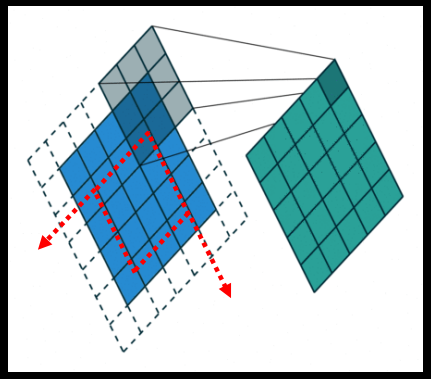


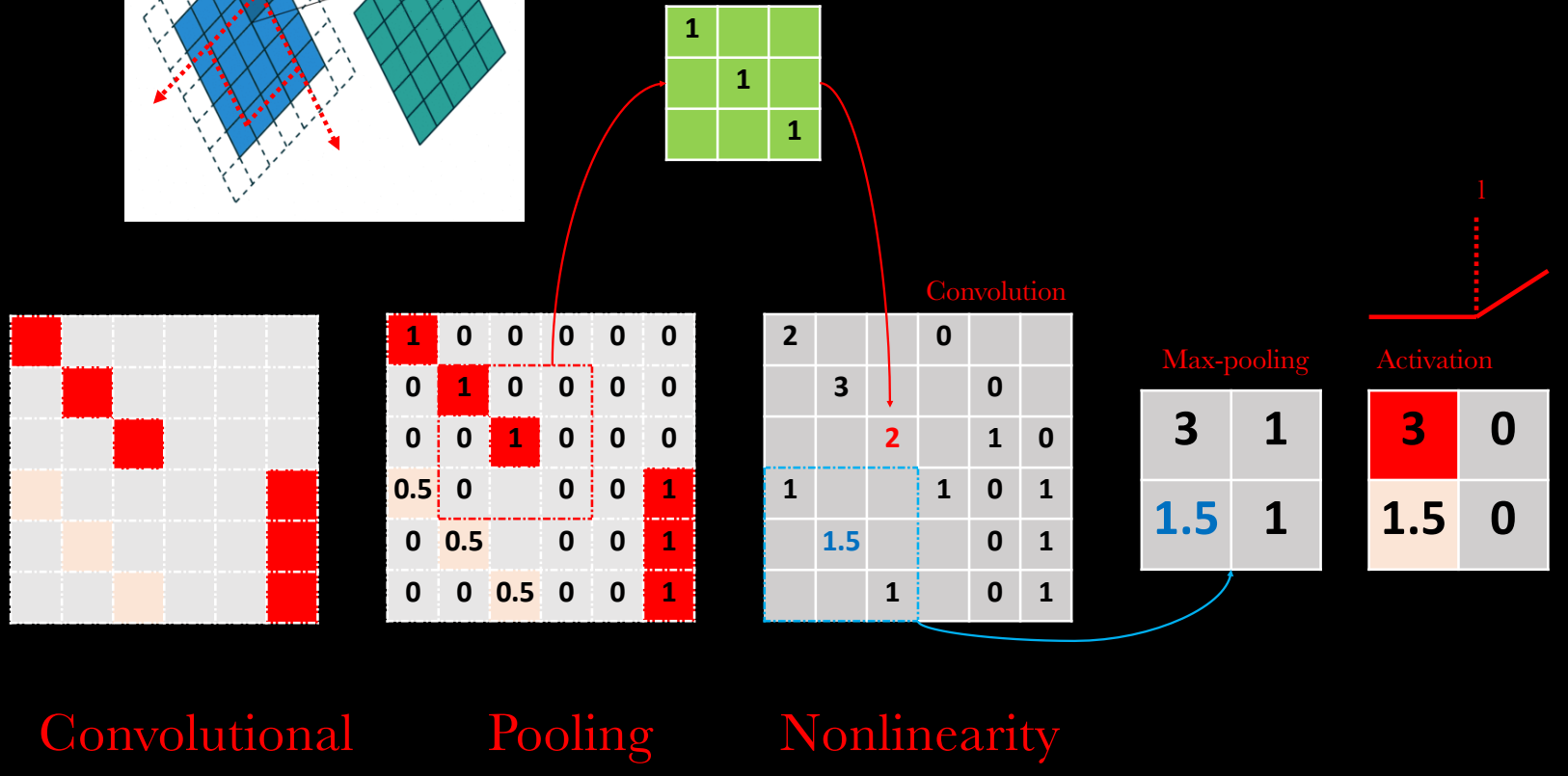
Dilation

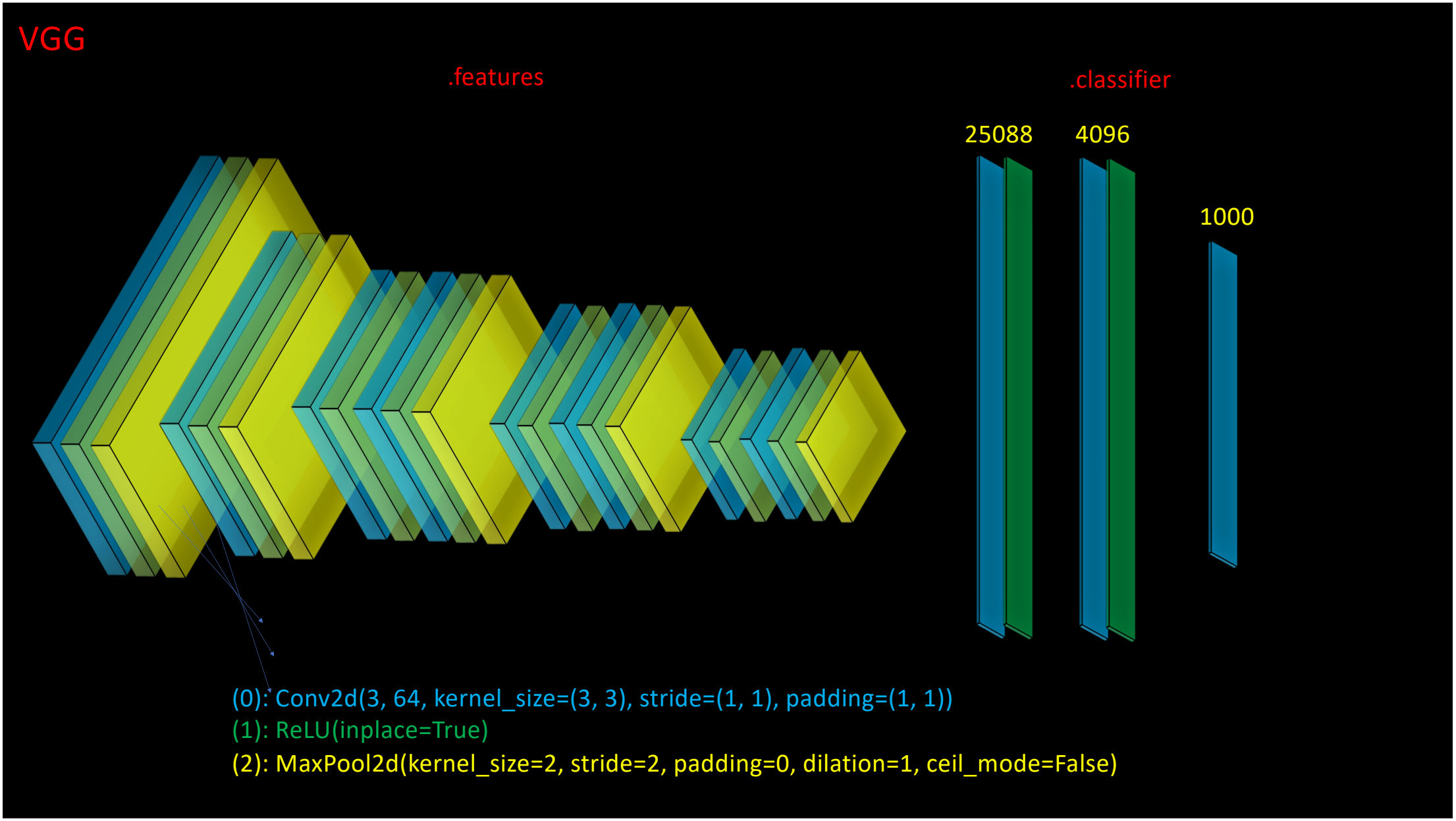




Convolutional Neural Networks






VGG
.features
.classifier
25088
4096
1000
(0): Conv2d(3, 64, kernel_size=(3, 3), stride=(1, 1), padding=(1, 1))
(1): ReLU(inplace=True)
(2): MaxPool2d(kernel_size=2, stride=2, padding=0, dilation=1, ceil_mode=False)

```
torchvision.models.vgg11()
VGG(
  (features): Sequential(
    (0): Conv2d(3, 64, kernel_size=(3, 3), stride=(1, 1), padding=(1, 1))
    (1): ReLU(inplace=True)
    (2): MaxPool2d(kernel_size=2, stride=2, padding=0, dilation=1, ceil_mode=False)
    (3): Conv2d(64, 128, kernel_size=(3, 3), stride=(1, 1), padding=(1, 1))
    (4): ReLU(inplace=True)
    (5): MaxPool2d(kernel_size=2, stride=2, padding=0, dilation=1, ceil_mode=False)
    (6): Conv2d(128, 256, kernel_size=(3, 3), stride=(1, 1), padding=(1, 1))
    (7): ReLU(inplace=True)
    (8): Conv2d(256, 256, kernel_size=(3, 3), stride=(1, 1), padding=(1, 1))
    (9): ReLU(inplace=True)
    (10): MaxPool2d(kernel_size=2, stride=2, padding=0, dilation=1, ceil_mode=False)
    (11): Conv2d(256, 512, kernel_size=(3, 3), stride=(1, 1), padding=(1, 1))
    (12): ReLU(inplace=True)
    (13): Conv2d(512, 512, kernel_size=(3, 3), stride=(1, 1), padding=(1, 1))
    (14): ReLU(inplace=True)
    (15): MaxPool2d(kernel_size=2, stride=2, padding=0, dilation=1, ceil_mode=False)
    (16): Conv2d(512, 512, kernel_size=(3, 3), stride=(1, 1), padding=(1, 1))
    (17): ReLU(inplace=True)
    (18): Conv2d(512, 512, kernel_size=(3, 3), stride=(1, 1), padding=(1, 1))
    (19): ReLU(inplace=True)
    (20): MaxPool2d(kernel_size=2, stride=2, padding=0, dilation=1, ceil_mode=False)
  )
```

Back propagation

- **Chain rule:**
  - $y = f(x), z = g(y),\ \ z = g(f(x)) = (g \circ f)(x)$

$$(g \circ f)'(x) = \frac{dg}{dx} = \frac{dg}{df}\frac{df}{dx}$$

  - $z = f(x, y)$, where $x = g(t), y = h(t)$

$$\frac{df}{dt} = \frac{\partial f}{\partial g}\frac{dg}{dt} + \frac{\partial f}{\partial h}\frac{dh}{dt}$$

- **Loss function:**

$$L = \frac{1}{2n}\sum_{i=1}^{n}(\hat{y} - y)^2$$

$$L' = \frac{1}{n}\sum_{i=1}^{n}(\hat{y} - y)$$

- **Sigmoid function:**

$$\sigma(x) = \frac{1}{1 + e^{-x}}$$

- **Derivative of sigmoid function:**

$$\begin{aligned}
\sigma(x)' &= \frac{d}{dx}(1 + e^{-x})^{-1} \\
&= -(1 + e^{-x})^{-2}(-e^{-x}) \\
&= \frac{e^{-x}}{(1 + e^{-x})^2} \\
&= \frac{1}{1 + e^{-x}}\frac{e^{-x}}{1 + e^{-x}} \\
&= \sigma(x)(1 - \sigma(x))
\end{aligned}$$

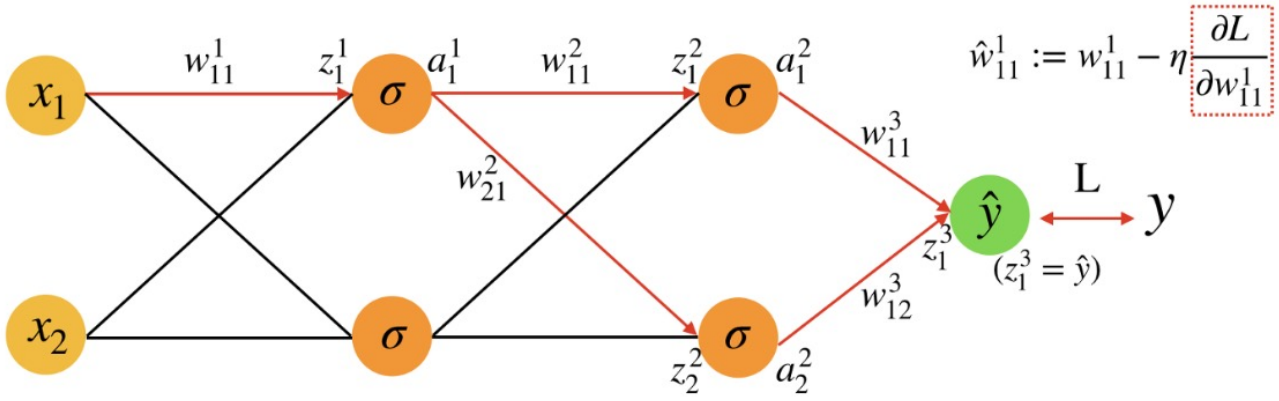


Fig.3

- The gradient of w_{11}^1 :

$$\frac{\partial L}{\partial w_{11}^1} = \frac{\partial L}{\partial z_1^3}[\sum_{i=1}^{2} \frac{\partial z_1^3}{\partial a_i^2}\frac{\partial a_i^2}{\partial z_i^2}\frac{\partial z_i^2}{\partial a_1^1}]\frac{\partial a_1^1}{\partial z_1^1}\frac{\partial z_1^1}{\partial w_{11}^1} = \frac{1}{n}(\hat{y}-y)[\sum_{i}^{2} w_{1i}^3\sigma'(z_i^2)w_{i1}^2]\sigma'(z_1^1)x_1$$

1. **2.** **3.**

1. $\frac{\partial L}{\partial z_1^3} = \frac{\partial}{\partial \hat{y}}\frac{1}{2n}(\hat{y}-y)^2 = \frac{1}{n}(\hat{y}-y),\ (z_1^3 = \hat{y})$

2. $\frac{\partial z_1^3}{\partial a_1^2} = \frac{\partial}{\partial a_1^2}(a_1^2 w_{11}^3 + a_2^2 w_{12}^3) = w_{11}^3,\ (z_1^3 = a_1^2 w_{11}^3 + a_2^2 w_{12}^3)$

3. $\frac{\partial a_1^2}{\partial z_1^2} = \sigma'(z_1^2) = \sigma(z_1^2)(1-\sigma(z_1^2)),\ (a_1^2 = \sigma(z_1^2))$

And so on…

$$\frac{\partial L}{\partial w_{21}^1} = \frac{1}{n}(\hat{y}-y)[\sum_{i}^{2} w_{1i}^3\sigma'(z_i^2)w_{i2}^2]\sigma'(z_2^1)x_1$$

$$\frac{\partial L}{\partial w_{12}^1} = \frac{1}{n}(\hat{y}-y)[\sum_{i}^{2} w_{1i}^3\sigma'(z_i^2)w_{i1}^2]\sigma'(z_1^1)x_2$$

$$\frac{\partial L}{\partial w_{21}^1} = \frac{1}{n}(\hat{y}-y)[\sum_{i}^{2} w_{1i}^3\sigma'(z_i^2)w_{i2}^2]\sigma'(z_2^1)x_2$$

$$\frac{\partial L}{\partial w_{11}^2} = \frac{\partial L}{\partial z_1^3}\frac{\partial z_1^3}{\partial a_1^2}\frac{\partial a_1^2}{\partial z_1^2}\frac{\partial z_1^2}{\partial w_{11}^2} = \frac{1}{n}(\hat{y}-y)w_{11}^3\sigma'(z_1^2)a_1^1$$

$$\frac{\partial L}{\partial w_{21}^2} = \frac{1}{n}(\hat{y}-y)w_{12}^3\sigma'(z_2^2)a_1^1$$

$$\frac{\partial L}{\partial w_{12}^2} = \frac{1}{n}(\hat{y}-y)w_{11}^3\sigma'(z_1^2)a_2^1$$

$$\frac{\partial L}{\partial w_{22}^2} = \frac{1}{n}(\hat{y}-y)w_{12}^3\sigma'(z_2^2)a_2^1$$

$$\frac{\partial L}{\partial w_{11}^3} = \frac{\partial L}{\partial z_1^3}\frac{\partial z_1^3}{\partial w_{11}^3} = \frac{1}{n}(\hat{y}-y)a_1^2 \qquad \frac{\partial L}{\partial w_{12}^3} = \frac{1}{n}(\hat{y}-y)a_2^2$$

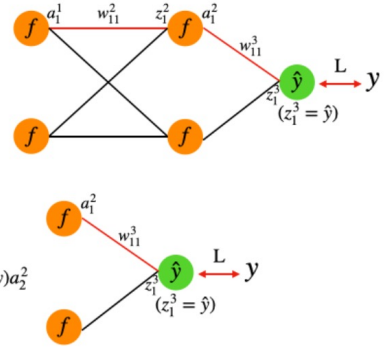
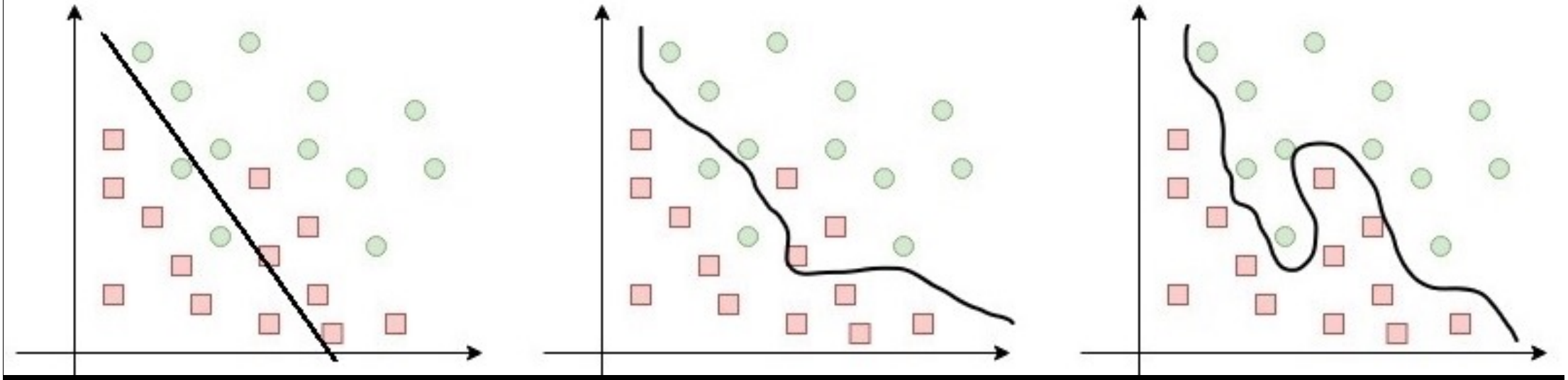


Fig.5

Control Overfitting by Regulization



$y = w_0 + w_1 x^1$

$y = w_0 + w_1 x^1 + w_2 x^2 + w_3 x^3$

$y = w_0 + w_1 x^1 + w_3 x^3 + w_4 x^4 \ldots\ldots$

$$L1\ Norm: \sum_{n}^{i} |w_i| = ||w_i||$$

$$L2\ Norm: \sum_{i}^{} |w_i|^2 = ||w_i||^2$$

Different ways of regulization

	weights	*penality*
$L1\ Norm: \sum_i^n \lvert w_i \rvert$ $= \lvert\lvert w_i \rvert\rvert$	$-1, 2, 3, 0, 1, -2$	1, 2, 3, 0, 1, 2 = 9
$L2\ Norm: \sum_i^n \lvert w_i \rvert^2$ $= \lvert\lvert w_i \rvert\rvert^2$		1, 4, 9, 0, 1, 4 = 19